

GIÁO TRÌNH TỰ HỌC THỰC CHIẾN TOÀN DIỆN

DATA MASTERY ALL-IN-ONE

Lộ Trình Từ Zero Đến Intern, Junior & Middle

Data Analyst – Data Scientist – Data Engineer

Cẩm nang thực chiến chuyên sâu với 11 Flagship Projects, ứng dụng AI Copilot hiện đại, hệ thống bài tập tự luyện có lời giải chi tiết, và bộ công thức chuẩn hóa CV chinh phục mọi nhà tuyển dụng.

TÁC GIẢ & ĐỘI NGŨ BIÊN SOẠN

Antigravity DeepMind Engineering Team & Data Career Mentors

Cập nhật xu hướng tuyển dụng mới nhất 2026

Năm Xuất Bản: 2026

Contents

1	Cẩm Nang Thiết Lập Môi Trường & Công Cụ Từ Con Số 0	7
1.1	Terminal Là Gì? Tại Sao Kỹ Sư Dữ Liệu Không Thể Chỉ Bấm Chuột?	7
1.2	Cài Đặt Bộ Công Cụ Lập Trình Chuẩn Doanh Nghiệp	8
1.2.1	1. Visual Studio Code (VS Code)	8
1.2.2	2. Cài Đặt Python 3.11/3.12 & Thiết Lập Môi Trường Ảo (venv)	8
1.2.3	3. Cài Đặt DBeaver (Trình Quản Trị Cơ Sở Dữ Liệu Miễn Phí)	9
1.3	Sinh Toàn Bộ Dữ Liệu Mẫu Cho 11 Projects Chỉ Bằng 1 Dòng Lệnh	9
1.4	Chạy File Python Đầu Tiên & Kiểm Tra Môi Trường	9
2	Bản Đồ Nghề Nghiệp Dữ Liệu & Phương Pháp Học Thực Chiến	11
2.1	Toàn Cảnh Ngành Dữ Liệu: Phân Định Rạch Ròi DA, DS, DE	11
2.2	Chiến Lược: Core Data → DA Foundation → DS/DE Specialization	12
2.3	Lợi Thế Độc Quyền Của Nền Tảng Toán - Tin	13
2.4	Phương Pháp Học Mới: Làm Chủ AI Copilot Trong Ngành Data	13
2.5	Tiêu Chuẩn Đầu Ra & Quản Lý GitHub Portfolio	14
2.6	Bài Tập Tự Luyện & Định Hướng	14
3	Level 0: Nền Tảng Dữ Liệu, Git & Linux CLI (Tuần 1–2)	16
3.1	Bản Chất Dữ Liệu & Các Định Dạng File Doanh Nghiệp	16
3.1.1	So Sánh Thực Chiến: CSV vs JSON vs Parquet vs Excel	17
3.2	Git & GitHub Thực Chiến Cho Kỹ Sư Dữ Liệu	17
3.3	Linux CLI & Xử Lý Dữ Liệu Nhanh Bằng Dòng Lệnh	18
3.3.1	1. Lệnh Điều Hướng Hệ Thống Tập Tin Từ Con Số 0	18
3.3.2	2. Thao Tác & Lọc Dữ Liệu Lớn Siêu Tốc	19
3.4	Bí Kíp AI Copilot: Sinh Dữ Liệu Giả Lập & Scripting	20

3.5	PROJECT 0: Xây Dựng Personal Data CLI Analyzer	20
3.5.1	Bài Toán Thực Tế	20
3.5.2	Mã Nguồn Thực Hiện Hoàn Chỉnh	20
3.6	Bài Tập Tự Luyện Level 0	22
4	Level 1: Lập Trình Python Từ Con Số 0 & Phân Tích Khám Phá (EDA) (Tuần 3–8)	24
4.1	Python Căn Bản Từ Con Số 0	24
4.1.1	1. Biến (Variables) & 4 Kiểu Dữ Liệu Nguyên Thủy	24
4.1.2	2. Các Toán Tử Toán Học & So Sánh	25
4.1.3	3. Cấu Trúc Rẽ Nhánh Điều Kiện (if – elif – else)	25
4.1.4	4. Vòng Lặp (for & while)	26
4.1.5	5. Bốn Cấu Trúc Dữ Liệu Cốt Lõi: List, Dict, Tuple, Set	26
4.1.6	6. Hàm (Functions) & Xử Lý Lỗi (try – except)	27
4.2	Nâng Cấp Kỹ Năng Pythonic & Tối Ưu Hiệu Năng	28
4.2.1	Comprehensions & Generators Tiết Kiệm Bộ Nhớ	28
4.3	NumPy: Sức Mạnh Tính Toán Vector (Vectorization)	29
4.4	Pandas Masterclass: Trái Tim Của Xử Lý Dữ Liệu	29
4.4.1	1. Đọc Dữ Liệu, Lọc & Named Aggregations	29
4.4.2	2. Kết Nối Bảng (Merge / Join) & Phép Gộp Nhóm (GroupBy)	30
4.5	PROJECT 1: E-Commerce Sales Explorer	30
4.5.1	Kịch Bản Thực Tế	30
4.6	Bài Tập Tự Luyện Level 1	32
5	Level 2: SQL & Cơ Sở Dữ Liệu Quan Hệ Từ Con Số 0 (Tuần 9–14)	34
5.1	Cơ Sở Dữ Liệu Quan Hệ (RDBMS) Từ Con Số 0	34
5.1.1	1. Tại Sao Không Dùng Excel Để Lưu Trữ Dữ Liệu Công Ty?	35
5.1.2	2. Bảng, Cột, Khóa Chính & Khóa Ngoại	35
5.2	Ngôn Ngữ SQL Căn Bản Từ Con Số 0	35
5.2.1	1. Tạo Bảng Với DDL (Data Definition Language)	35
5.2.2	2. Thêm, Sửa, Xóa Dữ Liệu Với DML (INSERT, UPDATE, DELETE)	36
5.2.3	3. Truy Vấn Dữ Liệu Cơ Bản (SELECT & WHERE)	36
5.2.4	4. Sắp Xếp, Giới Hạn & Khử Trùng Lặp (ORDER BY, LIMIT, DISTINCT)	37
5.2.5	5. Các Hàm Tổng Hợp (Aggregates) & Phép Gom Nhóm (GROUP BY, HAVING)	37

5.3	SQL Nâng Cao: JOINS, CTEs & Window Functions	38
5.3.1	1. Các Loại JOIN & Cạm Bẫy Fan-Out Trùng Lặp Dữ Liệu	38
5.3.2	2. CTE (Common Table Expressions) – Tư Duy Module Hóa SQL	38
5.3.3	3. Window Functions: ROW_NUMBER, RANK, DENSE_RANK	39
5.4	PROJECT 2: E-Commerce Analytics Database	39
5.4.1	Truy Vấn Phân Hạng Khách Hàng (RFM Segmentation)	40
5.5	Bài Tập Tự Luyện Level 2	41
6	Level 3: Thống Kê Kinh Doanh & Thử Nghiệm A/B Testing (Tuần 15–18)	43
6.1	Thống Kê Mô Tả Từ Con Số 0	43
6.1.1	1. Phân Loại Dữ Liệu Trong Thống Kê	43
6.1.2	2. Mean, Median, Mode & Ví Dụ Tính Bằng Tay	44
6.1.3	3. Phương Sai (Variance) & Độ Lệch Chuẩn (Standard Deviation)	44
6.2	Xác Suất & Định Lý Giới Hạn Trung Tâm (CLT)	45
6.3	Thống Kê Suy Luận & Kiểm Định Giả Thuyết (Hypothesis Testing)	45
6.3.1	Sai Lầm Loại I (α), Loại II (β) & Ý Nghĩa p-value	45
6.4	A/B Testing Thực Chiến: Sample Size & Cạm Bẫy SRM	46
6.5	PROJECT 3: Thử Nghiệm A/B Testing Tỷ Lệ Chuyển Đổi E-Commerce	46
6.5.1	Kịch Bản Nghiệp Vụ & Triển Khai Phân Tích	46
6.5.2	Mã Nguồn Kiểm Định Thống Kê Bằng Python (SciPy & Statsmodels)	46
6.6	Bài Tập Tự Luyện Level 3	48
7	Level 4: Data Analyst & Business Intelligence Chuyên Nghiệp (Tuần 19–24)	50
7.1	Excel Nâng Cao: Tự Động Hóa Với Power Query & Dynamic Arrays	50
7.2	Power BI & Nghệ Thuật Mô Hình Hóa Dữ Liệu (Data Modeling)	50
7.3	Ngôn Ngữ DAX Chuyên Sâu: Filter Context & Time Intelligence	51
7.3.1	Các Công Thức DAX Kinh Điển Chuẩn Doanh Nghiệp	51
7.4	Tư Duy Thiết Kế Dashboard: KPI Architecture Chuẩn Doanh Nghiệp	52
7.5	Bí Kíp AI Copilot: Sinh Công Thức DAX & Tạo Data Dictionary	53
7.6	MỐC ỨNG TUYỂN 1: Bắt Đầu Apply Data Analyst Intern / Fresher!	53
7.7	Bài Tập Tự Luyện Level 4	54

8	Level 5: Data Engineering – Data Modeling & Core ETL (Tuần 25–30)	56
8.1	Nền Tảng Data Warehousing: So Sánh Toàn Diện OLTP vs OLAP	56
8.2	Mô Hình Hóa Chiều Kimball (Kimball Dimensional Modeling)	56
8.2.1	Xử Lý Chiều Biến Đổi Chậm (Slowly Changing Dimensions - SCD)	57
8.3	Kiến Trúc ETL Hiện Đại & Tính Lũy Đẳng (Idempotency)	57
8.4	PROJECT 5: Automated Batch ETL Pipeline Chuẩn Production	58
8.4.1	Bài Toán Thực Tế	58
8.4.2	Mã Nguồn Pipeline Hoàn Chỉnh	58
8.5	Bài Tập Tự Luyện Level 5	60
9	Level 6: DE Professional – Docker, Apache Airflow & dbt (Tuần 31–36)	62
9.1	Docker & Containerization Cho Kỹ Sư Dữ Liệu	62
9.2	Apache Airflow: Trái Tim Điều Phối Hệ Thống (Orchestrator)	63
9.2.1	Mã Nguồn Một Airflow DAG Chuẩn Doanh Nghiệp (TaskFlow API)	63
9.3	dbt (Data Build Tool): Tái Định Nghĩa Analytics Engineering	65
9.3.1	Cấu Trúc Mô Hình Đa Tầng Chuẩn Mực Trong dbt	65
9.4	PROJECT 6: Production Analytics Engineering Pipeline	66
9.5	Bài Tập Tự Luyện Level 6	66
10	Level 7: Machine Learning & Predictive Analytics Thực Chiến (Tuần 37–44)	68
10.1	Machine Learning Căn Bản Từ Con Số 0	68
10.1.1	1. Lập Trình Truyền Thống vs Machine Learning	68
10.1.2	2. Ma Trận Đặc Trưng (X) & Vector Mục Tiêu (y)	69
10.1.3	3. Tại Sao Bắt Buộc Phải Chia Tập Train & Test?	69
10.2	Feature Engineering & Nghệ Thuật Ngăn Chặn Data Leakage	69
10.3	Vượt Qua Ảo Tưởng Accuracy: Ma Trận Đánh Giá Thực Tế	70
10.4	PROJECT 7: End-to-End Customer Churn Prediction System	70
10.4.1	Bài Toán Thực Tế	70
10.4.2	Mã Nguồn Pipeline Scikit-Learn Chuẩn Doanh Nghiệp	70
10.5	Khả Năng Diễn Giải Mô Hình Với SHAP (SHapley Values)	72
10.6	Bài Tập Tự Luyện Level 7	72

11 Level 8: Big Data & Lakehouse Architecture Với PySpark (Tuần 45–50)	75
11.1 Khi Nào Cần Big Data & Giới Hạn Của Pandas	75
11.2 Kiến Trúc Apache Spark Chuyên Sâu	75
11.3 PySpark Thực Chiến & Kỹ Thuật Broadcast Join	76
11.4 Kiến Trúc Lakehouse Hiện Đại Với Delta Lake	77
11.5 PROJECT 8: Big Data Transformation Engine (10+ Triệu Dòng)	77
11.6 Bài Tập Tự Luyện Level 8	78
12 Level 9: Nền Tảng Cloud Cho Data – AWS & GCP (Tuần 51–54)	80
12.1 Lựa Chọn Nền Tảng Đám Mây: AWS vs Google Cloud (GCP)	80
12.2 Kiến Trúc Pipeline Không Máy Chủ (Serverless Event-Driven)	80
12.3 Bảo Mật & Nguyên Tắc Phân Quyền Tối Thiểu (Least Privilege)	82
12.4 Quản Trị Chi Phí (FinOps Cho Kỹ Sư Dữ Liệu)	82
12.5 PROJECT 9: Serverless Cloud Data Platform	82
12.6 Bài Tập Tự Luyện Level 9	83
13 Level 10: MLOps & Đưa Mô Hình Lên Production (Tuần 55–58)	85
13.1 Khoảng Trống Giữa Jupyter Notebook & Hệ Thống Sản Xuất	85
13.2 MLflow: Quản Lý Vòng Đời Thử Nghiệm & Model Registry	85
13.3 Phục Vụ Mô Hình Thời Gian Thực Với FastAPI	86
13.4 Đóng Gói Docker & CI/CD Với GitHub Actions	88
13.5 Giám Sát Mô Hình: Data Drift & Concept Drift	89
13.6 PROJECT 10: Production ML Churn Prediction Microservice	89
13.7 Bài Tập Tự Luyện Level 10	90
14 Đồ Án Flagship Capstone: Modern Enterprise Data & AI Platform (Project 11)	92
14.1 Tầm Nhìn Của Dự Án Flagship Capstone	92
14.2 Kiến Trúc Kỹ Thuật Toàn Cảnh (End-to-End Architecture)	92
14.3 Cấu Trúc Repository Chuẩn Doanh Nghiệp (Enterprise Monorepo)	93
14.4 Kịch Bản 15 Phút Trình Bày Dự Án Chinh Phục Nhà Tuyển Dụng	94

15 Chiến Lược Nghề Nghiệp, Bộ CV Chuẩn ATS & 40 Câu Hỏi Phỏng Vấn Sát Thủ	95
15.1 Chân Dung Năng Lực Tuyển Dụng: Intern vs Junior vs Middle	95
15.2 Công Thức Vàng Viết Bullet Point CV Chuẩn ATS	96
15.3 Bộ 3 Mẫu CV Chuẩn ATS (Sẵn Sàng Ứng Tuyển)	96
15.3.1 Mẫu 1: Data Analyst Intern / Fresher CV	96
15.3.2 Mẫu 2: Data Engineer Intern / Junior CV	97
15.3.3 Mẫu 3: Data Scientist Intern / Junior CV	98
15.4 Ngân Hàng 40 Câu Hỏi Phỏng Vấn "Sát Thủ" Kèm Lời Giải	99
15.4.1 10 Câu Hỏi Cốt Lõi Về SQL & Cơ Sở Dữ Liệu	99
15.4.2 10 Câu Hỏi Về Python, Pandas & Kỹ Thuật Lập Trình	100
15.4.3 10 Câu Hỏi Về Thống Kê, A/B Testing & Machine Learning	101
15.4.4 10 Câu Hỏi Về Data Engineering, System Design & Hạ Tầng	102
16 Workbook Phần 1: 20 Bài Tập Thực Hành Excel & Báo Cáo Nghiệp Vụ	104
17 Workbook Phần 2: 20 Bài Tập Thực Hành Lập Trình Python & Pandas	108
18 Workbook Phần 3: 20 Bài Tập Thực Hành SQL & Cơ Sở Dữ Liệu Nâng Cao	112
19 Workbook Phần 4: 20 Bài Tập Xác Suất, Thống Kê & A/B Testing	117
20 Workbook Phần 5: 20 Tình Huống Phân Tích Nghiệp Vụ & Đo Lường Hiệu Quả (Business Analytics & Metrics)	121
21 Workbook Phần 6: 20 Bài Tập Thực Hành Data Engineering, Big Data & MLOps	127
22 Phụ Lục: Checklist Năng Lực & Tổng Kết Hành Trình Data Pro	135
22.1 Checklist 35 Tiêu Chí Đánh Giá Năng Lực Trước Khi Ứng Tuyển	135
22.2 Lời Kết: Tinh Thần Của Một Kỹ Sư Dữ Liệu Chân Chính	137

Chương 1

Cẩm Nang Thiết Lập Môi Trường & Công Cụ Từ Con Số 0

Kiến Thức Cốt Lõi: Mục Tiêu & Tiền Đề Cần Nhớ

Tiền đề bắt buộc: HOÀN TOÀN KHÔNG CÓ! Bạn chỉ cần một chiếc máy tính cá nhân (Windows, macOS hoặc Linux) có kết nối Internet.

Mục tiêu chương này:

- Hiểu rõ Terminal/Dòng lệnh (CLI) là gì và tại sao kỹ sư dữ liệu bắt buộc phải dùng CLI.
- Cài đặt và cấu hình Visual Studio Code (VS Code) thành môi trường làm việc Data chuyên nghiệp.
- Cài đặt Python 3.11/3.12, tạo và quản lý môi trường ảo cô lập (venv).
- Cài đặt Git, liên kết GitHub và cấu hình DBeaver (phần mềm GUI quản trị SQL miễn phí).
- Tự tay chạy chương trình Python đầu tiên và sinh bộ dữ liệu mẫu 11 projects chỉ bằng 1 dòng lệnh.

1.1 Terminal Là Gì? Tại Sao Kỹ Sư Dữ Liệu Không Thể Chỉ Bấm Chuột?

Người dùng máy tính phổ thông quen thao tác bằng giao diện đồ họa (GUI - Graphical User Interface): nhấp đúp chuột để mở thư mục, kéo thả file. Tuy nhiên, trong thế giới dữ liệu:

- Các máy chủ đám mây (AWS EC2, Google Cloud, Docker Containers) **hoàn toàn không có màn hình hay chuột**, chỉ có giao diện dòng lệnh văn bản qua mạng (SSH).
- Thao tác chuột không thể tự động hóa. Một dòng lệnh có thể xử lý 10,000 tệp tin trong 3 giây, trong khi nhấp chuột sẽ mất cả tuần.
- **Terminal / Command Prompt / Shell**: Là cửa sổ nơi bạn gõ các lệnh bằng chữ để yêu cầu hệ điều hành thực thi trực tiếp.

1.2 Cài Đặt Bộ Công Cụ Lập Trình Chuẩn Doanh Nghiệp

1.2.1 1. Visual Studio Code (VS Code)

Tải và cài đặt phiên bản mới nhất từ trang chủ <https://code.visualstudio.com/>. Sau khi cài đặt, mở mục **Extensions** (phím tắt **Ctrl + Shift + X** trên Windows/Linux hoặc **Cmd + Shift + X** trên Mac) và cài đặt 5 tiện ích mở rộng bắt buộc sau:

1. **Python** (bởi Microsoft): Hỗ trợ gợi ý code thông minh, highlight cú pháp.
2. **Pylance**: Động cơ phân tích kiểu dữ liệu và kiểm tra lỗi thời gian thực.
3. **SQLTools**: Truy vấn cơ sở dữ liệu trực tiếp trong VS Code.
4. **Docker**: Quản lý container và image trực quan.
5. **GitLens**: Soi chiếu lịch sử thay đổi code từng dòng.

1.2.2 2. Cài Đặt Python 3.11/3.12 & Thiết Lập Môi Trường Ảo (venv)

Thực Chiến & Best Practices: Quy Tắc Sống Còn: Luôn Sử Dụng Virtual Environment

Sai lầm lớn nhất của người mới: Mở terminal và gõ `pip install pandas`, cài thẳng mọi thư viện vào Python gốc của hệ điều hành. Sau 3 tháng, các dự án khác nhau yêu cầu các phiên bản thư viện xung đột lẫn nhau làm hỏng toàn bộ môi trường Python của máy tính!

Quy trình chuẩn mực bắt buộc: Mỗi dự án phải có một "chiếc hộp cách ly" riêng mang tên Virtual Environment (`.venv`):

```

1 # 1. Di chuyển vào thư mục dự án
2 cd ~/Data_Handbook_2026
3
4 # 2. Tạo môi trường ảo có lap ten la .venv
5 python3 -m venv .venv
6
7 # 3. Kích hoạt môi trường ảo
8 # Tren Linux / macOS:
9 source .venv/bin/activate
10 # Tren Windows (PowerShell):
11 # .venv\Scripts\Activate.ps1
12
13 # Khi thay dấu (.venv) xuất hiện ở đầu dòng lệnh là đã kích hoạt
    thành công!
14 # 4. Cài đặt toàn bộ thư viện cần thiết
15 pip install --upgrade pip
16 pip install pandas numpy scikit-learn matplotlib seaborn
    psycpg2-binary fastapi uvicorn pydantic requests
  
```

1.2.3 3. Cài Đặt DBeaver (Trình Quản Trị Cơ Sở Dữ Liệu Miễn Phí)

Tải bản Community Edition tại <https://dbeaver.io/>. DBeaver hỗ trợ kết nối tới mọi hệ quản trị cơ sở dữ liệu (PostgreSQL, MySQL, SQLite, DuckDB, ClickHouse) với giao diện trực quan, cho phép người mới xem trực tiếp cấu trúc bảng, khóa chính, và dữ liệu dòng trước khi viết câu lệnh SQL.

1.3 Sinh Toàn Bộ Dữ Liệu Mẫu Cho 11 Projects Chỉ Bằng 1 Dòng Lệnh

Để đảm bảo bạn có thể thực hành ngay lập tức 100% các ví dụ và dự án trong cuốn sách này mà không bao giờ gặp lỗi thiếu file (`FileNotFoundException`), cuốn sách đã chuẩn bị sẵn một script tự động sinh dữ liệu:

Thực Chiến & Best Practices: Khởi Tạo Dữ Liệu Thực Hành Toàn Diện

Mở Terminal trong thư mục dự án và thực thi lệnh sau:

```
1 python scripts/generate_all_datasets.py
```

Script sẽ tự động tạo thư mục `data/` và sinh ra đầy đủ 8 tệp dữ liệu chuẩn xác khớp từng tên cột và kiểu dữ liệu:

- `data/input.csv`: Dữ liệu cho Project 0 (CLI Profiler).
- `data/customers.csv`, `orders.csv`, `order_items.csv`, `payments.csv`: Bộ dữ liệu E-commerce đa bảng cho Project 1, 2 và 4.
- `data/ab_test_checkout.csv`: 50,000 phiên thử nghiệm A/B Testing cho Project 3.
- `data/customer_churn_features.csv`: 10,000 hồ sơ khách hàng cho Project 7 và 10.
- `data/raw_transactions.csv`: 10,000 giao dịch có chứa lỗi để kiểm thử Data Pipeline Project 5.

1.4 Chạy File Python Đầu Tiên & Kiểm Tra Môi Trường

Tạo một tệp tin mang tên `test_env.py` và dán đoạn mã sau:

```
1 import sys
2 import pandas as pd
3 import numpy as np
4
5 print("=" * 50)
6 print("[V] CHÚC MỪNG! BẠN ĐÃ THIẾT LẬP MÔI TRƯỜNG THÀNH CÔNG!")
7 print(f"Phiên bản Python: {sys.version.split()[0]}")
8 print(f"Phiên bản Pandas: {pd.__version__}")
9 print(f"Phiên bản NumPy : {np.__version__}")
10 print("=" * 50)
```

Chạy file bằng lệnh: `python test_env.py`. Nếu bạn nhìn thấy thông báo chúc mừng, bạn đã hoàn toàn sẵn sàng bước vào hành trình chinh phục Dữ liệu từ con số 0!

Chương 2

Bản Đồ Nghề Nghiệp Dữ Liệu & Phương Pháp Học Thực Chiến

2.1 Toàn Cảnh Ngành Dữ Liệu: Phân Định Rạch Ròi DA, DS, DE

Trong kỷ nguyên dữ liệu bùng nổ và sự trỗi dậy của Trí tuệ Nhân tạo (AI), thị trường tuyển dụng không còn tìm kiếm những "thợ gõ code" biết mọi thứ một cách nửa vời. Các doanh nghiệp hiện đại từ các tập đoàn Fintech, Thương mại điện tử (Shopee, Tiki, Grab) đến các ngân hàng lớn (MB, Techcombank, VPBank) đều tái cấu trúc bộ máy dữ liệu thành ba trụ cột chuyên biệt hóa cao độ: ****Data Analyst (DA)****, ****Data Scientist (DS)****, và ****Data Engineer (DE)****.

Kiến Thức Cốt Lõi: Bản Chất và Trách Nhiệm 3 Vị Trí Cốt Lõi

• Data Analyst (Kỹ sư Phân tích Dữ liệu):

- **Bản chất:** Cầu nối giữa dữ liệu kỹ thuật và quyết định kinh doanh. Trả lời câu hỏi: **"Chuyện gì đã xảy ra? (Descriptive) và Tại sao lại xảy ra? (Diagnostic)"**.
- **Đầu ra hàng ngày:** SQL queries trích xuất dữ liệu, Interactive BI Dashboards (Power BI, Tableau), Báo cáo A/B Testing, Phân tích chỉ số tăng trưởng (Retention, Churn, CAC, LTV).
- **KPI đo lường:** Tốc độ cung cấp báo cáo, độ chính xác của số liệu (Data accuracy), mức độ áp dụng của dashboard vào việc ra quyết định của các phòng ban (Business Adoption).

• Data Scientist (Nhà Khoa học Dữ liệu):

- **Bản chất:** Ứng dụng toán học, xác suất thống kê và học máy (Machine Learning) để giải quyết các bài toán tối ưu và dự báo. Trả lời câu hỏi: **"Điều gì sẽ xảy ra? (Predictive) và Chúng ta nên làm gì để đạt kết quả tốt nhất? (Prescriptive)"**.
- **Đầu ra hàng ngày:** Feature engineering pipelines, Mô hình ML phân loại/dự báo (Churn prediction, Fraud detection, Credit scoring, Recommendation system), Thiết kế thử nghiệm A/B Testing chặt chẽ.
- **KPI đo lường:** Độ chuẩn xác mô hình (AUC-ROC, F1-Score, RMSE), Giá trị kinh tế gia tăng từ quyết định tự động hóa (Revenue uplift, Risk cost reduction).

- **Data Engineer (Kỹ sư Hạ tầng Dữ liệu):**

- *Bản chất:* "Thợ xây" nền móng hạ tầng dữ liệu. Đảm bảo dữ liệu từ hàng trăm nguồn (APIs, CRM, ERP, Clickstream) được thu thập, làm sạch, mô hình hóa và sẵn sàng truy vấn với độ trễ thấp và độ tin cậy tuyệt đối.
- *Đầu ra hàng ngày:* Batch & Streaming Data Pipelines (Airflow, Kafka, Spark), Data Warehouse / Data Lakehouse schemas (Kimball Star Schema, Delta Lake, BigQuery, Snowflake), Data Quality checks (dbt tests, Great Expectations).
- *KPI đo lường:* Pipeline SLA (đúng giờ), Data Freshness (độ tươi mới), System Downtime (thời gian chết của hệ thống), Compute Cost Optimization (tiết kiệm chi phí cloud).

2.2 Chiến Lược: Core Data → DA Foundation → DS/DE Specialization

Sai lầm phổ biến nhất của người tự học từ con số 0 là cố gắng trở thành "Full-Stack Data Professional" ngay từ ngày đầu tiên: vừa học Kafka, Kubernetes, vừa học Deep Learning PyTorch, vừa vẽ dashboard Power BI. Kết quả là sau 6 tháng chỉ biết mỗi thứ một chút bề mặt, không làm được một dự án nào hoàn chỉnh và trượt ngay vòng phỏng vấn kỹ thuật đầu tiên.

Quy luật bất biến: Cả ba vị trí đều đứng chung trên một chiếc kiềng ba chân vững chắc mang tên ****Core Data****.

Kỹ Năng	Data Analyst (DA)	Data Scientist (DS)	Data Engineer (DE)
SQL	Siêu thành thạo: CTE, Window Functions, Aggregate	Rất thành thạo: Feature extraction, Complex Joins	Bậc thầy: Optimization, Partitioning, Execution Plan
Python	Xử lý số liệu: Pandas, NumPy, Data Cleaning	Chuyên sâu: Scikit-learn, Vectorization, Modeling	Kỹ thuật phần mềm: OOP, Logging, Testing, Airflow
Thống Kê	Mô tả, A/B Test cơ bản, Xu hướng	Suy luận sâu, Phân phối, Hypothesis, Multi-arm Bandit	Cơ bản: Data distribution check, Anomaly detection
Hạ Tầng	Data Modeling cơ bản (Star schema trong Power BI)	Docker, Model Serving API (FastAPI), MLflow	Đỉnh cao: Distributed Systems (Spark), Cloud, CI/CD, dbt

Lộ trình thông minh nhất:

- Giai đoạn 1 (Tháng 1–6):** Học toàn bộ phần cốt lõi Data + DA Foundation. Đạt đến mốc này, bạn đã đủ sức làm việc và có thể ****bắt đầu nộp đơn ứng tuyển Data Analyst Intern / Fresher****.
- Giai đoạn 2 (Tháng 7–12):** Rẽ nhánh chuyên sâu vào Data Engineering (Pipeline, Airflow, dbt, Spark) hoặc Data Science (Machine Learning, Evaluation, MLOps).
- Giai đoạn 3 (Tháng 13–15+):** Đồ án Capstone tích hợp toàn diện và tích lũy kinh nghiệm làm việc thực tế tại doanh nghiệp để tiến lên mức Junior và Middle.

2.3 Lợi Thế Độc Quyền Của Nền Tảng Toán - Tin

Nếu bạn xuất phát điểm từ chuyên ngành Toán - Tin (hoặc Toán ứng dụng, Khoa học máy tính), bạn đang nắm giữ một "kho báu" mà người chuyên ngành khác phải mất nhiều năm mới bù đắp được:

- **Từ Toán học sang Data Science:** Đại số tuyến tính là nền tảng của Ma trận và Vectorization trong NumPy/Scikit-learn; Xác suất thống kê là linh hồn của A/B Testing, Naive Bayes và Hypothesis Testing; Giải tích đa biến và Tối ưu hóa (Convex Optimization) là gốc rễ của Gradient Descent. Bạn không học vẹt công thức mà hiểu vì sao mô hình hội tụ.
- **Từ Tin học sang Data Engineering:** Cấu trúc dữ liệu và giải thuật (Complexity $O(n)$, Hash Table, B-Tree) giúp bạn hiểu tường tận cơ chế Indexing của cơ sở dữ liệu và Shuffle trong tính toán phân tán Spark. Kỹ năng tư duy module hóa giúp code ETL của bạn sạch sẽ, dễ bảo trì và chịu tải cao.

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Lưu Ý Sống Còn Cho Dân Toán - Tin

Tuyệt đối không biến CV thành CV học thuật lý thuyết thuần túy! Doanh nghiệp trả lương cho bạn để giải quyết bài toán kinh doanh, tạo ra doanh thu hoặc giảm chi phí, chứ không trả lương để bạn chứng minh một định lý. Trong CV và phỏng vấn, hãy dịch chuyển ngôn ngữ:

- Thay vì nói: *"Em hiểu rất sâu về thuật toán Singular Value Decomposition (SVD)"*.
- Hãy nói: *"Em đã áp dụng kỹ thuật ma trận phân rã SVD để xây dựng hệ thống gợi ý sản phẩm, giúp tăng 8.5% tỷ lệ nhấp chuột (CTR) và xử lý 100,000 người dùng với độ trễ dưới 50ms"*.

2.4 Phương Pháp Học Mới: Làm Chủ AI Copilot Trong Ngành Data

Năm 2026 đánh dấu bước ngoặt: Một Data Engineer hoặc Data Scientist biết tận dụng AI hiệu quả có năng suất gấp 3 đến 5 lần một kỹ sư làm thủ công. Chúng ta áp dụng phương pháp ****Vibe Coding Có Kiểm Soát (Controlled AI-Assisted Development)****:

Bí Kíp AI Copilot & Prompt Engineering: Khung Prompting 4 Bước Cho Data Tasks

Khi làm việc với các LLM (ChatGPT, Claude, Cursor), hãy luôn tuân thủ cấu trúc prompt chuẩn mực:

1. **Context (Bối cảnh):** Định vị vai trò (Senior Data Engineer / Principal Analyst) và quy mô hệ thống (PostgreSQL 16, 5 triệu bản ghi).
2. **Data Contract (Hợp đồng dữ liệu):** Cung cấp DDL schema chính xác, kiểu dữ liệu, các ràng buộc và 3 dòng dữ liệu mẫu (Mock data).
3. **Business Objective (Mục tiêu nghiệp vụ):** Nêu rõ câu hỏi kinh doanh cần trả lời hoặc yêu cầu kỹ thuật (Idempotent ETL, tính Churn rate 30 ngày, tối ưu execution plan).
4. **Constraints & Output Format (Ràng buộc):** Yêu cầu code có type hinting, logging, unit

test pytest, tuân thủ PEP8, không dùng nested loops, giải thích thời gian chạy $O(\cdot)$.

Thực Chiến & Best Practices: Quy Tắc 3 "KHÔNG" Khi Dùng AI

1. **Không bao giờ copy code mà không đọc hiểu từng dòng:** Nếu bạn không thể giải thích tại sao dòng code đó tồn tại, bạn sẽ trượt phỏng vấn kỹ thuật ngay lập tức.
2. **Không đưa dữ liệu nhạy cảm của công ty lên LLMs công cộng:** Luôn ẩn danh hóa (anonymize) dữ liệu hoặc dùng mock data.
3. **Không tin tưởng 100% các phép tính thống kê mà AI tự tính nhầm:** Luôn yêu cầu AI viết script Python/SQL để tính toán và tự bạn kiểm thử lại kết quả.

2.5 Tiêu Chuẩn Đầu Ra & Quản Lý GitHub Portfolio

Một GitHub Portfolio "ghi điểm" với nhà tuyển dụng quốc tế và Việt Nam cần đáp ứng các tiêu chí sau:

- **Không chỉ nộp Notebook:** Phải có cấu trúc thư mục chuẩn kỹ thuật phần mềm ('src/', 'tests/', 'config/', 'docker-compose.yml', 'README.md', 'requirements.txt').
- **README xuất sắc:** README là "gương mặt đại diện" của dự án. Phải chứa sơ đồ kiến trúc (Architecture Diagram), bài toán kinh doanh, công nghệ sử dụng, số liệu đo lường cụ thể và hướng dẫn 1 dòng lệnh để chạy thử (One-command run).

2.6 Bài Tập Tự Luyện & Định Hướng

Bài Tập Tự Luyện: Định Hình Kế Hoạch 15 Tháng Của Bản Thân

1. **Bài 1 (Trắc nghiệm tư duy):** Một doanh nghiệp gặp sự cố: Báo cáo doanh thu trên Power BI vào 8h sáng chạy quá chậm và dữ liệu bị lệch 5% so với hệ thống kế toán. Đây là vấn đề chính thuộc về trách nhiệm của role nào: DA, DS hay DE? Giải thích vì sao.
2. **Bài 2 (Thiết kế mục tiêu):** Dựa trên thế mạnh của bạn (Toán - Tin), hãy chọn mốc ứng tuyển đầu tiên của bạn: Bạn sẽ nhắm tới DA Intern vào tuần 24, hay nhắm tới DE/DS Intern vào tuần 36? Hãy liệt kê 3 dự án cốt lõi bạn bắt buộc phải hoàn thành trước ngày nộp hồ sơ.
3. **Bài 3 (Thực hành AI Prompting):** Hãy viết một prompt hoàn chỉnh gửi tới ChatGPT/Claude để yêu cầu sinh ra một bộ dữ liệu giả lập (mock data) gồm 1,000 dòng giao dịch bán lẻ E-commerce dạng CSV, có cố tình chèn 3% giá trị null ở cột 'customer_age' và 1% bản ghi bị trùng lặp mã đơn hàng ('order_id').

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- **Bài 1:** Đây là sự phối hợp giữa **Data Engineer (DE)** và **Data Analyst (DA)**. Trong đó, DE chịu trách nhiệm chính về mặt kỹ thuật: kiểm tra đường ống ETL tải dữ liệu đêm qua có bị lỗi/chạy muộn không (Pipeline SLA), dữ liệu kế toán trích xuất có bị thiếu sót do logic trích xuất không (Data extraction bug), và tối ưu hóa thời gian tính toán của Data Warehouse. DA phối hợp kiểm tra định nghĩa công thức tính doanh thu (KPI definition) trên Power BI để xác minh xem có sự sai khác về logic phân loại đơn hàng (ví dụ: đã trừ đơn hủy/trả hàng chưa).
- **Bài 2:** Với nền tảng Toán - Tin, lộ trình khuyến nghị là nhắm tới **DA Intern ở tuần 24** để sớm bước chân vào doanh nghiệp tiếp xúc dữ liệu thật (với 3 project: E-commerce EDA, E-commerce Analytics SQL Database, Power BI Dashboard). Sau đó nâng cấp lên **DE hoặc DS Junior ở tuần 36–44**.
- **Bài 3:** Prompt mẫu chuẩn: *"Đóng vai trò là Senior Data Engineer. Hãy viết một đoạn script Python sử dụng thư viện Faker và Pandas để tạo ra 1,000 dòng dữ liệu giao dịch bán lẻ E-commerce với các cột: order_id (str), customer_id (str), order_date (YYYY-MM-DD), amount (float), customer_age (int), payment_method (str). Yêu cầu kỹ thuật: Tự động đưa vào chính xác 3% giá trị NaN ở cột customer_age và nhân bản (duplicate) ngẫu nhiên 10 dòng order_id. Xuất kết quả ra file ecom_mock.csv và in kèm thống kê số dòng null và duplicate để kiểm chứng."*

Chương 3

Level 0: Nền Tảng Dữ Liệu, Git & Linux CLI (Tuần 1–2)

Kiến Thức Cốt Lõi: Mục Tiêu & Tiền Đề Cần Nắm

Tiền đề bắt buộc: Bạn chỉ cần đã hoàn thành Chương 0 (đã cài đặt VS Code và mở được cửa sổ Terminal). **Chưa cần biết gì về hệ điều hành Linux hay Git trước đó!**

Mục tiêu chương này:

- Hiểu từ con số 0 các loại dữ liệu: Có cấu trúc, Bán cấu trúc, Phi cấu trúc.
- Hiểu cơ chế lưu trữ dạng dòng (CSV) vs dạng cột (Parquet) và vì sao Parquet lại giúp tiết kiệm 80% dung lượng ổ cứng.
- Làm chủ các câu lệnh Linux CLI căn bản nhất: `pwd`, `ls`, `cd`, `mkdir`, `cat`, `grep`, `awk`, `cut`, `sort`, `uniq`.
- Hiểu cách Git hoạt động từ con số 0: Vùng làm việc (Working Tree), Vùng chờ (Staging Area), Kho lưu trữ (Repository), và quy tắc bảo mật với `.gitignore`.
- Tự tay viết và chạy công cụ dòng lệnh đầu tiên (**Project 0: Personal Data CLI Analyzer**) phân tích dữ liệu tự động.

3.1 Bản Chất Dữ Liệu & Các Định Dạng File Doanh Nghiệp

Trước khi viết bất kỳ dòng mã xử lý dữ liệu nào, bạn phải hiểu rõ bản chất của dữ liệu và định dạng vật lý lưu trữ nó trên ổ đĩa. Trong môi trường doanh nghiệp, dữ liệu tồn tại ở ba dạng chính:

- **Dữ liệu có cấu trúc (Structured Data):** Có schema định nghĩa chặt chẽ (tên cột, kiểu dữ liệu, ràng buộc). Ví dụ: Bảng trong PostgreSQL, MySQL, BigQuery.
- **Dữ liệu bán cấu trúc (Semi-structured Data):** Không tuân theo bảng cố định nhưng có cấu trúc phân cấp, thẻ tag hoặc cặp key-value. Ví dụ: JSON từ REST API, tài liệu MongoDB, file XML cấu hình.

- **Dữ liệu phi cấu trúc (Unstructured Data):** Không có định dạng bảng định trước. Ví dụ: Nội dung review sản phẩm của khách hàng, tệp âm thanh ghi âm cuộc gọi của tổng đài viên, tệp video camera giám sát.

3.1.1 So Sánh Thực Chiến: CSV vs JSON vs Parquet vs Excel

Định Dạng	Lưu Trữ	Nén Dữ Liệu	Tốc Độ Đọc	Mục Đích Khuyến Dùng
CSV	Dạng dòng (Text)	Kém	Rất chậm	Xuất báo cáo nhỏ (< 50MB) cho con người đọc
JSON	Phân cấp (Text)	Kém	Chậm	Giao tiếp REST API, log sự kiện web/app
Excel (.xlsx)	Nhị phân/XML	Trung bình	Rất chậm	Nghiệp vụ kinh doanh, báo cáo tài chính phòng ban
Parquet	Dạng cột (Columnar)	Tuyệt hảo (Snappy)	Cực nhanh	Chuẩn lưu trữ Data Lake, Big Data, Data Warehouse

Kiến Thức Cốt Lõi: Bản Chất Của Lưu Trữ Dạng Cột (Columnar Storage)

Tại sao **Parquet** lại thống trị thế giới Data Engineering và Data Science hiện đại?

- **I/O Pruning (Cắt giảm đọc đĩa):** Khi bạn thực hiện câu lệnh SQL: `SELECT AVG(order_total) FROM sales;`, định dạng dòng (CSV) buộc ổ cứng phải đọc toàn bộ 100 cột trên mỗi dòng từ đĩa vào RAM rồi mới bỏ đi 99 cột thừa. Trong khi đó, định dạng cột (Parquet) chỉ cần nhảy trực tiếp đến khối lưu trữ cột `order_total`, giảm tới 90–99% lượng dữ liệu đọc từ đĩa cứng (I/O).
- **Tỷ lệ nén cực đại:** Vì tất cả giá trị trong cùng một cột có cùng kiểu dữ liệu (ví dụ cột số nguyên, hoặc cột mã bưu điện), các thuật toán nén (Snappy, GZIP, Dictionary Encoding) có thể nén dữ liệu giảm từ 10GB CSV xuống chỉ còn 1.5GB Parquet!

3.2 Git & GitHub Thực Chiến Cho Kỹ Sư Dữ Liệu

Git không chỉ là công cụ lưu trữ mã nguồn, nó là cuốn nhật ký hoạt động và lá chắn bảo vệ an toàn cho hệ sinh thái dữ liệu của doanh nghiệp.

Thực Chiến & Best Practices: Quy Chuẩn Commit & Gitignore Bắt Buộc

1. Chuẩn hóa Commit (Conventional Commits):

- `feat:` add incremental extraction for order table (Tính năng mới)
- `fix:` resolve null pointer exception in currency converter (Sửa lỗi)

- refactor: optimize sql execution plan for churn query (Tối ưu hóa)
- docs: update data dictionary for dim_customer (Tài liệu hóa)

2. Tập .gitignore Chuẩn Mục Trong Mọi Data Project: Tuyệt đối không bao giờ đẩy (push) tệp dữ liệu lớn hoặc thông tin bảo mật lên GitHub:

```

1 # Không đẩy dữ liệu thô và dữ liệu xử lý lên Git
2 *.csv
3 *.parquet
4 *.xlsx
5 data/raw/
6 data/processed/
7
8 # Không bao giờ đẩy credentials, API keys, mật khẩu database
9 .env
10 *.pem
11 secrets/
12
13 # Mọi trường ảo và cache Python
14 __pycache__/
15 *.pyc
16 .venv/
17 .ipynb_checkpoints/

```

3.3 Linux CLI & Xử Lý Dữ Liệu Nhanh Bằng Dòng Lệnh

3.3.1 1. Lệnh Điều Hướng Hệ Thống Tập Tin Từ Con Số 0

Trước khi lọc dữ liệu lớn, bạn phải biết cách di chuyển và quản lý tệp tin trong hệ điều hành:

Thực Chiến & Best Practices: Các Lệnh Điều Hướng Cốt Lõi Bắt Buộc Phải Nhớ

- `pwd` (*Print Working Directory*): In ra đường dẫn thư mục hiện tại mà bạn đang đứng.
- `ls -lh`: Liệt kê tất cả các tệp tin và thư mục kèm dung lượng dễ đọc (*Human-readable*: KB, MB, GB).
- `cd <tên_thư_mục>`: Đi vào trong thư mục đó.
- `cd ..`: Lùi ra thư mục cha bên ngoài một cấp.
- `cd` : Nhảy ngay về thư mục gốc người dùng (Home Directory).
- `mkdir -p data/raw`: Tạo thư mục mới (cờ `-p` tự động tạo các thư mục cha nếu chưa tồn tại).

- `touch notes.txt`: Tạo một tệp tin văn bản rỗng mới.
- `cp file.csv backup.csv`: Sao chép (copy) tệp tin.
- `mv file.csv data/`: Di chuyển tệp tin vào thư mục `data/` hoặc đổi tên tệp.
- `rm file.txt`: Xóa tệp tin vĩnh viễn (**CẢNH BÁO**: Trên Linux dòng lệnh không có thùng rác Recycle Bin, xóa là mất vĩnh viễn!).

3.3.2 2. Thao Tác & Lọc Dữ Liệu Lớn Siêu Tốc

Một kỹ sư dữ liệu giỏi có thể chẩn đoán lỗi trên tập dữ liệu 10 triệu bản ghi trong 5 giây chỉ bằng các lệnh Bash mà không cần đợi mở Jupyter Notebook:

Thực Chiến & Best Practices: Các Lệnh Bash Thao Tác Dữ Liệu Siêu Tốc

```
1 # 1. Xem 5 dòng đầu tiên và 5 dòng cuối cùng
2 head -n 5 sales_data.csv
3 tail -n 5 sales_data.csv
4
5 # 2. Đếm tổng số dòng trong file CSV
6 wc -l sales_data.csv
7
8 # 3. Tìm tất cả các dòng hàng bị lỗi 'PAYMENT_FAILED'
9 grep "PAYMENT_FAILED" access_log.txt | wc -l
10
11 # 4. Trích xuất cột 2 và cột 5 từ file CSV phân tách bởi dấu phẩy
12 cut -d ',' -f 2,5 sales_data.csv | head -n 10
13
14 # 5. Lọc ra các giá trị duy nhất và đếm tần suất (Value Counts cực nhanh)
15 # Lay cột status (cột 4), sắp xếp và đếm số lần xuất hiện
16 cut -d ',' -f 4 sales_data.csv | sort | uniq -c | sort -nr
17
18 # 6. Dùng AWK để tính tổng doanh thu trên cột 3 (Amount)
19 awk -F ',' '{sum += $3} END {print "Tổng Doanh Thu = ", sum}'
    sales_data.csv
```

3.4 Bí Kíp AI Copilot: Sinh Dữ Liệu Giả Lập & Scripting

Bí Kíp AI Copilot & Prompt Engineering: Prompt Sinh Mock Data Chuẩn Schema Doanh Nghiệp

Khi bạn cần kiểm thử một pipeline hoặc xây dựng portfolio mà không có dữ liệu thực tế, hãy dùng prompt sau:

"Đóng vai trò là Senior Data Engineer. Hãy viết một script Python hoàn chỉnh sử dụng thư viện Faker để sinh ra 5,000 dòng dữ liệu giao dịch bán hàng thương mại điện tử. Tập xuất ra định dạng CSV và Parquet tên là 'ecommerce_transactions'. Các cột bao gồm: 1. 'transaction_id' (UUIDv4 dạng chuỗi) 2. 'customer_id' (định dạng 'CUST_' kèm 5 chữ số ngẫu nhiên từ 10000 đến 15000 để tạo ra hành vi mua lặp lại) 3. 'order_timestamp' (phân bố ngẫu nhiên trong 90 ngày gần nhất) 4. 'category' (chọn từ ['Electronics', 'Fashion', 'Home', 'Beauty', 'Sports']) 5. 'amount' (phân phối chuẩn lệch phải, giá trị từ 10 đến 2,500 USD, làm tròn 2 chữ số thập phân) 6. 'status' (tỷ lệ 85% 'COMPLETED', 10% 'CANCELLED', 5% 'REFUNDED') Yêu cầu: Chèn cố tình 2% giá trị null ở cột 'amount' và 5% khách hàng có thông tin địa chỉ email rỗng để làm bài tập Data Cleaning. Viết mã tối ưu hóa chạy dưới 3 giây."

3.5 PROJECT 0: Xây Dựng Personal Data CLI Analyzer

3.5.1 Bài Toán Thực Tế

Mỗi khi nhận được một tệp dữ liệu mới từ đối tác hoặc đội ngũ vận hành, Data Analyst hay Data Engineer mất từ 15 đến 30 phút để mở Jupyter Notebook, nạp thư viện Pandas, viết các dòng lệnh xem kích thước bảng, đếm số lượng null, kiểm tra trùng lặp và tính các chỉ số thống kê.

Mục tiêu của **Project 0** là xây dựng một công cụ dòng lệnh (CLI Tool) bằng Python thuần mang tên **data-analyzer** có thể chạy trực tiếp từ Terminal:

```
1 python src/analyzer.py --file data/input.csv --output report.txt
```

3.5.2 Mã Nguồn Thực Hiện Hoàn Chỉnh

```
1 import argparse
2 import sys
3 import pandas as pd
4 import numpy as np
5
6 def analyze_dataset(file_path: str, output_path: str = None) -> None:
7     """Doc va phan tich ho so tong quan cua tap du lieu CSV/Parquet."""
8     print(f"[*] Đang tải dữ liệu từ: {file_path}...")
9     try:
10         if file_path.endswith('.csv'):
11             df = pd.read_csv(file_path)
```

```

12         elif file_path.endswith('.parquet'):
13             df = pd.read_parquet(file_path)
14         else:
15             print("[!] Loi: Chi ho tro dinh dang .csv va .parquet!")
16             sys.exit(1)
17     except Exception as e:
18         print(f"[!] Khong the doc file: {e}")
19         sys.exit(1)
20
21     # Thu thap thong tin thong ke
22     total_rows, total_cols = df.shape
23     duplicate_rows = df.duplicated().sum()
24     memory_usage_mb = df.memory_usage(deep=True).sum() / (1024 * 1024)
25
26     report_lines = []
27     report_lines.append("=" * 60)
28     report_lines.append("          BAO CAO HO SO DU LIEU (DATA PROFILING
REPORT)")
29     report_lines.append("=" * 60)
30     report_lines.append(f"Tong so ban ghi (Rows)      : {total_rows:,}")
31     report_lines.append(f"Tong so cot (Columns)      : {total_cols}")
32     report_lines.append(f"Ban ghi trung lap (Dupes) : {duplicate_rows}
({duplicate_rows/total_rows*100:.2f}%)")
33     report_lines.append(f"Dung luong bo nho (RAM)      :
{memory_usage_mb:.2f} MB\n")
34
35     report_lines.append("-" * 60)
36     report_lines.append("CHI TIET CAC COT VA GIA TRI THIEU (MISSING
VALUES):")
37     report_lines.append("-" * 60)
38     for col in df.columns:
39         null_count = df[col].isnull().sum()
40         null_pct = (null_count / total_rows) * 100
41         dtype = str(df[col].dtype)
42         unique_cnt = df[col].nunique()
43         report_lines.append(f"{col:<20} | Type: {dtype:<10} | Null:
{null_count:>6} ({null_pct:>5.1f}%) | Unique: {unique_cnt}")
44
45     report_lines.append("\n" + "-" * 60)
46     report_lines.append("THONG KE COT DU LIEU SO (NUMERICAL METRICS):")
47     report_lines.append("-" * 60)
48     num_cols = df.select_dtypes(include=[np.number]).columns
49     if len(num_cols) > 0:
50         desc = df[num_cols].describe().T[['mean', 'std', 'min', '50%',
'max']]
51         desc.rename(columns={'50%': 'median'}, inplace=True)
52         report_lines.append(desc.to_string())
53     else:
54         report_lines.append("Khong co cot du lieu dang so.")

```

```

55
56     full_report = "\n".join(report_lines)
57     print(full_report)
58
59     if output_path:
60         with open(output_path, "w", encoding="utf-8") as f:
61             f.write(full_report)
62             print(f"\n[+] Đã lưu báo cáo vào: {output_path}")
63
64 if __name__ == "__main__":
65     parser = argparse.ArgumentParser(description="Công cụ CLI phân tích
66     nhanh dữ liệu CSV/Parquet")
67     parser.add_argument("--file", "-f", required=True, help="Đường dẫn đến
68     file dữ liệu")
69     parser.add_argument("--output", "-o", required=False, help="Đường dẫn
        lưu file text báo cáo")
        args = parser.parse_args()
        analyze_dataset(args.file, args.output)

```

3.6 Bài Tập Tự Luyện Level 0

Bài Tập Tự Luyện: Hệ Thống 5 Bài Tập Dòng Lệnh & Quản Lý Dự Án

- Bài 1 (Bash - Phân tích log):** Cho file `server_logs.txt`. Hãy viết một câu lệnh Bash một dòng (one-liner) đếm số lượng địa chỉ IP duy nhất (cột thứ 1) gửi yêu cầu thành công mã HTTP 200 (cột thứ 9).
- Bài 2 (So sánh kích thước):** Hãy tải một file CSV 100MB, chuyển đổi sang Parquet bằng Pandas. Ghi lại dung lượng chênh lệch và so sánh thời gian đọc 1 cột của 2 file.
- Bài 3 (Git - Xử lý xung đột):** Tạo 2 branch `feature/sql-query` và `feature/python-etl`. Cùng sửa dòng thứ 5 trong file `README.md`, sau đó tiến hành merge vào branch `main` và xử lý conflict phát sinh.
- Bài 4 (CLI Enhancements):** Mở rộng Project 0 thêm tính năng tự động phát hiện các cột dạng chuỗi nhưng thực chất là ngày tháng (`datetime`) để đưa ra khuyến nghị chuyển đổi kiểu dữ liệu.
- Bài 5 (Cron Job):** Viết một dòng cấu hình Cron job trên Linux để tự động thực thi script phân tích dữ liệu `analyzer.py` vào đúng 02:00 sáng hàng ngày từ Thứ Hai đến Thứ Sáu.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Hướng Dẫn Kiểm Tra

- Bài 1:** Lệnh Bash chuẩn:

```
awk '$9 == "200" {print $1}' server_logs.txt | sort -u | wc -l
```

2

- **Bài 2:** Với dữ liệu bảng thông thường, Parquet giảm dung lượng từ 100MB xuống còn khoảng 15–25MB (nén Snappy). Thời gian đọc một cột số lượng/doanh thu trên Parquet nhanh gấp 10–30 lần so với CSV vì cơ chế Columnar I/O Pruning.
- **Bài 3:** Khi gặp conflict, Git sẽ đánh dấu <<<<<< HEAD, =====, >>>>>> branch_name. Cần mở file, thống nhất nội dung cần giữ lại, xóa các ký tự đánh dấu của Git, sau đó thực hiện `git add README.md` và `git commit -m "fix: resolve merge conflict in README"`.
- **Bài 4:** Gợi ý code: Thử parse thử nghiệm 50 giá trị đầu tiên của cột object qua `pd.to_datetime(sample, errors='coerce')`. Nếu tỷ lệ parse thành công đạt > 90%, in ra cảnh báo: "Khuyen nghi doi sang datetime".
- **Bài 5:** Dòng crontab chuẩn:

```
1 0 2 * * 1-5 /usr/bin/python3 /path/to/src/analyzer.py --file
  /data/daily.csv >> /logs/analyzer.log 2>&1
```

2

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Project 0 Vào CV Của Bạn

Personal Data Profiling CLI Tool | Python, Argparse, Pandas, Bash

- Xây dựng công cụ dòng lệnh (CLI) bằng Python hỗ trợ tự động hóa việc kiểm tra chất lượng dữ liệu ban đầu cho các tệp CSV và Parquet lên tới 500,000 bản ghi.
- Tự động trích xuất các chỉ số quan trọng (missing values, duplicate rate, distribution metrics) chỉ trong dưới 2 giây, giúp cắt giảm 80% thời gian thăm dò dữ liệu thủ công.

Chương 4

Level 1: Lập Trình Python Từ Con Số 0 & Phân Tích Khám Phá (EDA) (Tuần 3–8)

Kiến Thức Cốt Lõi: Mục Tiêu & Tiền Đề Cần Nhớ

Tiền đề bắt buộc: Bạn chỉ cần đã hoàn thành Chương 0 (đã cài đặt VS Code và Python 3.11 chạy được). **Không cần bất kỳ kinh nghiệm lập trình nào trước đó!**

Mục tiêu chương này:

- Nắm vững cú pháp Python từ con số 0: Biến, Kiểu dữ liệu, Toán tử, Cấu trúc rẽ nhánh, Vòng lặp, Hàm và Cấu trúc dữ liệu (`list`, `dict`, `tuple`, `set`).
- Hiểu kỹ thuật lập trình nâng cao: Comprehensions, Generators (tiết kiệm RAM), Decorators, Type Hinting và Xử lý ngoại lệ (`try-except`).
- Làm chủ thư viện **NumPy** với tính toán Vectorization thay thế vòng lặp `for`.
- Trở thành bậc thầy thư viện **Pandas**: Thao tác DataFrame, GroupBy Aggregations, Nối bảng (Merge/Join), xử lý Missing Data và Outliers.
- Hoàn thành **Project 1: E-commerce Sales Explorer** và trả lời 5 câu hỏi kinh doanh chiến lược của doanh nghiệp.

4.1 Python Căn Bản Từ Con Số 0

4.1.1 1. Biến (Variables) & 4 Kiểu Dữ Liệu Nguyên Thủy

Biến trong lập trình giống như một chiếc hộp có dán nhãn, dùng để lưu trữ dữ liệu trong bộ nhớ RAM của máy tính. Bạn gán giá trị cho biến bằng dấu bằng `=`.

Thực Chiến & Best Practices: Các Kiểu Dữ Liệu Cơ Bản Trong Python

```

1 # 1. Số nguyên (int - Integer): Số không có phần thập phân
2 customer_age = 28
3 total_orders = 15
4
5 # 2. Số thực (float - Floating-point): Số có dấu phẩy thập phân
6 order_amount = 250.75
7 vat_rate = 0.08
8
9 # 3. Chuỗi ký tự (str - String): Đoạn văn bản đặt trong dấu nháy đơn
    hoặc nháy kép
10 customer_name = "Nguyen Van An"
11 city = 'Ha Noi'
12
13 # 4. Dung/Sai logic (bool - Boolean): Chỉ nhận 1 trong 2 giá trị:
    True hoặc False
14 is_active_member = True
15 has_discount_coupon = False
16
17 # In ra màn hình và kiểm tra kiểu dữ liệu bằng hàm type()
18 print("Tên khách hàng:", customer_name)
19 print("Kiểu dữ liệu của order_amount là:", type(order_amount)) #
    <class 'float'>
20
21 # Ép kiểu dữ liệu (Type Casting)
22 str_price = "500" # Đây là chuỗi ký tự, không thể cộng
    tru
23 numeric_price = float(str_price) # Ép sang số thực: 500.0
24 print("Giá sau khi tính thuế:", numeric_price * 1.08)

```

4.1.2 2. Các Toán Tử Toán Học & So Sánh

- **Toán tử số học:** + (cộng), - (trừ), * (nhân), / (chia thập phân), // (chia lấy phần nguyên: $7 // 2 = 3$), % (chia lấy dư - modulo: $7 \% 2 = 1$), ** (lũy thừa: $2 ** 3 = 8$).
- **Toán tử so sánh (Trả về True hoặc False):** == (bằng nhau), != (khác nhau), > (lớn hơn), < (nhỏ hơn), >=, <=.
- **Toán tử logic:** and (cả 2 cùng đúng), or (chỉ cần 1 trong 2 đúng), not (phủ định).

4.1.3 3. Cấu Trúc Rẽ Nhánh Điều Kiện (if – elif – else)

Máy tính đưa ra quyết định dựa trên điều kiện logic. **Lưu ý sống còn:** Python sử dụng **thụt đầu dòng (Indentation - 4 dấu cách)** để xác định khối mã lệnh nào thuộc về điều kiện đó!

```

1 total_spending = 450.0
2
3 if total_spending >= 500.0:
4     discount = 0.15
5     customer_tier = "VIP"
6 elif total_spending >= 200.0:
7     discount = 0.05
8     customer_tier = "Gold"
9 else:
10    discount = 0.0
11    customer_tier = "Standard"
12
13 final_payment = total_spending * (1 - discount)
14 print(f"Hang: {customer_tier} | Giảm giá: {discount*100}% | Cần thanh
    toan: ${final_payment:.2f}")

```

4.1.4 4. Vòng Lặp (for & while)

```

1 # Vòng lặp for kết hợp với hàm range(start, stop, step)
2 print("Đếm số chẵn từ 2 đến 10:")
3 for i in range(2, 11, 2):
4     print(i, end=" ")
5 print("\n")
6
7 # Duyệt qua từng phần tử trong một danh sách
8 order_amounts = [120.0, 45.0, 300.0, 85.0]
9 total_revenue = 0.0
10
11 for amount in order_amounts:
12     if amount > 200.0:
13         print(f"[!] Phát hiện đơn hàng lớn: ${amount}")
14     total_revenue += amount
15
16 print(f"Tổng doanh thu hôm nay: ${total_revenue}")

```

4.1.5 5. Bốn Cấu Trúc Dữ Liệu Cốt Lõi: List, Dict, Tuple, Set

Thực Chiến & Best Practices: So Sánh Bản Chất 4 Cấu Trúc Dữ Liệu

1. **List (Danh sách - Ký hiệu [])**: Có thứ tự, có thể thay đổi giá trị (Mutable), cho phép trùng lặp.

```

1 cities = ["Hanoi", "Saigon", "Danang"]
2 cities.append("Cantho")           # Thêm vào cuối: ['Hanoi',
    'Saigon', 'Danang', 'Cantho']
3 first_city = cities[0]           # Lấy phần tử đầu tiên: 'Hanoi'

```

```

4 last_city = cities[-1]           # Lay phan tu cuoi cung: 'Cantho'
5 sub_list = cities[1:3]           # Cat lat (Slicing): ['Saigon',
                                   'Danang']
6

```

2. **Dictionary (Từ điển - Ký hiệu {})**: Lưu trữ cặp Khóa – Giá trị (Key: Value). Cực kỳ quan trọng để lưu thông tin bản ghi dữ liệu.

```

1 customer = {
2     "id": "CUST_1001",
3     "name": "Nguyen Van An",
4     "spending": 350.50,
5     "is_active": True
6 }
7 print(customer["name"])           # Truy xuất: 'Nguyen
                                   Van An'
8 customer["email"] = "an.nguyen@data.vn" # Them khoa moi
9 spending = customer.get("bonus", 0.0)   # Dung .get() de tranh
                                   loi neu khong ton tai key
10

```

3. **Tuple (Bộ giá trị - Ký hiệu ())**: Có thứ tự nhưng **bất biến (Immutable)**, không thể thêm/sửa/xóa sau khi tạo. Dùng cho tọa độ, cấu hình.

```

1 coordinates = (21.0285, 105.8542) # Vi do, kinh do Ha Noi
2

```

4. **Set (Tập hợp - Ký hiệu {phần tử})**: Không có thứ tự, **tự động loại bỏ** hoàn toàn các phần tử trùng lặp.

```

1 raw_categories = ["Tech", "Home", "Tech", "Beauty", "Home"]
2 unique_categories = set(raw_categories)
3 print(unique_categories) # {'Tech', 'Home', 'Beauty'}
4

```

4.1.6 6. Hàm (Functions) & Xử Lý Lỗi (try – except)

```

1 def calculate_net_income(revenue: float, tax_rate: float = 0.1) -> float:
2     """Tính thu nhập ròng sau thuế với giá trị thuế mặc định là 10%."""
3     if revenue < 0:
4         raise ValueError("Doanh thu không thể là số âm!")
5     net_income = revenue * (1 - tax_rate)
6     return round(net_income, 2)
7
8 # Su dung try-except de chuong trinh khong bi sap khi gap loi du lieu
9 user_inputs = [1000.0, -50.0, 2500.0]

```

```

10
11 for rev in user_inputs:
12     try:
13         net = calculate_net_income(rev)
14         print(f"Doanh thu: ${rev} -> Thu nhập thực: ${net}")
15     except ValueError as err:
16         print(f"[!] Canh bao loi du lieu: {err}")

```

4.2 Nâng Cấp Kỹ Năng Pythonic & Tối Ưu Hiệu Năng

Sau khi đã nắm vững các cú pháp căn bản, đây là lúc bạn chuyển đổi tư duy từ người mới thành kỹ sư dữ liệu viết code sạch sẽ, tối ưu:

4.2.1 Comprehensions & Generators Tiết Kiệm Bộ Nhớ

Thực Chiến & Best Practices: List Comprehension vs Generator Memory Benchmark

Thay vì viết 5 dòng vòng lặp `for` để tạo một danh sách mới, hãy dùng cú pháp thanh thoát **List Comprehension**:

```

1 # Cách viết cũ kĩ (5 dòng):
2 squares = []
3 for x in range(10):
4     if x % 2 == 0:
5         squares.append(x ** 2)
6
7 # Cách viết Pythonic (1 dòng duy nhất):
8 squares_pythonic = [x ** 2 for x in range(10) if x % 2 == 0]

```

Sức mạnh của Generator với `yield`: Khi xử lý tệp log 20GB, **List Comprehension** sẽ tạo ra danh sách 20GB trong RAM và làm sập máy tính. Ngược lại, **Generator** chỉ sinh dữ liệu khi được gọi:

```

1 def stream_large_log(file_path: str):
2     """Generator đọc từng dòng của file dung lượng cực lớn."""
3     with open(file_path, 'r', encoding='utf-8') as f:
4         for line in f:
5             if "ERROR" in line:
6                 yield line.strip()
7
8 # RAM tiêu thụ luôn giữ ở mức gần như 0 MB!
9 for err in stream_large_log("data/raw_transactions.csv"):
10     pass

```

4.3 NumPy: Sức Mạnh Tính Toán Vector (Vectorization)

Kiến Thức Cốt Lõi: Tại Sao Vectorization Lại Nhanh Hơn Vòng Lặp For 150 Lần?

NumPy quản lý mảng `ndarray` trong các vùng nhớ liên tục của C-language. Khi bạn thực hiện phép tính trên 1 triệu phần tử, thay vì duyệt tuần tự 1 triệu lần trong môi trường máy ảo Python (Interpreter), NumPy phát các chỉ số phần cứng SIMD (Single Instruction, Multiple Data) của chip vì xử lý để tính toán đồng thời hàng nghìn phép tính trong một chu kỳ xung nhịp!

```

1 import numpy as np
2 import time
3
4 # Tạo mảng 1 triệu phần tử
5 arr = np.random.rand(1_000_000)
6
7 # Cách 1: Vòng lặp For thuần túy
8 start = time.perf_counter()
9 res_for = [x * 1.08 for x in arr]
10 time_for = time.perf_counter() - start
11
12 # Cách 2: Vectorization bằng NumPy
13 start = time.perf_counter()
14 res_vec = arr * 1.08
15 time_vec = time.perf_counter() - start
16
17 print(f"Thời gian dùng For Loop : {time_for:.4f}s")
18 print(f"Thời gian dùng Vectorized : {time_vec:.4f}s")
19 print(f"-> NumPy nhanh hơn: {time_for / time_vec:.1f} LAN!")

```

4.4 Pandas Masterclass: Trái Tim Của Xử Lý Dữ Liệu

Pandas cung cấp hai cấu trúc dữ liệu chính: `Series` (cột 1 chiều) và `DataFrame` (bảng 2 chiều có dòng và cột).

4.4.1 1. Đọc Dữ Liệu, Lọc & Named Aggregations

```

1 import pandas as pd
2
3 # Đọc dữ liệu Orders đã sinh ở Chương 0
4 df_orders = pd.read_csv("data/orders.csv",
5                          parse_dates=['order_purchase_timestamp'])
6
7 # Xem 5 dòng đầu tiên và kích thước bảng
8 print("Kích thước bảng:", df_orders.shape)
9 print(df_orders.head(3))

```

```

9
10 # Loc đơn hàng: Chỉ lấy đơn COMPLETED trong năm 2025
11 completed_2025 = df_orders[
12     (df_orders['order_status'] == 'COMPLETED') &
13     (df_orders['order_purchase_timestamp'] >= '2025-01-01')
14 ]
15 print("Số đơn hoàn thành:", len(completed_2025))

```

4.4.2 2. Kết Nối Bảng (Merge / Join) & Phép Gộp Nhóm (GroupBy)

```

1 # Doc bang chi tiet san pham
2 df_items = pd.read_csv("data/order_items.csv")
3
4 # Merge đơn hàng với chi tiết sản phẩm giống như INNER JOIN trong SQL
5 df_merged = pd.merge(df_orders, df_items, on='order_id', how='inner')
6
7 # Named Aggregations: Tính tổng doanh thu và số lượng sản phẩm theo
  Category
8 category_report = df_merged.groupby('category').agg(
9     total_revenue=('price', 'sum'),
10    avg_price=('price', 'mean'),
11    total_quantity=('quantity', 'sum'),
12    order_count=('order_id', 'nunique')
13 ).reset_index()
14
15 category_report['total_revenue'] =
16     category_report['total_revenue'].round(2)
17 print("=== BAO CAO HIEU NANG DANH MUC SAN PHAM ===")
18 print(category_report.sort_values(by='total_revenue', ascending=False))

```

4.5 PROJECT 1: E-Commerce Sales Explorer

4.5.1 Kịch Bản Thực Tế

Sử dụng bộ dữ liệu E-commerce đã tạo tại thư mục `data/` để tiến hành phân tích khám phá toàn diện, trả lời 5 câu hỏi kinh doanh của Ban Giám đốc:

1. Doanh thu và tăng trưởng đơn hàng theo từng tháng (MoM Growth Rate).
2. Kiểm chứng quy luật Pareto: 20% khách hàng đóng góp bao nhiêu % doanh thu?
3. Tỷ lệ hủy/hoàn tiền theo từng phương thức thanh toán.
4. Danh mục sản phẩm bán chạy nhất theo từng quý.
5. Giá trị giỏ hàng trung bình (AOV - Average Order Value).

```

1  import pandas as pd
2  import numpy as np
3
4  def run_project1():
5      print("[*] Dang tai cac tap du lieu E-commerce...")
6      orders = pd.read_csv("data/orders.csv",
7      parse_dates=['order_purchase_timestamp'])
8      items = pd.read_csv("data/order_items.csv")
9      payments = pd.read_csv("data/payments.csv")
10
11     # Merge cac bang
12     df = orders.merge(items, on='order_id', how='inner').merge(payments,
13     on='order_id', how='inner')
14
15     # 1. Doanh thu theo thang va tang truong MoM
16     df['order_month'] = df['order_purchase_timestamp'].dt.to_period('M')
17     monthly = df.groupby('order_month')['price'].agg(['sum',
18     'count']).reset_index()
19     monthly.columns = ['Month', 'Revenue', 'Orders']
20     monthly['MoM_Growth_%'] = (monthly['Revenue'].pct_change() *
21     100).round(2)
22     print("\n--- 1. TANG TRUONG DOANH THU THEO THANG ---")
23     print(monthly.head(6))
24
25     # 2. Quy luat Pareto (80/20)
26     cust_rev =
27     df.groupby('customer_id')['price'].sum().sort_values(ascending=False).reset_index()
28     total_rev = cust_rev['price'].sum()
29     cust_rev['cum_pct'] = (cust_rev['price'].cumsum() / total_rev) * 100
30     top_20_cust = int(len(cust_rev) * 0.20)
31     pareto_share = cust_rev.iloc[top_20_cust]['cum_pct']
32     print(f"\n--- 2. QUY LUAT PARETO ---")
33     print(f"Top 20% khach hang tao ra: {pareto_share:.2f}% tong doanh
34     thu!")
35
36     if __name__ == '__main__':
37         run_project1()

```

4.6 Bài Tập Tự Luyện Level 1

Bài Tập Tự Luyện: 5 Bài Tập Python & Pandas Thực Chiến

1. **Bài 1 (Căn bản):** Viết một hàm nhận vào danh sách số nguyên, sử dụng vòng lặp để tìm số lớn nhất và tính giá trị trung bình mà không dùng hàm có sẵn `max()` và `sum()`.
2. **Bài 2 (Dictionary):** Cho một danh sách các giao dịch bán hàng dạng list of dicts. Hãy viết code tính tổng tiền của từng khách hàng và xuất ra một dict mới dạng `{customer_id: total_spent}`.
3. **Bài 3 (Pandas Cleaning):** Đọc file `data/input.csv`. Hãy lọc bỏ toàn bộ các dòng bị trùng lặp (`drop_duplicates`), sau đó điền giá trị trung vị của cột `age` vào những dòng bị khuyết (`fillna`).
4. **Bài 4 (Pandas Datetime):** Từ file `data/orders.csv`, hãy tạo thêm 2 cột mới: `day_of_week` (Thứ trong tuần: Monday, Tuesday...) và `is_weekend` (True nếu là Thứ Bảy hoặc Chủ Nhật).
5. **Bài 5 (Pandas Pivot):** Sử dụng hàm `pivot_table` trên dữ liệu hợp nhất để hiển thị tổng doanh thu với dòng là các Tháng (`order_month`) và cột là các Danh mục (`category`).

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Hướng Dẫn Kiểm Tra

- **Bài 1:** Khởi tạo `max_val = lst[0]`, `total = 0`. Duyệt qua từng phần tử `for x in lst`: `total += x`; `if x > max_val: max_val = x`. Trung bình là `total / len(lst)`.

- **Bài 2:**

```
1 res = {}
2 for tx in transactions:
3     cid, amt = tx['customer_id'], tx['amount']
4     res[cid] = res.get(cid, 0.0) + amt
5
```

- **Bài 3:**

```
1 df = pd.read_csv("data/input.csv").drop_duplicates()
2 median_age = df['age'].median()
3 df['age'] = df['age'].fillna(median_age)
4
```

- **Bài 4:**

```
1 df['day_of_week'] = df['order_purchase_timestamp'].dt.day_name()
2 df['is_weekend'] =
    df['order_purchase_timestamp'].dt.dayofweek.isin([5, 6])
```

3

• Bài 5:

```
1 pivot_sales = df.pivot_table(index='order_month',  
    columns='category', values='price', aggfunc='sum',  
    fill_value=0)
```

2

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Dự Án Python Vào CV**E-Commerce Exploratory Data Analysis Pipeline | Python, Pandas, NumPy, Data Cleaning**

- Xây dựng pipeline phân tích khám phá (EDA) trên tập dữ liệu đa bảng gồm hơn 100,000 giao dịch bán lẻ, xử lý 3% dữ liệu khuyết và khử trùng lặp bản ghi.
- Ứng dụng kỹ thuật Vectorization của NumPy và Named Aggregations của Pandas, tối ưu hóa tốc độ xử lý gấp 50 lần so với vòng lặp truyền thống.
- Trích xuất các chỉ số kinh doanh then chốt (MoM Growth Rate, Pareto 80/20 Share, AOV), trực tiếp hỗ trợ chiến lược tối ưu hóa nguồn hàng của bộ phận kinh doanh.

Chương 5

Level 2: SQL & Cơ Sở Dữ Liệu Quan Hệ Từ Con Số 0 (Tuần 9–14)

Kiến Thức Cốt Lõi: Mục Tiêu & Tiền Đề Cần Nhớ

Tiền đề bắt buộc: Bạn chỉ cần đã cài đặt DBeaver và PostgreSQL ở Chương 0. **Không cần bất kỳ kiến thức cơ sở dữ liệu nào trước đó!**

Mục tiêu chương này:

- Hiểu tường tận bản chất Cơ sở dữ liệu quan hệ (RDBMS): Bảng, Cột, Dòng, Khóa chính (Primary Key), Khóa ngoại (Foreign Key).
- Viết thành thạo từ con số 0 các câu lệnh DDL (CREATE TABLE), DML (INSERT, UPDATE, DELETE) và DQL (SELECT, WHERE, ORDER BY, LIMIT, GROUP BY, HAVING).
- Làm chủ các loại phép nối bảng: INNER JOIN, LEFT JOIN, FULL JOIN, SELF JOIN và cách phòng tránh cạm bẫy Fan-Out nhân bản doanh thu.
- Nắm vững kỹ thuật SQL phân tích nâng cao: CTEs (Common Table Expressions) và **Window Functions** (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, Running Total).
- Hiểu cơ chế hoạt động bên dưới của Index B-Tree và đọc hiểu kế hoạch thực thi câu lệnh qua EXPLAIN ANALYZE.
- Hoàn thành **Project 2: E-commerce Analytics Database** với 50 câu truy vấn phân tích nghiệp vụ thực tế.

5.1 Cơ Sở Dữ Liệu Quan Hệ (RDBMS) Từ Con Số 0

5.1.1 1. Tại Sao Không Dùng Excel Để Lưu Trữ Dữ Liệu Công Ty?

Một bảng tính Excel chỉ chứa tối đa 1,048,576 dòng, không hỗ trợ nhiều người cùng ghi dữ liệu một lúc mà không bị xung đột, và cực kỳ dễ bị xóa nhầm công thức. Cơ sở dữ liệu quan hệ (RDBMS - Relational Database Management System) như **PostgreSQL** được thiết kế để:

- Lưu trữ hàng trăm triệu bản ghi an toàn tuyệt đối ngay cả khi mất điện đột ngột (chuẩn ACID).
- Phục vụ hàng nghìn người dùng và ứng dụng truy vấn đồng thời cùng lúc.
- Quản lý quan hệ chặt chẽ giữa các đối tượng kinh doanh (Khách hàng → Đơn hàng → Sản phẩm).

5.1.2 2. Bảng, Cột, Khóa Chính & Khóa Ngoại

- **Bảng (Table):** Một tập hợp dữ liệu được tổ chức dưới dạng hai chiều gồm các cột và các dòng.
- **Cột (Column / Field):** Đại diện cho một thuộc tính của dữ liệu (ví dụ: `email`, `age`, `signup_date`). Mỗi cột phải có một kiểu dữ liệu cố định.
- **Dòng (Row / Record):** Đại diện cho một đối tượng duy nhất (ví dụ: một khách hàng cụ thể hoặc một đơn hàng cụ thể).
- **Khóa Chính (Primary Key - PK):** Một hoặc nhiều cột có giá trị **duy nhất và không được phép để trống (NOT NULL)** dùng để định danh cho từng dòng trong bảng (ví dụ: `customer_id`).
- **Khóa Ngoại (Foreign Key - FK):** Cột trong bảng này tham chiếu đến Khóa Chính của một bảng khác, tạo ra sợi dây liên kết quan hệ giữa 2 bảng.

5.2 Ngôn Ngữ SQL Căn Bản Từ Con Số 0

SQL (Structured Query Language) là ngôn ngữ chuẩn quốc tế để giao tiếp với cơ sở dữ liệu, chia làm 3 nhóm câu lệnh chính:

5.2.1 1. Tạo Bảng Với DDL (Data Definition Language)

Thực Chiến & Best Practices: Các Kiểu Dữ Liệu Cốt Lõi Trong PostgreSQL

- Số: `INTEGER` (Số nguyên thường), `BIGINT` (Số nguyên lớn), `NUMERIC(12, 2)` (Số tiền chính xác có 2 chữ số thập phân).
- Chuỗi: `VARCHAR(50)` (Chuỗi giới hạn 50 ký tự), `TEXT` (Đoạn văn bản dài bất kỳ).
- Thời gian: `DATE` (YYYY-MM-DD), `TIMESTAMP` (Ngày giờ chính xác đến giây).
- Logic: `BOOLEAN` (TRUE hoặc FALSE).

```

1  -- Tao bang Danh muc Khach hang tu con so 0
2  CREATE TABLE dim_customers (
3      customer_id VARCHAR(50) PRIMARY KEY,      -- Khoa chinh, khong trung lap
4      customer_name VARCHAR(100) NOT NULL,      -- Bat buoc phai co ten
5      email VARCHAR(100) UNIQUE,                -- Khong duoc phep trung email
6      age INT CHECK (age >= 18 AND age <= 100), -- Rang buoc tuoi phai hop le
7      city VARCHAR(50) DEFAULT 'Chua xac dinh', -- Gia tri mac dinh neu de
        trong
8      signup_date DATE NOT NULL
9  );

```

5.2.2 2. Thêm, Sửa, Xóa Dữ Liệu Với DML (INSERT, UPDATE, DELETE)

```

1  -- 1. Chen du lieu moi (INSERT)
2  INSERT INTO dim_customers (customer_id, customer_name, email, age, city,
        signup_date)
3  VALUES
4      ('CUST_001', 'Nguyen Van A', 'an.nguyen@email.com', 25, 'Hanoi',
        '2025-01-15'),
5      ('CUST_002', 'Tran Thi B', 'b.tran@email.com', 32, 'Saigon',
        '2025-02-10');
6
7  -- 2. Cap nhat du lieu (UPDATE)
8  -- CANH BAO: Luon luon phai co menh de WHERE!
9  UPDATE dim_customers
10 SET city = 'Da Nang', age = 26
11 WHERE customer_id = 'CUST_001';
12
13 -- 3. Xoa du lieu (DELETE)
14 -- CANH BAO: Neu khong co WHERE, toan bo du lieu trong bang se bi xoa sach!
15 DELETE FROM dim_customers
16 WHERE customer_id = 'CUST_002';

```

5.2.3 3. Truy Vấn Dữ Liệu Cơ Bản (SELECT & WHERE)

```

1  -- Cu phap tong quat: SELECT cot1, cot2 FROM ten_bang WHERE dieu_kien;
2
3  -- Lay tat ca khach hang o Hanoi co do tuoi tren 20
4  SELECT
5      customer_id,
6      customer_name,
7      age,
8      city
9  FROM dim_customers
10 WHERE city = 'Hanoi' AND age >= 20;
11

```

```

12 -- Các toán tử lọc phạm vi trong WHERE:
13 -- 1. Khoảng giá trị:
14 SELECT * FROM fact_orders WHERE order_amount BETWEEN 100 AND 500;
15
16 -- 2. Danh sách lựa chọn (IN):
17 SELECT * FROM dim_customers WHERE city IN ('Hanoi', 'Saigon', 'Danang');
18
19 -- 3. Tìm kiếm chuỗi gần đúng (LIKE):
20 -- Dấu % đại diện cho chuỗi ký tự bất kỳ
21 SELECT * FROM dim_customers WHERE customer_name LIKE 'Nguyen%'; -- Bắt đầu
    bằng Nguyen
22 SELECT * FROM dim_customers WHERE email LIKE '%@gmail.com'; -- Cuối
    gmail
23
24 -- 4. Kiểm tra giá trị trong (NULL):
25 -- LƯU Ý: Không bao giờ được viết WHERE age = NULL, mà bắt buộc phải là IS
    NULL!
26 SELECT * FROM dim_customers WHERE email IS NULL;

```

5.2.4 4. Sắp Xếp, Giới Hạn & Khử Trùng Lặp (ORDER BY, LIMIT, DISTINCT)

```

1 -- Sắp xếp giảm dần (DESC) theo doanh thu và lấy Top 5
2 SELECT order_id, customer_id, order_amount
3 FROM fact_orders
4 ORDER BY order_amount DESC
5 LIMIT 5;
6
7 -- Lấy danh sách các thành phố duy nhất không bị trùng lặp
8 SELECT DISTINCT city
9 FROM dim_customers
10 ORDER BY city ASC;

```

5.2.5 5. Các Hàm Tổng Hợp (Aggregates) & Phép Gộp Nhóm (GROUP BY, HAVING)

Kiến Thức Cốt Lõi: Bản Chất Phép Gộp Nhóm & Sự Khác Biệt Giữa WHERE và HAVING

- **WHERE:** Lọc từng dòng dữ liệu ban đầu **TRƯỚC KHI** gộp nhóm. Bạn không thể viết: `WHERE SUM(amount) > 1000` vì lúc này hàm SUM chưa hề được tính!
- **GROUP BY:** Gộp tất cả các dòng có cùng giá trị trên một cột thành một nhóm duy nhất.
- **HAVING:** Lọc các nhóm **SAU KHI** các hàm tổng hợp (COUNT, SUM, AVG, MIN, MAX) đã được tính toán xong cho từng nhóm.

```

1 -- Tính tổng doanh thu và số đơn hàng của từng thành phố
2 -- Chỉ giữ lại những thành phố có tổng doanh thu trên 50,000 USD

```

```

3  SELECT
4      c.city,
5      COUNT(o.order_id) AS total_orders,
6      SUM(o.order_amount) AS total_revenue,
7      ROUND(AVG(o.order_amount), 2) AS avg_order_value
8  FROM fact_orders o
9  JOIN dim_customers c ON o.customer_id = c.customer_id
10 WHERE o.order_status = 'COMPLETED'           -- 1. Lọc tung đơn hàng
      COMPLETED trước
11 GROUP BY c.city                               -- 2. Gộp nhóm theo tung thành
      pho
12 HAVING SUM(o.order_amount) >= 50000.0         -- 3. Chỉ lấy các thành pho
      dat doanh thu cao
13 ORDER BY total_revenue DESC;                  -- 4. Sắp xếp giảm dần

```

5.3 SQL Nâng Cao: JOINS, CTEs & Window Functions

Sau khi đã thuần thục nền tảng cơ bản, đây là những vũ khí tối thượng giúp bạn giải quyết các bài toán dữ liệu lớn trong doanh nghiệp:

5.3.1 1. Các Loại JOIN & Cạm Bẫy Fan-Out Trùng Lặp Dữ Liệu

- **INNER JOIN**: Chỉ giữ lại các bản ghi có khóa xuất hiện ở CẢ HAI bảng.
- **LEFT JOIN**: Giữ lại TOÀN BỘ bản ghi ở bảng bên trái; nếu bảng bên phải không có dữ liệu khớp thì điền NULL.
- **Cạm bẫy Fan-Out Trap**: Khi nối bảng Đơn hàng (1 dòng) với Chi tiết đơn hàng (3 dòng), bảng Đơn hàng bị nhân bản thành 3 dòng. Nếu vội vàng tính SUM(order_total), doanh thu sẽ bị tính không gấp 3 lần! Hãy dùng CTE để tính tiền tổng hợp trước khi JOIN.

5.3.2 2. CTE (Common Table Expressions) – Tư Duy Module Hóa SQL

Thay vì lồng các subqueries hỗn loạn khó đọc, CTE sử dụng mệnh đề WITH để tạo ra các bảng tạm ảo có cấu trúc tuần tự, trong sáng:

```

1  WITH monthly_revenue AS (
2      SELECT
3          DATE_TRUNC('month', order_date)::DATE AS sales_month,
4          SUM(order_amount) AS revenue
5      FROM fact_orders
6      WHERE order_status = 'COMPLETED'
7      GROUP BY DATE_TRUNC('month', order_date)::DATE
8  ),
9  monthly_growth AS (
10     SELECT

```

```

11     sales_month,
12     revenue,
13     LAG(revenue, 1) OVER(ORDER BY sales_month) AS prev_month_revenue
14 FROM monthly_revenue
15 )
16 SELECT
17     sales_month,
18     revenue,
19     prev_month_revenue,
20     ROUND((revenue - prev_month_revenue) / prev_month_revenue * 100.0, 2)
21     AS mom_growth_pct
22 FROM monthly_growth;

```

5.3.3 3. Window Functions: ROW_NUMBER, RANK, DENSE_RANK

Window Function thực hiện tính toán trên một tập hợp các dòng có liên quan mà vẫn **bảo toàn từng dòng dữ liệu gốc ban đầu**:

```

1  SELECT
2      category,
3      product_id,
4      revenue,
5      -- Luôn danh số thứ tự liên tục không nhảy cóc: 1, 2, 3, 4
6      ROW_NUMBER() OVER(PARTITION BY category ORDER BY revenue DESC) AS
7      row_num,
8      -- Bảng nhau thì chung hàng, nhưng nhảy cóc: 1, 2, 2, 4
9      RANK() OVER(PARTITION BY category ORDER BY revenue DESC) AS rnk,
10     -- Bảng nhau thì chung hàng, không nhảy cóc: 1, 2, 2, 3 (Pho biến nhất
11     de lay Top N)
12     DENSE_RANK() OVER(PARTITION BY category ORDER BY revenue DESC) AS
13     dense_rnk
14 FROM product_sales;

```

5.4 PROJECT 2: E-Commerce Analytics Database

Khởi tạo schema hoàn chỉnh trong PostgreSQL và viết các truy vấn phân tích nghiệp vụ then chốt:

```

1  -- 1. SCHEMA KHO DU LIEU E-COMMERCE
2  CREATE TABLE dim_customers_dwh (
3      customer_id VARCHAR(50) PRIMARY KEY,
4      customer_name VARCHAR(100) NOT NULL,
5      city VARCHAR(50),
6      signup_date DATE NOT NULL
7  );
8
9  CREATE TABLE fact_orders_dwh (
10     order_id VARCHAR(50) PRIMARY KEY,

```

```

11     customer_id VARCHAR(50) REFERENCES dim_customers_dwh(customer_id),
12     order_date  TIMESTAMP NOT NULL,
13     order_status VARCHAR(20) NOT NULL,
14     order_amount NUMERIC(12, 2) NOT NULL
15 );
16
17 -- Tao Index B-Tree tang toc truy van theo khach hang va thoi gian
18 CREATE INDEX idx_orders_cust_date ON fact_orders_dwh (customer_id,
    order_date);

```

5.4.1 Truy Vấn Phân Hạng Khách Hàng (RFM Segmentation)

```

1  WITH rfm_base AS (
2      SELECT
3          customer_id,
4          MAX(order_date) AS last_order_date,
5          COUNT(order_id) AS frequency,
6          SUM(order_amount) AS monetary
7      FROM fact_orders_dwh
8      WHERE order_status = 'COMPLETED'
9      GROUP BY customer_id
10 ),
11 rfm_scores AS (
12     SELECT
13         customer_id,
14         NTILE(4) OVER(ORDER BY last_order_date ASC) AS r_score,    --
15         Recency (Gan day nhat)
16         NTILE(4) OVER(ORDER BY frequency DESC) AS f_score,        --
17         Frequency (Tan suat)
18         NTILE(4) OVER(ORDER BY monetary DESC) AS m_score         --
19         Monetary (Gia tri)
20     FROM rfm_base
21 )
22 SELECT
23     customer_id,
24     r_score, f_score, m_score,
25     CASE
26         WHEN r_score = 4 AND f_score = 4 AND m_score = 4 THEN 'Champions
27         (VIP nhat)'
28         WHEN f_score >= 3 AND m_score >= 3 THEN 'Loyal Customers (Trung
29         thanh)'
30         WHEN r_score = 1 THEN 'At Risk (Nguy co mat khach)'
31         ELSE 'General (Thong thuong)'
32     END AS customer_segment
33 FROM rfm_scores;

```

5.5 Bài Tập Tự Luyện Level 2

Bài Tập Tự Luyện: 5 Bài Tập SQL Phỏng Vấn Sát Thủ

1. **Bài 1 (Căn bản):** Viết câu lệnh tạo bảng `dim_products` với các cột: `product_id` (Khóa chính), `product_name` (Bắt buộc), `price` (Kiểm tra phải > 0), và `category`.
2. **Bài 2 (Gom nhóm):** Viết câu lệnh truy vấn tìm ra các danh mục (`category`) có từ 5 sản phẩm trở lên và có giá bán trung bình lớn hơn 100 USD.
3. **Bài 3 (Top N Per Group):** Viết câu lệnh lấy ra Top 2 đơn hàng có giá trị cao nhất của TỪNG khách hàng bằng `DENSE_RANK()`.
4. **Bài 4 (Tăng trưởng):** Viết câu lệnh sử dụng `LAG()` để tính chênh lệch doanh thu tuyệt đối giữa đơn hàng hiện tại và đơn hàng liền trước đó của từng khách hàng.
5. **Bài 5 (Index Optimization):** Phân tích vì sao truy vấn sau bị chạy chậm (Sequential Scan): `SELECT * FROM fact_orders WHERE LOWER(customer_id) = 'cust_101'`; và đưa ra giải pháp sửa đổi.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Hướng Dẫn Kiểm Tra

• Bài 1:

```
1 CREATE TABLE dim_products (  
2     product_id VARCHAR(50) PRIMARY KEY,  
3     product_name VARCHAR(150) NOT NULL,  
4     price NUMERIC(10, 2) CHECK (price > 0),  
5     category VARCHAR(50)  
6 );  
7
```

• Bài 2:

```
1 SELECT category, COUNT(product_id) AS prod_count, AVG(price) AS  
   avg_price  
2 FROM dim_products GROUP BY category  
3 HAVING COUNT(product_id) >= 5 AND AVG(price) > 100.0;  
4
```

• Bài 3:

```
1 WITH ranked_orders AS (  
2     SELECT order_id, customer_id, order_amount,  
3           DENSE_RANK() OVER(PARTITION BY customer_id ORDER BY  
   order_amount DESC) AS rnk  
4     FROM fact_orders_dwh  
5 )
```

```
6 SELECT * FROM ranked_orders WHERE rnk <= 2;  
7
```

- **Bài 4:**

```
1 SELECT order_id, customer_id, order_date, order_amount,  
2       order_amount - LAG(order_amount, 1) OVER(PARTITION BY  
3       customer_id ORDER BY order_date) AS diff_prev  
4 FROM fact_orders_dwh;
```

- **Bài 5:** Do bọc hàm LOWER() quanh cột, chỉ mục thông thường bị vô hiệu hóa. Giải pháp: Chuẩn hóa dữ liệu viết thường ngay từ khi nạp (không cần bọc hàm lúc query), hoặc tạo một Functional Index: `CREATE INDEX idx_lower_cust ON fact_orders (LOWER(customer_id));`.

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Kỹ Năng SQL Vào CV

E-Commerce SQL Analytics Database | PostgreSQL, Schema Design, Window Functions

- Thiết kế kiến trúc cơ sở dữ liệu quan hệ chuẩn hóa 3NF trên PostgreSQL gồm 5 bảng, tối ưu hóa các chỉ mục B-Tree và Composite Indexes giúp giảm 70% thời gian quét đĩa.
- Xây dựng hơn 40 truy vấn phân tích nghiệp vụ phức tạp sử dụng CTEs và Window Functions để thực hiện phân tích giữ chân khách hàng (Cohort Retention) và phân khúc khách hàng (RFM Segmentation).

Chương 6

Level 3: Thống Kê Kinh Doanh & Thử Nghiệm A/B Testing (Tuần 15–18)

Kiến Thức Cốt Lõi: Mục Tiêu & Tiền Đề Cần Nhớ

Tiền đề bắt buộc: Bạn chỉ cần đã hoàn thành Chương 3 (Lập trình Python và Pandas cơ bản). **Không cần kiến thức toán cao cấp hay thống kê trước đó!**

Mục tiêu chương này:

- Hiểu từ con số 0 các loại dữ liệu thống kê: Định tính (Nominal, Ordinal) và Định lượng (Discrete, Continuous).
- Tự tay tính toán và hiểu bản chất trực quan của Mean, Median, Mode, Phương sai (Variance) và Độ lệch chuẩn (Standard Deviation).
- Nắm vững Định lý Giới Hạn Trung Tâm (CLT) và Phân phối chuẩn (Normal Distribution).
- Hiểu căn cơ Thống kê suy luận: p-value, Khoảng tin cậy 95%, Sai lầm Loại I (α) và Loại II (β).
- Tự tay thiết kế và phân tích thử nghiệm A/B Testing hoàn chỉnh (**Project 3**) bằng Python.

6.1 Thống Kê Mô Tả Từ Con Số 0

6.1.1 1. Phân Loại Dữ Liệu Trong Thống Kê

Mọi dữ liệu bạn gặp trong doanh nghiệp đều thuộc một trong hai nhóm:

1. Dữ liệu định tính (Categorical Data - Dữ liệu chữ / phân loại):

- *Định danh (Nominal)*: Không có thứ tự hơn kém. Ví dụ: Giới tính (Nam, Nữ), Thành phố (Hà Nội, Sài Gòn), Phương thức thanh toán (Thẻ, Ví điện tử).

- *Thứ bậc (Ordinal)*: Có thứ tự cao thấp rõ ràng. Ví dụ: Hạng khách hàng (Đồng, Bạc, Vàng, Kim Cương), Đánh giá sao (1 sao đến 5 sao).

2. Dữ liệu định lượng (Numerical Data - Dữ liệu số tính toán được):

- *Rời rạc (Discrete)*: Đếm được bằng số nguyên. Ví dụ: Số lượng đơn hàng (1, 2, 3 đơn), Số lượt nhấp chuột.
- *Liên tục (Continuous)*: Đo lường được trên trục số thực liên tục. Ví dụ: Doanh thu (\$125.50), Thời gian tải trang web (1.24 giây), Cân nặng.

6.1.2 2. Mean, Median, Mode & Ví Dụ Tính Bằng Tay

Giả sử một cửa hàng có 5 đơn hàng với giá trị: [10, 12, 15, 18, 95] (USD):

- **Giá trị trung bình (Mean)**:

$$\bar{x} = \frac{10 + 12 + 15 + 18 + 95}{5} = \frac{150}{5} = 30 \text{ USD}$$

- **Giá trị trung vị (Median)**: Sắp xếp các số theo thứ tự tăng dần và lấy số nằm chính giữa. Ở đây, giá trị trung vị là **15 USD**.
- **Nhận xét sống còn**: Đơn hàng cá biệt 95 USD (Outlier) đã kéo mức trung bình lên 30 USD (gấp đôi mức chỉ tiêu thông thường của 80% khách hàng!). Nếu báo cáo với giám đốc là "*Khách hàng chi trung bình 30 USD*", chiến lược giá của công ty sẽ bị định giá quá cao và thất bại! Do đó, ****với dữ liệu lệch, luôn luôn dùng Median****.

6.1.3 3. Phương Sai (Variance) & Độ Lệch Chuẩn (Standard Deviation)

Để biết các con số phân tán rải rác hay tập trung sát quanh giá trị trung bình, ta dùng:

- **Phương sai mẫu (s^2)**: Trung bình cộng của bình phương các khoảng cách từ từng điểm dữ liệu đến giá trị trung bình:

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}$$

- **Độ lệch chuẩn ($s = \sqrt{s^2}$)**: Căn bậc hai của phương sai, đưa độ phân tán về cùng đơn vị đo ban đầu (USD). Nếu s nhỏ: doanh thu các ngày rất đều đặn; nếu s lớn: doanh thu trời sập thất thường, rủi ro cao.

6.2 Xác Suất & Định Lý Giới Hạn Trung Tâm (CLT)

Kiến Thức Cốt Lõi: Định Lý Giới Hạn Trung Tâm (Central Limit Theorem - CLT)

Cho dù tổng thể dữ liệu ban đầu có bất kỳ hình dạng phân phối nào (lệch, nhị thức, hay đồng đều), nếu ta lấy các mẫu ngẫu nhiên độc lập có kích thước đủ lớn ($n \geq 30$) từ tổng thể đó, thì **phân phối của các giá trị trung bình mẫu ($\bar{X}$) sẽ xấp xỉ phân phối chuẩn (Normal Distribution)**:

$$\bar{X} \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$$

Ý nghĩa sống còn: CLT là nền tảng toán học cho phép chúng ta ước lượng khoảng tin cậy (Confidence Interval) và thực hiện các kiểm định thống kê giả thuyết (như Z-test, T-test) trong A/B Testing mà không cần quan tâm phân phối ban đầu của từng người dùng là gì!

6.3 Thống Kê Suy Luận & Kiểm Định Giả Thuyết (Hypothesis Testing)

Mọi thử nghiệm A/B Testing đều bắt đầu từ việc thiết lập cặp giả thuyết:

- **Giả thuyết vô hiệu (H_0):** "Không có sự khác biệt có ý nghĩa thống kê giữa nhóm Control (A) và nhóm Treatment (B)".
- **Giả thuyết đối (H_1):** "Nhóm Treatment (B) tạo ra sự khác biệt có ý nghĩa thống kê so với nhóm Control (A)".

6.3.1 Sai Lầm Loại I (α), Loại II (β) & Ý Nghĩa p-value

Thực Tế / Quyết Định	Chấp Nhận H_0 (Không đổi giao diện)	Bác Bỏ H_0 (Ra mắt giao diện mới)
H_0 Thực Sự Đúng	Quyết định chính xác ($1 - \alpha$)	Sai Lầm Loại I (α - False Positive): Báo động giả! Tưởng tính năng mới tốt nhưng thực ra chỉ là do may rủi.
H_0 Thực Sự Sai	Sai Lầm Loại II (β - False Negative): Bỏ lỡ cơ hội! Tính năng mới thực sự hiệu quả nhưng không phát hiện ra.	Quyết định chính xác: Statistical Power ($1 - \beta$) (Độ nhạy thử nghiệm).

Định nghĩa chuẩn của p-value: p -value là xác suất quan sát thấy sự chênh lệch lớn như trong thí nghiệm (hoặc cực đoan hơn), **giả định rằng giả thuyết vô hiệu H_0 là đúng**. Nếu $p\text{-value} < \alpha$ (thường là 0.05), ta có đủ bằng chứng để bác bỏ H_0 .

6.4 A/B Testing Thực Chiến: Sample Size & Cạm Bẫy SRM

Trước khi bấm nút bắt đầu thử nghiệm, bạn phải tính toán kích thước mẫu tối thiểu (Sample Size) cần thu thập:

$$n = \frac{2 \times (Z_{1-\alpha/2} + Z_{1-\beta})^2 \times p(1-p)}{\Delta^2}$$

Trong đó: Δ là MDE (Minimum Detectable Effect - mức chênh lệch nhỏ nhất bạn muốn phát hiện), p là tỷ lệ chuyển đổi hiện tại (Baseline Conversion Rate), $Z_{1-\alpha/2} = 1.96$ (với $\alpha = 0.05$), và $Z_{1-\beta} = 0.84$ (với Power = 80%).

Thực Chiến & Best Practices: Cạm Bẫy SRM (Sample Ratio Mismatch)

Nếu bạn cấu hình chia traffic 50:50 cho nhóm Control và Treatment, nhưng khi kết thúc thử nghiệm nhận được: Control = 10,000 người, Treatment = 8,900 người. **ĐỪNG VỘI ĐỌC P-VALUE CỦA TỶ LỆ CHUYỂN ĐỔI!** Sự lệch tỷ lệ này báo hiệu lỗi phân luồng kỹ thuật (Bot filtering, crash app ở nhóm B, hoặc lỗi caching). Bạn phải chạy kiểm định Chi-Square goodness-of-fit để kiểm tra SRM:

```
1 from scipy.stats import chisquare
2
3 observed = [10000, 8900]
4 expected = [9450, 9450]
5 stat, p_val = chisquare(observed, f_exp=expected)
6
7 if p_val < 0.001:
8     print("[!] NGHAY LAP TUC HUY KET QUA: Phat hien loi nghiem trong
      SRM (Sample Ratio Mismatch)!")
```

6.5 PROJECT 3: Thử Nghiệm A/B Testing Tỷ Lệ Chuyển Đổi E-Commerce

6.5.1 Kịch Bản Nghiệp Vụ & Triển Khai Phân Tích

Đội ngũ Product vừa thiết kế lại trang Thanh toán (Checkout Page) nhằm tăng tỷ lệ chuyển đổi mua hàng (Conversion Rate). Dữ liệu thu thập sau 14 ngày gồm:

- Nhóm Control: 25,000 lượt truy cập, 3,000 đơn hàng thành công (CR = 12.0%).
- Nhóm Treatment: 25,000 lượt truy cập, 3,250 đơn hàng thành công (CR = 13.0%).

6.5.2 Mã Nguồn Kiểm Định Thống Kê Bằng Python (SciPy & Statsmodels)

```
1 import numpy as np
2 from statsmodels.stats.proportion import proportions_ztest,
  confint_proportions_2indep
3
4 # 1. Khai bao thong so thuc nghiem
```

```

5 conversions = np.array([3250, 3000]) # [Treatment, Control]
6 total_users = np.array([25000, 25000])
7
8 cr_treatment = conversions[0] / total_users[0]
9 cr_control = conversions[1] / total_users[1]
10 absolute_lift = cr_treatment - cr_control
11 relative_lift = (absolute_lift / cr_control) * 100
12
13 print(f"[*] Baseline CR (Control)      : {cr_control:.4f} (12.0%)")
14 print(f"[*] Treatment CR                : {cr_treatment:.4f} (13.0%)")
15 print(f"[+] Relative Uplift              : +{relative_lift:.2f}%\n")
16
17 # 2. Thuc hien Two-proportion Z-Test (Hai phia, alpha = 0.05)
18 z_stat, p_value = proportions_ztest(count=conversions, nobs=total_users,
19                                     alternative='two-sided')
20
21 # 3. Tinh Khoang tin cay 95% cho muc chenh lech ty le
22 ci_low, ci_upp = confint_proportions_2indep(
23     count1=conversions[0], nobs1=total_users[0],
24     count2=conversions[1], nobs2=total_users[1],
25     method='wald', alpha=0.05
26 )
27
28 print(f"[*] Z-statistic                  : {z_stat:.4f}")
29 print(f"[*] P-value                        : {p_value:.6f}")
30 print(f"[*] 95% Confidence Interval         : [{ci_low*100:.3f}%,
31     {ci_upp*100:.3f}%]")
32
33 # 4. Dua ra ket luan kinh doanh
34 if p_value < 0.05:
35     print("\n[V] KET LUAN: Chenh lech co y nghia thong ke (Statistically
36         Significant!)")
37     print(f"    Giao dien moi giup tang CR it nhat {ci_low*100:.2f}% va
38         nhieu nhat {ci_upp*100:.2f}%.")
39     print(f"    KHUYEN NGHI KINH DOANH: Trien khai 100% giao dien moi cho
40         toan bo nguoi dung.")
41 else:
42     print("\n[X] KET LUAN: Khong du bang chung thong ke de bac bo H0.
43         Khong nen rollout.")

```

6.6 Bài Tập Tự Luyện Level 3

Bài Tập Tự Luyện: 5 Bài Tập Thống Kê & Thiết Kế Thử Nghiệm Thực Tế

- Bài 1 (Quy tắc 68-95-99.7):** Thời gian phản hồi của API thanh toán tuân theo phân phối chuẩn với giá trị trung bình $\mu = 200\text{ms}$ và độ lệch chuẩn $\sigma = 30\text{ms}$. Có bao nhiêu phần trăm yêu cầu sẽ mất hơn 260ms để xử lý?
- Bài 2 (Định lý Bayes):** Giả sử tỷ lệ gian lận thẻ tín dụng trên hệ thống là 0.1% ($P(\text{Fraud}) = 0.001$). Một mô hình AI phát hiện gian lận có độ nhạy (True Positive Rate) là 95% và tỷ lệ báo động giả (False Positive Rate) là 1%. Nếu hệ thống báo một giao dịch là "Gian lận", xác suất thực tế giao dịch đó đúng là gian lận là bao nhiêu?
- Bài 3 (Simpson's Paradox):** Tìm hiểu nghịch lý Simpson trong phân tích dữ liệu. Hãy đưa ra một ví dụ thực tế về tỷ lệ chuyển đổi khi gộp chung hai nhóm người dùng Desktop và Mobile thì nhóm B thắng nhóm A, nhưng khi tách riêng từng thiết bị thì nhóm A lại thắng nhóm B ở cả hai thiết bị!
- Bài 4 (The Peeking Problem):** Giải thích tại sao việc một Data Analyst kiểm tra giá trị p -value mỗi ngày một lần trong suốt 14 ngày thử nghiệm và dừng lại ngay khi $p < 0.05$ lại làm sai lầm Loại I thực tế tăng từ 5% lên tới gần 30%?
- Bài 5 (Tính Kích Thước Mẫu):** Viết hàm Python tính toán số lượng mẫu tối thiểu cần thiết cho mỗi biến thể khi biết: Baseline Conversion Rate = 8%, MDE mong muốn phát hiện = 10% tương đối (tức là tăng lên 8.8%), với $\alpha = 0.05$ và Power = 80%.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- Bài 1:** $260\text{ms} = \mu + 2\sigma$. Theo quy tắc phân phối chuẩn, có 95% dữ liệu nằm trong $[\mu - 2\sigma, \mu + 2\sigma]$. Do đó phần đuôi vượt quá $+2\sigma$ chiếm $(100\% - 95\%)/2 = 2.5\%$.

- Bài 2:** Áp dụng định lý Bayes:

$$P(\text{Fraud}|\text{Alert}) = \frac{P(\text{Alert}|\text{Fraud})P(\text{Fraud})}{P(\text{Alert})} = \frac{0.95 \times 0.001}{(0.95 \times 0.001) + (0.01 \times 0.999)} \approx 8.69\%$$

Kết quả gây kinh ngạc: Dù AI chính xác 95%, khi có báo động, khả năng đúng chỉ là 8.69% vì tỷ lệ gian lận cơ sở cực kỳ hiếm (Base Rate Fallacy)!

- Bài 3:** Nghịch lý Simpson xảy ra do biến gây nhiễu (Confounding variable - ở đây là loại thiết bị) phân bố không đều giữa 2 nhóm. Nếu nhóm B nhận nhiều traffic Mobile hơn (vốn có tỷ lệ chuyển đổi tự nhiên thấp hơn Desktop), tổng kết quả bị kéo tụt xuống.
- Bài 4:** Đây là vấn đề "Multiple Hypothesis Testing". Mỗi lần peeking tương đương một lần tung xúc xắc độc lập. Xác suất tích lũy xuất hiện ít nhất một lần $p < 0.05$ ngẫu nhiên qua 14 ngày tăng vọt theo công thức $1 - (1 - 0.05)^{14} \approx 51\%$!

- **Bài 5:** Dùng `statsmodels.stats.power.NormalIndPower().solve_power`. Kích thước mẫu cần khoảng 47,500 người dùng cho MỖI biến thể (tổng cộng 95,000 người dùng).

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Dự Án Thống Kê & A/B Testing Vào CV

E-Commerce Conversion Rate Optimization via A/B Testing | Python, SciPy, Experimentation

- Thiết kế và phân tích thử nghiệm A/B Testing quy mô 50,000 người dùng nhằm đánh giá hiệu quả của quy trình thanh toán mới, đảm bảo các tiêu chuẩn kiểm tra Sample Ratio Mismatch (SRM).
- Thực hiện kiểm định tỷ lệ Two-proportion Z-test và ước lượng khoảng tin cậy 95%, chứng minh tính năng mới giúp nâng tỷ lệ chuyển đổi thêm +8.3% với độ tin cậy thống kê cao ($p = 0.004$).
- Biên soạn báo cáo phân tích rủi ro và khuyến nghị kinh doanh, loại trừ các ảnh hưởng của nghịch lý Simpson và hiệu ứng Novelty Effect trước khi triển khai 100%.

Chương 7

Level 4: Data Analyst & Business Intelligence Chuyên Nghiệp (Tuần 19–24)

7.1 Excel Nâng Cao: Tự Động Hóa Với Power Query & Dynamic Arrays

Nhiều người mới thường xem thường Excel và cho rằng chỉ có Python/SQL mới là "dân chuyên". Tuy nhiên, trong thực tế doanh nghiệp, hơn 80% các cuộc họp chiến lược của ban giám đốc vẫn xoay quanh các bảng tính Excel. Kỹ năng Excel chuyên nghiệp giúp bạn giải quyết nhanh các bài toán ad-hoc chỉ trong 5 phút thay vì mất nửa ngày viết script:

Thực Chiến & Best Practices: Bộ Công Thức Hiện Đại Thay Thế VLOOKUP Cũ Kỹ

- **XLOOKUP** (Thay thế hoàn toàn VLOOKUP và HLOOKUP):
`=XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found], [match_mode])`. Không bị giới hạn tìm kiếm từ trái sang phải, không bị lỗi khi chèn thêm cột mới.
- **Dynamic Array Formulas**:
 - Lọc dữ liệu động: `=FILTER(A2:D100, (C2:C100="Tech") * (D2:D100>500), "Không có dữ liệu")`
 - Lấy danh sách duy nhất và sắp xếp: `=SORT(UNIQUE(B2:B100))`
- **Power Query (ETL trong Excel)**: Kết nối trực tiếp vào thư mục chứa 12 file doanh thu hàng tháng, tự động gộp bảng, unpivot cột ngày tháng và làm sạch dữ liệu chỉ bằng một nút bấm "Refresh All".

7.2 Power BI & Nghệ Thuật Mô Hình Hóa Dữ Liệu (Data Modeling)

Một báo cáo Power BI chạy chậm chạp hoặc ra số sai lệch 99% là do mô hình hóa dữ liệu cầu thả, không phải do giao diện hiển thị!

Kiến Thức Cốt Lõi: Mô Hình Star Schema (Mô Hình Ngôi Sao)

Trong Business Intelligence, luôn luôn tái cấu trúc dữ liệu về dạng **Star Schema**:

- **Bảng Sự Kiện (Fact Table - Ở trung tâm)**: Chứa các số đo định lượng (Measures) phát sinh liên tục theo thời gian: `fact_sales`, `fact_transactions`. Chứa các khóa ngoại (Foreign Keys) và số liệu (Số lượng, Đơn giá, Chiết khấu).
- **Bảng Chiều/Danh Mục (Dimension Tables - Xung quanh cánh sao)**: Chứa thông tin ngữ cảnh để lọc và cắt lát dữ liệu: `dim_customer`, `dim_product`, `dim_date`, `dim_store`.
- **Nguyên tắc kết nối**: Mỗi quan hệ luôn là **1-Nhiều (1 – *)**, với hướng lọc một chiều (Single Cross-Filter Direction) từ Dimension Table sang Fact Table.
- **Tuyệt đối tránh hướng lọc hai chiều (Both)**: Lọc hai chiều gây nhập nhằng quan hệ (Ambiguous paths) và làm suy giảm nghiêm trọng hiệu năng của động cơ VertiPaq.

7.3 Ngôn Ngữ DAX Chuyên Sâu: Filter Context & Time Intelligence

DAX (Data Analysis Expressions) không phải là Excel Formula. DAX hoạt động dựa trên hai khái niệm trừu tượng: ****Row Context**** (ngữ cảnh dòng) và ****Filter Context**** (ngữ cảnh bộ lọc).

Kiến Thức Cốt Lõi: Hàm Vạn Năng CALCULATE() & Chuyển Đổi Ngữ Cảnh

Hàm `CALCULATE()` là hàm duy nhất trong DAX có khả năng ****sửa đổi, ghi đè hoặc mở rộng Filter Context hiện tại****:

`CALCULATE(<Expression>, <Filter_1>, <Filter_2>, ...)`

Khi `CALCULATE` được gọi, nó thực hiện:

1. Đánh giá các bộ lọc mới được truyền vào.
2. Thay thế hoặc gộp các bộ lọc này vào Filter Context ban đầu của visual.
3. Chuyển đổi mọi Row Context hiện hữu thành Filter Context tương đương (**Context Transition**).

7.3.1 Các Công Thức DAX Kinh Điển Chuẩn Doanh Nghiệp

```

1 // 1. Tổng Doanh Thu Cơ Bản (Explicit Measure)
2 Total Sales = SUM(fact_sales[sales_amount])
3
4 // 2. Doanh Thu Cùng Kỳ Năm Trước (Same Period Last Year)
5 Sales SPLY =
6 CALCULATE(

```

```

7      [Total Sales],
8      SAMEPERIODLASTYEAR(dim_date[Date])
9  )
10
11 // 3. Tỷ Lệ Tăng Trưởng Doanh Thu So Với Năm Trước (% YoY Growth)
12 YoY Sales Growth % =
13 VAR CurrentSales = [Total Sales]
14 VAR PreviousSales = [Sales SPLY]
15 RETURN
16 DIVIDE(CurrentSales - PreviousSales, PreviousSales, 0.0)
17
18 // 4. Doanh Thu Lũy Kế Từ Đầu Năm (Year-To-Date)
19 Sales YTD =
20 TOTALYTD([Total Sales], dim_date[Date])
21
22 // 5. Tỷ Trọng Doanh Thu So Với Toàn Bộ Danh Mục (% of All Category)
23 Sales Contribution % =
24 DIVIDE(
25     [Total Sales],
26     CALCULATE([Total Sales], ALL(dim_product[Category])),
27     0.0
28 )

```

7.4 Tư Duy Thiết Kế Dashboard: KPI Architecture Chuẩn Doanh Nghiệp

Thực Chiến & Best Practices: Mô Hình Báo Cáo 3 Tầng Cho Lãnh Đạo

Đừng bao giờ nhồi nhét mọi biểu đồ lên một trang duy nhất. Hãy tổ chức cấu trúc báo cáo theo 3 tầng kim tự tháp:

1. Tầng 1: Executive Dashboard (Dành cho C-Level - CEO, CFO):

- Chỉ hiển thị 4–6 thẻ KPI cốt lõi nhất: Doanh thu, Lợi nhuận gộp, Số lượng khách hàng mới, CAC, LTV.
- Luôn đi kèm so sánh với Target (Mục tiêu) và Kỳ trước (MoM, YoY) qua màu sắc trực quan (Xanh/Đỏ).

2. Tầng 2: Tactical Dashboard (Dành cho Trưởng phòng - Marketing/Sales Manager):

- Cắt lát theo chiều: Kênh bán hàng (Online vs Offline), Vùng địa lý, Nhóm sản phẩm.
- Phân tích xu hướng biến động theo tuần/tháng.

3. Tầng 3: Operational Detail (Dành cho Chuyên viên vận hành):

- Bảng chi tiết từng đơn hàng, danh sách khách hàng có nguy cơ rời bỏ (Churn Risk Table), hỗ trợ xuất Excel để xử lý tức thời.

7.5 Bí Kíp AI Copilot: Sinh Công Thức DAX & Tạo Data Dictionary

Bí Kíp AI Copilot & Prompt Engineering: Prompt Tạo DAX Phức Tạp & Viết KPI Dictionary

"Đóng vai trò là Microsoft Certified Fabric & Power BI Solution Architect. Tôi có mô hình Star Schema gồm: - Fact Table: 'fact_orders(order_id, customer_id, order_date, revenue)' - Dimension Tables: 'dim_date(date, year, month, quarter)', 'dim_customers(customer_id, segment)'. Hãy viết các mã DAX chuẩn mực sau: 1. 'New Customer Revenue': Tính tổng doanh thu chỉ đến từ các khách hàng thực hiện đơn hàng đầu tiên trong tháng được chọn trên bộ lọc. 2. 'Churned Customers Count': Đếm số lượng khách hàng đã từng mua hàng trong quá khứ nhưng không có đơn hàng nào trong 90 ngày tính đến ngày lọc cuối cùng. Yêu cầu: Sử dụng biến 'VAR ... RETURN' để code trong sáng, tối ưu hiệu năng bộ nhớ và giải thích rõ cách thức hoạt động của Filter Context trong từng công thức."

7.6 MỐC ỨNG TUYỂN 1: Bắt Đầu Apply Data Analyst Intern / Fresher!

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Chiến Lược Nộp Hồ Sơ Tại Tuần 24

Đến thời điểm này (Tuần 24), bạn đã trang bị đủ 100% các kỹ năng bắt buộc của một ****Data Analyst****:

- **Kỹ năng:** SQL nâng cao, Python/Pandas EDA, Thống kê & A/B Testing, Excel Power Query, Power BI & DAX.
- **Hồ sơ GitHub:** Đã có 4 dự án thực chiến (Data CLI, E-commerce EDA, SQL Analytics DB, Power BI Executive Dashboard).

HÀNH ĐỘNG NGAY: Đừng chờ đợi đến khi học xong Machine Learning hay Big Data mới xin việc! Hãy hoàn thiện CV 1 trang theo mẫu ở Chương 14, tạo tài khoản trên ITviec, LinkedIn, TopCV và nộp đơn vào các vị trí:

- *Data Analyst Intern / Trainee*
- *Junior BI Developer / Business Analyst (Technical)*
- *Data Operations Specialist*

Việc vừa đi làm thực tế tại doanh nghiệp vừa học tiếp các chương về Data Engineering và Machine Learning sẽ giúp bạn tiến bộ nhanh gấp 3 lần so với chỉ tự học ở nhà!

7.7 Bài Tập Tự Luyện Level 4

Bài Tập Tự Luyện: 5 Bài Tập Mô Hình Hóa & Viết DAX Doanh Nghiệp

- Bài 1 (Date Table):** Viết câu lệnh DAX khởi tạo một bảng lịch `dim_date` đầy đủ các cột: Date, Year, Month Number, Month Name, Quarter, Year-Month từ ngày 01/01/2023 đến 31/12/2026.
- Bài 2 (DAX - SUM vs SUMX):** Phân tích sự khác biệt về bản chất giữa `SUM(fact_sales[amount])` và `SUMX(fact_sales, fact_sales[quantity] * fact_sales[unit_price])`. Khi nào bắt buộc phải dùng SUMX?
- Bài 3 (DAX - Moving Average):** Viết measure DAX tính toán trung bình động doanh thu 30 ngày gần nhất (30-Day Rolling Revenue) dựa trên bảng ngày chuẩn.
- Bài 4 (Data Modeling):** Một doanh nghiệp có 2 bảng Fact: `fact_actual_sales` (ghi nhận doanh số thực tế theo ngày) và `fact_sales_target` (ghi nhận mục tiêu doanh số theo tháng). Hãy thiết kế mô hình quan hệ chuẩn Star Schema để có thể so sánh Actual vs Target trên cùng một biểu đồ Power BI mà không nối trực tiếp 2 bảng Fact với nhau.
- Bài 5 (DAX - Cumulative Total):** Viết công thức DAX tính tổng doanh thu tích lũy từ ngày đầu tiên thành lập công ty cho đến ngày hiện tại đang được chọn (Running Total).

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- Bài 1:**

```

1 dim_date =
2 ADDCOLUMNS(
3     CALENDAR(DATE(2023, 1, 1), DATE(2026, 12, 31)),
4     "Year", YEAR([Date]),
5     "MonthNum", MONTH([Date]),
6     "MonthName", FORMAT([Date], "mmmm"),
7     "Quarter", "Q" & FORMAT([Date], "q"),
8     "YearMonth", FORMAT([Date], "yyyy-mm")
9 )
10

```

- Bài 2:** SUM chỉ cộng dồn một cột đã có sẵn trong bảng. SUMX là hàm lặp (Iterator), nó tạo ra một Row Context ảo duyệt từng dòng, nhân `quantity * unit_price` rồi mới cộng tổng, giúp tiết kiệm bộ nhớ RAM vì không cần tạo thêm cột vật lý trong bảng dữ liệu.

- Bài 3:**

```

1 30D Rolling Sales =
2 CALCULATE(

```

```
3     [Total Sales],
4     DATESINPERIOD(dim_date[Date], MAX(dim_date[Date]), -30, DAY)
5 )
6
```

- **Bài 4:** Sử dụng mô hình **Shared Dimensions (Chiều Dùng Chung)**: Tạo 2 bảng Dimension dùng chung là `dim_date` và `dim_product`. Cả 2 bảng Fact `fact_actual_sales` và `fact_sales_target` đều nối quan hệ 1 – * với các bảng Dimension dùng chung này.

- **Bài 5:**

```
1 Running Total =
2 CALCULATE(
3     [Total Sales],
4     FILTER(ALL(dim_date[Date]), dim_date[Date] <=
5         MAX(dim_date[Date]))
6 )
```

Chương 8

Level 5: Data Engineering – Data Modeling & Core ETL (Tuần 25–30)

8.1 Nền Tảng Data Warehousing: So Sánh Toàn Diện OLTP vs OLAP

Khi khối lượng dữ liệu doanh nghiệp vượt qua hàng chục triệu bản ghi, việc chạy các truy vấn phân tích trực tiếp trên cơ sở dữ liệu vận hành (Production OLTP) sẽ làm tê liệt toàn bộ hệ thống bán hàng. Đây là lý do kiến trúc Kho Dữ Liệu (**Data Warehouse - OLAP**) ra đời:

Tiêu Chí	Hệ Thống OLTP (Giao dịch)	Hệ Thống OLAP (Phân tích)
Mục Đích	Hỗ trợ vận hành hàng ngày (Thêm giỏ hàng, đặt đơn, chuyển khoản)	Phục vụ báo cáo chiến lược, phân tích xu hướng, dự báo kinh doanh
Bản Chất Dữ Liệu	Dữ liệu thời gian thực hiện tại, thay đổi liên tục từng giây	Dữ liệu lịch sử tích lũy nhiều năm, bất biến (Immutable), ghi một lần đọc nhiều lần
Mô Hình Dữ Liệu	Chuẩn hóa cao độ (3NF) để tránh dư thừa và tối ưu hóa ghi (Write-optimized)	Phi chuẩn hóa (Star/Snowflake Schema) để tối ưu hóa đọc và kết nối (Read-optimized)
Khối Lượng Thao Tác	Hàng triệu giao dịch nhỏ mỗi giây, mỗi giao dịch chỉ sửa 1–2 dòng	Hàng nghìn truy vấn phức tạp mỗi ngày, mỗi truy vấn quét hàng triệu bản ghi
Công Nghệ Tiêu Biểu	PostgreSQL, MySQL, SQL Server, Oracle	Snowflake, Google BigQuery, ClickHouse, AWS Redshift, DuckDB

8.2 Mô Hình Hóa Chiều Kimball (Kimball Dimensional Modeling)

Phương pháp tiếp cận của Ralph Kimball là tiêu chuẩn vàng của ngành kỹ thuật dữ liệu toàn cầu trong hơn 30 năm qua:

Kiến Thức Cốt Lõi: 4 Bước Thiết Kế Mô Hình Chiều Chuẩn Mực

1. **Bước 1: Chọn Quy Trình Kinh Doanh (Business Process):** Ví dụ: Giao dịch bán hàng, Sự kiện nhập chuôt, Quá trình giao vận.
2. **Bước 2: Tuyên Bố Độ Mịn Của Dữ Liệu (Declare the Grain):** Định nghĩa chính xác một dòng trong bảng Fact đại diện cho cái gì? Ví dụ: *"Mỗi dòng là một mặt hàng cụ thể trong một đơn hàng được thanh toán thành công"*.
3. **Bước 3: Xác Định Các Chiều (Identify Dimensions):** Bối cảnh "Ai, Cái gì, Ở đâu, Khi nào, Tại sao?". Ví dụ: Khách hàng, Sản phẩm, Chi nhánh, Ngày tháng, Khuyến mãi.
4. **Bước 4: Xác Định Các Số Đo (Identify Facts):** Các giá trị số định lượng có thể cộng trừ được: Doanh thu, Số lượng, Tiền chiết khấu, Phí ship.

8.2.1 Xử Lý Chiều Biến Đổi Chậm (Slowly Changing Dimensions - SCD)

Khi khách hàng Nguyễn Văn A chuyển địa chỉ từ Hà Nội vào TP. Hồ Chí Minh, hệ thống Data Warehouse xử lý lịch sử như thế nào?

- **SCD Type 1 (Ghi đè):** Cập nhật đè trực tiếp Hà Nội thành TP. Hồ Chí Minh. *Hậu quả:* Mất toàn bộ dấu vết lịch sử; các đơn hàng năm ngoái của ông A tại Hà Nội nay bị tính nhầm cho TP. Hồ Chí Minh!
- **SCD Type 2 (Thêm dòng mới - Tiêu chuẩn ngành):** Giữ nguyên dòng cũ, đánh dấu `is_current = FALSE` và `valid_to = '2026-03-01'`. Tạo thêm một dòng mới với Surrogate Key mới, địa chỉ là TP. Hồ Chí Minh, `is_current = TRUE` và `valid_from = '2026-03-01'`. Bảo cáo lịch sử được bảo toàn tuyệt đối!

8.3 Kiến Trúc ETL Hiện Đại & Tính Lũy Đẳng (Idempotency)**Kiến Thức Cốt Lõi: Nguyên Tắc Sống Còn Của Data Pipeline: Tính Lũy Đẳng (Idempotency)**

Một Data Pipeline được gọi là **Idempotent (Lũy đẳng)** nếu việc thực thi lại đường ống đó 1 lần hay 10 lần đối với cùng một khoảng thời gian dữ liệu (ví dụ: ngày 28/09/2026) đều tạo ra **chính xác một trạng thái duy nhất trong Kho Dữ Liệu**, không gây trùng lặp bản ghi và không làm sai lệch số liệu!

Cách đạt được tính lũy đẳng:

1. **Chiến lược Delete-Insert / Partition Overwrite:** Trước khi nạp dữ liệu ngày *T*, tự động xóa sạch dữ liệu ngày *T* cũ trong bảng đích:

```
1 DELETE FROM fact_daily_sales WHERE sales_date = '2026-09-28';
2 INSERT INTO fact_daily_sales SELECT ... FROM staging_sales WHERE
  sales_date = '2026-09-28';
```

3

2. **Chiến lược Upsert (Merge):** Sử dụng câu lệnh `ON CONFLICT (primary_key) DO UPDATE` trong PostgreSQL hoặc `MERGE INTO` trong BigQuery/Snowflake.

8.4 PROJECT 5: Automated Batch ETL Pipeline Chuẩn Production

8.4.1 Bài Toán Thực Tế

Xây dựng một đường ống ETL bằng Python chạy tự động mỗi đêm: Trích xuất giao dịch từ một REST API giả lập và tệp CSV vận hành, xác thực dữ liệu chặt chẽ bằng thư viện **Pydantic**, làm sạch và nạp lũy đăng vào bảng Fact trong PostgreSQL Data Warehouse.

8.4.2 Mã Nguồn Pipeline Hoàn Chỉnh

```

1  import os
2  import logging
3  from datetime import datetime
4  from typing import List, Optional
5  import pandas as pd
6  from pydantic import BaseModel, Field, ValidationError
7  import psycopg2
8  from psycopg2.extras import execute_batch
9
10 # 1. Cấu hình Logging chuyên nghiệp
11 logging.basicConfig(
12     level=logging.INFO,
13     format='%(asctime)s [%(levelname)s] [%(filename)s:%(lineno)d]
14           %(message)s'
15 )
16 logger = logging.getLogger("ETL_Pipeline")
17
18 # 2. Định nghĩa Data Contract bằng Pydantic
19 class TransactionRecord(BaseModel):
20     order_id: str
21     customer_id: str
22     amount: float = Field(gt=0, description="Giá trị đơn hàng phải lớn hơn 0")
23     order_status: str
24     created_at: datetime
25     discount: Optional[float] = 0.0
26
27 # 3. Hàm Extract & Validate
28 def extract_and_validate(csv_path: str) -> List[TransactionRecord]:
29     logger.info(f"Dang doc va kiem tra du lieu tu: {csv_path}")

```

```

29     df = pd.read_csv(csv_path)
30     valid_records: List[TransactionRecord] = []
31     quarantine_records = []
32
33     for idx, row in df.iterrows():
34         try:
35             record = TransactionRecord(
36                 order_id=str(row['order_id']),
37                 customer_id=str(row['customer_id']),
38                 amount=float(row['amount']),
39                 order_status=str(row['order_status']).upper(),
40                 created_at=pd.to_datetime(row['created_at']),
41                 discount=float(row.get('discount', 0.0))
42             )
43             valid_records.append(record)
44         except (ValidationError, Exception) as e:
45             quarantine_records.append({"row_index": idx, "error": str(e),
"raw_data": row.to_dict()})
46
47     logger.info(f"Kiem tra hoan tat: {len(valid_records)} hop le,
{len(quarantine_records)} bi loi bo vao Quarantine!")
48     return valid_records
49
50 # 4. Ham Load Luy Dang vao PostgreSQL (Idempotent Load)
51 def load_to_warehouse(records: List[TransactionRecord], db_conn_str: str)
-> None:
52     if not records:
53         logger.warning("Khong co ban ghi hop le nao de nap!")
54         return
55
56     insert_query = """
57     INSERT INTO fact_transactions (order_id, customer_id, amount,
order_status, created_at, discount, loaded_at)
58     VALUES (%s, %s, %s, %s, %s, %s, NOW())
59     ON CONFLICT (order_id) DO UPDATE SET
60         customer_id = EXCLUDED.customer_id,
61         amount = EXCLUDED.amount,
62         order_status = EXCLUDED.order_status,
63         discount = EXCLUDED.discount,
64         loaded_at = NOW();
65     """
66
67     data_tuples = [
68         (r.order_id, r.customer_id, r.amount, r.order_status,
r.created_at, r.discount)
69         for r in records
70     ]
71
72     try:

```

```

73         with psycopg2.connect(db_conn_str) as conn:
74             with conn.cursor() as cur:
75                 execute_batch(cur, insert_query, data_tuples,
page_size=1000)
76                 conn.commit()
77                 logger.info(f"[V] Nạp thành công {len(records)} bản ghi
vao PostgreSQL DWH!")
78     except Exception as e:
79         logger.error(f"[X] Lỗi khi nạp vào Data Warehouse: {e}")
80         raise

```

8.5 Bài Tập Tự Luyện Level 5

Bài Tập Tự Luyện: 5 Bài Tập Mô Hình Hóa & Lập Trình Pipeline

- Bài 1 (Kimball Modeling):** Thiết kế mô hình Star Schema cho ứng dụng gọi xe công nghệ (như Grab/Gojek). Xác định: Grain, 1 Fact Table và 4 Dimension Tables tương ứng.
- Bài 2 (SCD Type 2 SQL Trigger):** Viết câu lệnh DDL khởi tạo bảng `dim_customers_scd2` có các cột `surrogate_key`, `customer_id`, `address`, `valid_from`, `valid_to`, `is_current`.
- Bài 3 (Watermarking):** Viết đoạn mã Python lưu trữ trạng thái `last_max_timestamp` vào một tệp JSON hoặc bảng metadata để pipeline chỉ trích xuất những bản ghi mới hơn giá trị mốc này ở lần chạy tiếp theo.
- Bài 4 (Dead Letter Queue):** Thiết kế giải pháp xử lý cho 5% các bản ghi bị lỗi dữ liệu (Invalid format) trong pipeline để không làm gián đoạn toàn bộ tiến trình xử lý nhưng vẫn đảm bảo có thể điều tra lại sau đó.
- Bài 5 (Data Quality Check):** Viết một hàm kiểm tra chất lượng dữ liệu tự động xác minh tính toàn vẹn tham chiếu (Referential Integrity): Kiểm tra xem có đơn hàng nào trong `fact_orders` chứa `customer_id` không hề tồn tại trong `dim_customers` hay không.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- Bài 1:**
 - **Grain:** Mỗi dòng là một cuộc xe hoàn thành hoặc bị hủy.
 - **Fact:** `fact_trips(trip_id, driver_id, passenger_id, date_id, location_id, fare_amount, tip_amount, distance_km, duration_mins)`.
 - **Dimensions:** `dim_driver`, `dim_passenger`, `dim_date`, `dim_location`.
- Bài 2:**

```

1 CREATE TABLE dim_customers_scd2 (
2     surrogate_key BIGSERIAL PRIMARY KEY,

```

```
3     customer_id VARCHAR(50) NOT NULL,  
4     address TEXT NOT NULL,  
5     valid_from TIMESTAMP NOT NULL,  
6     valid_to TIMESTAMP DEFAULT '9999-12-31 23:59:59',  
7     is_current BOOLEAN DEFAULT TRUE  
8 );  
9
```

- **Bài 3:** Sử dụng câu lệnh: `SELECT MAX(updated_at) FROM fact_table` sau mỗi mẻ nạp thành công và lưu vào bảng `etl_watermark_control`.
- **Bài 4:** Tạo cơ chế **Quarantine / Dead Letter Queue (DLQ)**: Toàn bộ bản ghi sai định dạng được đẩy vào bảng riêng `quarantine_transactions` kèm thông báo Slack/Email gửi tới Data Engineer trực ca.
- **Bài 5:** Dùng truy vấn `LEFT JOIN ... WHERE dim.id IS NULL`. Nếu kết quả trả về > 0 , pipeline sẽ phát tín hiệu cảnh báo mức độ nghiêm trọng (Severity: High).

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Dự Án ETL Vào CV Kỹ Sư Dữ Liệu

Automated Batch ETL Data Pipeline | Python, PostgreSQL, Pydantic, Logging

- Xây dựng đường ống trích xuất, biến đổi và nạp dữ liệu (ETL) tự động hàng ngày xử lý hơn 250,000 giao dịch từ REST APIs và tệp CSV vào kho dữ liệu PostgreSQL.
- Áp dụng mô hình hóa chiều Kimball Star Schema với cơ chế quản lý lịch sử biến đổi chậm SCD Type 2 và thiết kế cơ chế nạp lũy đẳng (Idempotent Load) ngăn chặn hoàn toàn trùng lặp dữ liệu.
- Tích hợp thư viện Pydantic và hệ thống giám sát kiểm định chất lượng dữ liệu (Data Quality Checks), giảm thiểu 95% lỗi dữ liệu bản xâm nhập vào hệ thống báo cáo.

Chương 9

Level 6: DE Professional – Docker, Apache Airflow & dbt (Tuần 31–36)

9.1 Docker & Containerization Cho Kỹ Sư Dữ Liệu

Trong môi trường sản xuất, bạn không thể yêu cầu quản trị viên cài đặt thủ công từng phiên bản Python, thư viện C, hay cơ sở dữ liệu trên từng máy chủ. **Docker** đóng gói toàn bộ mã nguồn, môi trường chạy, biến môi trường và thư viện phụ thuộc vào một đơn vị độc lập gọi là **Container**.

Thực Chiến & Best Practices: Tập docker-compose.yml Khởi Động Toàn Bộ Stack Dữ Liệu

Chỉ với lệnh `docker compose up -d`, toàn bộ hệ sinh thái gồm PostgreSQL và Apache Airflow sẽ được khởi chạy đồng bộ:

```
1 version: '3.8'
2 services:
3   postgres_dwh:
4     image: postgres:16-alpine
5     container_name: postgres_dwh
6     environment:
7       POSTGRES_USER: data_admin
8       POSTGRES_PASSWORD: SecretPassword123
9       POSTGRES_DB: analytics_warehouse
10    ports:
11      - "5432:5432"
12    volumes:
13      - postgres_data:/var/lib/postgresql/data
14
15   airflow_webserver:
16     image: apache/airflow:2.8.1-python3.11
17     container_name: airflow_webserver
18     restart: always
19     depends_on:
```

```

20     - postgres_dwh
21     environment:
22     - AIRFLOW__CORE__LOAD_EXAMPLES=False
23     -
24     AIRFLOW__CORE__SQL_ALCHEMY_CONN=postgresql+psycopg2://data_admin:SecretPassword1
25     ports:
26     - "8080:8080"
27     volumes:
28     - ./dags:/opt/airflow/dags
29     - ./logs:/opt/airflow/logs
30 volumes:
31     postgres_data:

```

9.2 Apache Airflow: Trái Tim Điều Phối Hệ Thống (Orchestrator)

Airflow không thực hiện tính toán dữ liệu trực tiếp; nó đóng vai trò là "Nhạc trưởng" điều phối các công việc (Tasks) chạy đúng lịch trình, đúng thứ tự phụ thuộc, tự động thử lại khi có lỗi mạng và ghi lại toàn bộ lịch sử thực thi.

Kiến Thức Cốt Lõi: Các Khái Niệm Cốt Lõi Của Airflow 2.x

- **DAG (Directed Acyclic Graph):** Đồ thị có hướng không chu trình. Một luồng công việc phải có điểm bắt đầu và điểm kết thúc rõ ràng, không được phép lặp vòng tròn vô tận ($A \rightarrow B \rightarrow C \rightarrow A$).
- **TaskFlow API:** Cú pháp hiện đại sử dụng Decorator `@dag` và `@task`, tự động chuyển giao dữ liệu giữa các bước qua XComs mà không cần viết boilerplate code phức tạp.
- **Cạm bẫy Catchup:** Luôn thiết lập `catchup=False`! Nếu không, khi bạn kích hoạt một DAG có lịch chạy từ 1 năm trước, Airflow sẽ ngay lập tức kích hoạt hàng trăm tiến trình chạy đồng thời làm sập toàn bộ máy chủ!

9.2.1 Mã Nguồn Một Airflow DAG Chuẩn Doanh Nghiệp (TaskFlow API)

```

1 from datetime import datetime, timedelta
2 from airflow.decorators import dag, task
3 import requests
4 import json
5
6 default_args = {
7     'owner': 'data_engineering_team',
8     'depends_on_past': False,
9     'email_on_failure': True,

```

```

10     'retries': 3,
11     'retry_delay': timedelta(minutes=5),
12 }
13
14 @dag(
15     dag_id='ecommerce_daily_analytics_pipeline',
16     default_args=default_args,
17     description='Pipeline tu dong trich xuất, kiểm tra và nạp dữ liệu đơn hàng',
18     schedule_interval='0 2 * * *', # Chạy vào 02:00 sáng mỗi ngày
19     start_date=datetime(2026, 1, 1),
20     catchup=False,
21     tags=['production', 'ecommerce', 'elt']
22 )
23 def ecommerce_pipeline():
24
25     @task()
26     def extract_raw_transactions() -> list:
27         print("[*] Đang gọi REST API để trích xuất giao dịch ngày hôm qua...")
28         response =
29         requests.get("https://api.mockcommerce.com/v1/orders/daily",
30                     timeout=30)
31         response.raise_for_status()
32         orders = response.json()
33         print(f"[+] Đã trích xuất được: {len(orders)} đơn hàng.")
34         return orders
35
36     @task()
37     def validate_and_stage(orders: list) -> str:
38         clean_orders = [o for o in orders if float(o.get('amount', 0)) > 0]
39         staged_file_path =
40         f"/tmp/staged_orders_{datetime.now().strftime('%Y%m%d')}.json"
41         with open(staged_file_path, 'w', encoding='utf-8') as f:
42             json.dump(clean_orders, f)
43         print(f"[+] Đã lưu {len(clean_orders)} đơn hàng hợp lệ vào: {staged_file_path}")
44         return staged_file_path
45
46     @task()
47     def trigger_dbt_transformation(staged_file: str):
48         print(f"[*] Đang kích hoạt dbt models trên file: {staged_file}")
49         # Thực thi lệnh dbt run để biến đổi dữ liệu trong Data Warehouse
50         print("[V] Hoàn tất toàn bộ tiến trình pipeline ngày!")
51
52     # Thiết lập luồng phụ thuộc (Dependency Chain)
53     raw_data = extract_raw_transactions()
54     staged_data = validate_and_stage(raw_data)
55     trigger_dbt_transformation(staged_data)

```

53

54 `ecommerce_dag_instance = ecommerce_pipeline()`

9.3 dbt (Data Build Tool): Tái Định Nghĩa Analytics Engineering

Kiến Thức Cốt Lõi: Triết Lý Hiện Đại: Biến Đổi Dữ Liệu Ngay Trong Kho (In-Warehouse ELT)

Trước đây với ETL cổ điển, dữ liệu được kéo về máy chủ Python để biến đổi rồi mới đẩy vào DWH. Với **dbt**, toàn bộ dữ liệu thô được nạp thẳng vào Data Warehouse trước (Extract & Load). Sau đó, **dbt** chỉ quản lý mã SQL **SELECT**, biên dịch chúng thành các bảng (TABLE) hoặc khung nhìn (VIEW) bên trong Data Warehouse bằng chính sức mạnh tính toán khổng lồ của DWH!

9.3.1 Cấu Trúc Mô Hình Đa Tầng Chuẩn Mực Trong dbt

- **Tầng 1: Staging (models/staging/)**: Làm sạch cơ bản, đổi tên cột chuẩn `snake_case`, ép kiểu dữ liệu. Quan hệ 1 – 1 với bảng nguồn.
- **Tầng 2: Intermediate (models/intermediate/)**: Thực hiện các phép JOIN phức tạp, logic nghiệp vụ nội bộ.
- **Tầng 3: Marts (models/marts/)**: Các bảng Fact và Dimension chuẩn Star Schema cung cấp trực tiếp cho Power BI và đội ngũ phân tích.

Thực Chiến & Best Practices: Ví Dụ Một Model dbt: `fct_customer_orders.sql`

```

1  {{
2      config(
3          materialized='incremental',
4          unique_key='order_id',
5          on_schema_change='fail'
6      )
7  }}
8
9  WITH staging_orders AS (
10     SELECT * FROM {{ ref('stg_ecommerce_orders') }}
11     {% if is_incremental() %}
12         -- Chỉ trích xuất các bản ghi mới hoặc được cập nhật kể từ
13         -- lần chạy trước
14         WHERE order_updated_at > (SELECT MAX(order_updated_at) FROM
15             {{ this }})
16     {% endif %}
17 )
18 customers AS (

```

```

17     SELECT * FROM {{ ref('dim_customers') }}
18 )
19 SELECT
20     o.order_id,
21     o.customer_id,
22     c.customer_city,
23     o.order_amount,
24     o.order_date,
25     CURRENT_TIMESTAMP AS dbt_updated_at
26 FROM staging_orders o
27 LEFT JOIN customers c ON o.customer_id = c.customer_id;

```

9.4 PROJECT 6: Production Analytics Engineering Pipeline

Dự án này tích hợp toàn bộ:

1. Khởi chạy môi trường đa dịch vụ bằng Docker Compose.
2. Xây dựng DAG Airflow tự động kéo dữ liệu mỗi ngày và xử lý lỗi tự động.
3. Tích hợp dbt để kiểm thử chất lượng dữ liệu (unique, not_null) và tạo bảng Data Marts.
4. Xuất tự động tài liệu Data Lineage Graph cho toàn bộ công ty.

9.5 Bài Tập Tự Luyện Level 6

Bài Tập Tự Luyện: 5 Bài Tập Docker, Airflow & dbt Thực Tế

1. **Bài 1 (Docker):** Viết một Dockerfile tối ưu hóa bằng Multi-stage build để đóng gói ứng dụng Python ETL, giảm dung lượng ảnh từ 1GB xuống dưới 150MB.
2. **Bài 2 (Airflow Branching):** Sử dụng BranchPythonOperator trong Airflow để kiểm tra kích thước file tải về: Nếu dung lượng > 10MB thì rẽ sang nhánh xử lý lô song song, nếu ≤ 10MB thì xử lý tuần tự.
3. **Bài 3 (dbt Generic Tests):** Viết file YAML cấu hình các kiểm thử tự động cho model fct_orders: Cột order_id phải unique và not_null; cột status chỉ nhận các giá trị ['PENDING', 'SUCCESS', 'FAILED'].
4. **Bài 4 (dbt Jinja Macro):** Viết một Macro Jinja trong dbt tên là cents_to_dollars(column_name) tự động chia giá trị cho 100 và làm tròn 2 chữ số thập phân.
5. **Bài 5 (Airflow SLA Alerting):** Cấu hình thông số SLA (Service Level Agreement) cho một

DAG Airflow sao cho nếu toàn bộ pipeline chạy quá 30 phút mà chưa hoàn tất, hệ thống sẽ tự động gửi email cảnh báo cho đội kỹ sư trực ban.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- **Bài 1:** Sử dụng base image nhẹ `python:3.11-slim`, dùng cờ `--no-cache-dir` khi cài pip và xóa các file tạm build tools ở stage cuối.
- **Bài 2:** Sử dụng decorator `@task.branch` trả về `task_id` của nhánh tương ứng dựa trên điều kiện `os.path.getsize(file_path)`.
- **Bài 3:** Khai báo trong file `schema.yml`:

```
1 models:
2   - name: fct_orders
3     columns:
4       - name: order_id
5         tests: [unique, not_null]
6       - name: status
7         tests:
8           - accepted_values:
9             values: ['PENDING', 'SUCCESS', 'FAILED']
10
```

- **Bài 4:** Macro chuẩn:

```
1 {% macro cents_to_dollars(column_name) %}
2   ROUND(CAST({{ column_name }} AS NUMERIC) / 100.0, 2)
3 {% endmacro %}
4
```

- **Bài 5:** Khai báo tham số `sla=timedelta(minutes=30)` trong `default_args` và định nghĩa hàm callback `sla_miss_callback`.

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Dự Án DE Pro Vào CV

Production Analytics Data Platform | Docker, Apache Airflow, dbt, PostgreSQL

- Container hóa toàn bộ hệ thống cơ sở dữ liệu và công cụ điều phối luồng dữ liệu sử dụng Docker Compose, tối ưu hóa quá trình triển khai nội bộ và môi trường phát triển.
- Thiết kế và triển khai 5 DAGs Apache Airflow theo TaskFlow API với cơ chế tự động thử lại (Automated Retries), kiểm soát SLA và hệ thống cảnh báo qua Webhook.
- Xây dựng hơn 20 mô hình biến đổi dữ liệu (dbt Models) theo kiến trúc Staging-Intermediate-Marts với mô hình gia tăng (Incremental Loading), cắt giảm 60% thời gian tính toán kho dữ liệu và thiết lập bộ kiểm thử toàn vẹn dữ liệu tự động.

Chương 10

Level 7: Machine Learning & Predictive Analytics Thực Chiến (Tuần 37–44)

Kiến Thức Cốt Lõi: Mục Tiêu & Tiền Đề Cần Nhớ

Tiền đề bắt buộc: Bạn cần đã hoàn thành Chương 3 (Python, Pandas) và Chương 5 (Thống kê cơ bản). **Chưa cần biết gì về trí tuệ nhân tạo hay thuật toán học máy trước đó!**

Mục tiêu chương này:

- Hiểu từ con số 0 bản chất của Machine Learning và phân biệt với lập trình truyền thống.
- Nắm vững 2 bài toán cốt lõi: Hồi quy (Regression) và Phân loại (Classification).
- Hiểu vì sao phải chia tập dữ liệu thành Train và Test để chống học vẹt (Overfitting).
- Làm chủ Scikit-Learn Pipeline và ColumnTransformer để ngăn chặn 100% cạm bẫy Rò rỉ dữ liệu (Data Leakage).
- Đánh giá mô hình dựa trên Confusion Matrix, Precision, Recall và ROC-AUC thay vì Accuracy ảo tưởng.
- Giải thích quyết định của mô hình bằng lý thuyết trò chơi **SHAP**.
- Hoàn thành **Project 7: Customer Churn Prediction System** sẵn sàng đưa vào CV ứng tuyển Data Scientist / ML Engineer Intern.

10.1 Machine Learning Căn Bản Từ Con Số 0

10.1.1 1. Lập Trình Truyền Thống vs Machine Learning

- **Lập trình truyền thống (Software 1.0):** Con người tự nghĩ ra quy tắc (Rules) và nạp dữ liệu (Data) vào máy tính để nhận kết quả (Answers).

Data + Rules (if/else do con người viết) $\longrightarrow$ Answers

Ví dụ: Nếu khách hàng không đăng nhập > 30 ngày thì gắn cờ "Nguy cơ rời bỏ".

- **Học máy (Machine Learning - Software 2.0):** Chúng ta cung cấp dữ liệu lịch sử (Data) cùng với kết quả thực tế trong quá khứ (Answers), và máy tính sẽ **tự động học ra quy tắc toán học tối ưu nhất!**

Data + Answers (Lịch sử thực tế) $\longrightarrow$ Rules (Mô hình / Model)

10.1.2 2. Ma Trận Đặc Trưng (X) & Vector Mục Tiêu (y)

Trong học máy có giám sát (Supervised Learning):

- **Đặc trưng (X - Features / Input):** Bảng gồm n dòng và m cột chứa các thông tin quan sát được của đối tượng (ví dụ: age, monthly_charges, tenure_months).
- **Mục tiêu (y - Target / Label):** Cột kết quả duy nhất mà ta muốn dự báo trong tương lai (ví dụ: is_churn nhận giá trị 1 hoặc 0).

10.1.3 3. Tại Sao Bắt Buộc Phải Chia Tập Train & Test?

Nếu bạn cho một học sinh ôn tập đúng 100 câu hỏi và đề thi cuối kỳ cũng dùng y hệt 100 câu hỏi đó, học sinh có thể đạt điểm 10 tuyệt đối chỉ bằng cách học vẹt đáp án mà không hiểu bản chất! Trong Machine Learning, hiện tượng này gọi là **Overfitting (Quá khớp / Học vẹt)**. Để đo lường trung thực khả năng ứng dụng trên dữ liệu chưa từng thấy:

- Ta chia tập dữ liệu ban đầu thành: **80% Train Set** (để mô hình học) và **20% Test Set** (khóa kín lại, chỉ dùng để thi sát hạch cuối cùng).

10.2 Feature Engineering & Nghệ Thuật Ngăn Chặn Data Leakage

Kiến Thức Cốt Lõi: Cạm Bẫy Chết Người: Data Leakage (Rò Rỉ Dữ Liệu)

Data Leakage xảy ra khi thông tin từ tập kiểm thử (Test Set) hoặc thông tin chỉ xuất hiện *sau khi sự kiện diễn ra* bị rò rỉ vào quá trình huấn luyện mô hình.

- **Ví dụ kinh điển:** Chuẩn hóa dữ liệu (StandardScaler) trên toàn bộ tập dữ liệu **TRƯỚC KHI** chia `train_test_split`. Điều này làm cho giá trị trung bình μ và độ lệch chuẩn σ của tập kiểm thử bị tiết lộ vào tập huấn luyện!
- **Rò rỉ dòng thời gian (Look-ahead Bias):** Sử dụng cột "Thời gian gọi điện khiếu nại hủy dịch vụ" để dự báo khách hàng có rời bỏ (Churn) hay không trong khi cuộc gọi đó chỉ xảy ra sau khi khách hàng đã có ý định rời bỏ!

Giải pháp tuyệt đối: Luôn luôn sử dụng **Scikit-learn Pipeline** và **ColumnTransformer!**

10.3 Vượt Qua Ảo Tưởng Accuracy: Ma Trận Đánh Giá Thực Tế

Khi dự báo tỷ lệ khách hàng rời bỏ mạng viễn thông (tỷ lệ rời bỏ thực tế chỉ là 5%):

- Nếu một mô hình "ngớ ngẩn" luôn luôn dự báo tất cả mọi người là "**Ở lại (Non-Churn)**", nó sẽ đạt độ chính xác **Accuracy = 95%**!
- Nhưng nó phát hiện được bao nhiêu người rời bỏ thực sự? **Chính xác là 0%**! Doanh nghiệp vẫn mất trắng 5% khách hàng này.

Thực Chiến & Best Practices: Chọn Đúng Chỉ Số Cho Đúng Bài Toán Kinh Doanh

- **Precision (Độ chuẩn xác - Tỷ lệ đoán trúng trong số những ca báo động):**

$$\text{Precision} = \frac{TP}{TP + FP}$$

→ Ưu tiên khi **chi phí cho một cảnh báo giả (False Positive) rất đắt đỏ**. Ví dụ: Bộ lọc thư rác (Spam Filter) - Bạn thà để sót một email spam còn hơn chuyển nhầm email quan trọng của đối tác kinh doanh vào hòm thư rác!

- **Recall (Độ thu hồi - Tỷ lệ tóm được trong tổng số ca bệnh/gian lận thực tế):**

$$\text{Recall} = \frac{TP}{TP + FN}$$

→ Ưu tiên khi **bỏ sót một ca bệnh (False Negative) là thảm họa**! Ví dụ: Phát hiện ung thư, phát hiện giao dịch rửa tiền gian lận ngân hàng.

- **ROC-AUC (Receiver Operating Characteristic - Area Under Curve):** Đánh giá khả năng phân tách xếp hạng xác suất của mô hình độc lập với ngưỡng phân loại (Threshold-independent). Điểm chuẩn: 0.5 (ngang tung đồng xu), > 0.8 (Tốt), > 0.9 (Xuất sắc).

10.4 PROJECT 7: End-to-End Customer Churn Prediction System

10.4.1 Bài Toán Thực Tế

Xây dựng pipeline Machine Learning hoàn chỉnh dự báo khách hàng rời bỏ cho doanh nghiệp thương mại điện tử dựa trên hành vi giao dịch và tần suất đăng nhập.

10.4.2 Mã Nguồn Pipeline Scikit-Learn Chuẩn Doanh Nghiệp

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split, StratifiedKFold,
  cross_val_score
4 from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```

5 from sklearn.compose import ColumnTransformer
6 from sklearn.pipeline import Pipeline
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.metrics import classification_report, roc_auc_score
9 import joblib
10
11 # 1. Tai du lieu da trich xuat tu PostgreSQL
12 df = pd.read_csv("data/customer_churn_features.csv")
13
14 X = df.drop(columns=['customer_id', 'is_churn'])
15 y = df['is_churn']
16
17 # 2. Chia tap du lieu theo ty le phan tang (Stratified)
18 X_train, X_test, y_train, y_test = train_test_split(
19     X, y, test_size=0.2, random_state=42, stratify=y
20 )
21
22 # 3. Dinh nghia cac cot so va cot chu
23 numeric_features = ['tenure_months', 'monthly_charges',
24                     'total_transactions', 'days_since_last_login']
25 categorical_features = ['contract_type', 'payment_method', 'gender']
26
27 # 4. Dong goi Preprocessing bang ColumnTransformer
28 preprocessor = ColumnTransformer(
29     transformers=[
30         ('num', StandardScaler(), numeric_features),
31         ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
32          categorical_features)
33     ]
34 )
35
36 # 5. Xay dung Scikit-Learn Pipeline hoan chinh (Tranh Data Leakage 100%)
37 full_pipeline = Pipeline(steps=[
38     ('preprocessor', preprocessor),
39     ('classifier', RandomForestClassifier(n_estimators=200, max_depth=8,
40                                         class_weight='balanced', random_state=42))
41 ])
42
43 # 6. Kiem dinh cheo 5-Fold Stratified Cross Validation
44 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
45 cv_scores = cross_val_score(full_pipeline, X_train, y_train, cv=cv,
46                             scoring='roc_auc')
47
48 print(f"[*] 5-Fold CV ROC-AUC Scores: {cv_scores}")
49 print(f"[+] Average CV ROC-AUC      : {cv_scores.mean():.4f} (+/-
50     {cv_scores.std():.4f})")
51
52 # 7. Huan luyen tren toan bo tap Train va Danh gia tren Test Set doc lap
53 full_pipeline.fit(X_train, y_train)

```

```

49 y_pred_proba = full_pipeline.predict_proba(X_test)[: , 1]
50 y_pred = (y_pred_proba >= 0.40).astype(int) # Tối ưu hóa ngưỡng
    (Threshold tuning)
51
52 test_auc = roc_auc_score(y_test, y_pred_proba)
53 print(f"\n[V] Test Set ROC-AUC Score: {test_auc:.4f}\n")
54 print("Bao Cao Chi Tiet Hieu Nang (Classification Report):")
55 print(classification_report(y_test, y_pred))
56
57 # 8. Dong goi model de dua vao Production
58 joblib.dump(full_pipeline, "models/churn_prediction_v1.joblib")
59 print("[+] Da dong goi model vao file 'models/churn_prediction_v1.joblib'
    thanh cong!")

```

10.5 Khả Năng Diễn Giải Mô Hình Với SHAP (SHapley Values)

Trong ngành ngân hàng và bảo hiểm, quy định pháp luật yêu cầu bạn phải giải thích được lý do vì sao một khách hàng bị từ chối cấp thẻ tín dụng. Thư viện ****SHAP**** dựa trên lý thuyết trò chơi của nhà toán học đoạt giải Nobel Lloyd Shapley, lượng hóa mức độ đóng góp chính xác của từng đặc trưng:

```

1 import shap
2
3 # Trich xuất mô hình cây từ pipeline đã huấn luyện
4 rf_model = full_pipeline.named_steps['classifier']
5 X_test_transformed =
    full_pipeline.named_steps['preprocessor'].transform(X_test)
6
7 # Tính toán SHAP values
8 explainer = shap.TreeExplainer(rf_model)
9 shap_values = explainer.shap_values(X_test_transformed)
10
11 # Giải thích quyết định: Đặc trưng nào khiến khách hàng này có nguy cơ rời
    bỏ cao nhất?
12 # shap.summary_plot(shap_values[1], X_test_transformed)

```

10.6 Bài Tập Tự Luyện Level 7

Bài Tập Tự Luyện: 5 Bài Tập Xây Dựng & Tối Ưu Mô Hình ML

- Bài 1 (Regularization):** Phân tích sự khác biệt về mặt toán học giữa chuẩn hóa L_1 (Lasso) và L_2 (Ridge). Vì sao Lasso lại có khả năng triệt tiêu hoàn toàn trọng số về 0 để tự động chọn lọc đặc trưng (Feature Selection)?
- Bài 2 (Threshold Tuning):** Giả sử chi phí giữ chân một khách hàng có nguy cơ rời bỏ là 10

USD (tặng voucher), nhưng nếu mất khách hàng đó công ty thiệt hại 200 USD. Hãy viết đoạn mã Python tìm ngưỡng xác suất tối ưu ($Threshold \in [0.1, 0.9]$) để cực tiểu hóa tổng chi phí tổn thất kinh doanh.

3. **Bài 3 (Target Encoding):** Viết một class biến đổi dữ liệu tùy chỉnh kế thừa từ `BaseEstimator`, `TransformerMixin` của Scikit-learn để thực hiện Target Encoding có tính toán làm mịn (Smoothing) nhằm ngăn chặn Overfitting.
4. **Bài 4 (XGBoost vs LightGBM):** So sánh cơ chế phân nhánh cây theo chiều sâu (Level-wise) của XGBoost và phân nhánh theo lá tốt nhất (Leaf-wise) của LightGBM. Trường hợp nào LightGBM có nguy cơ bị Overfitting cao hơn?
5. **Bài 5 (Error Analysis):** Viết đoạn mã trích xuất 20 trường hợp False Positives (mô hình đoán churn nhưng thực tế không churn) và tính toán khoảng cách trung bình của các đặc trưng này so với mẫu huấn luyện để phát hiện nhóm dữ liệu bị dị biệt.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- **Bài 1:** Lasso phạt hàm mất mát bằng $\lambda \sum |w_i|$ (hình viên kim cương có góc nhọn cắt các trục tọa độ tại chính xác 0). Ridge phạt bằng $\lambda \sum w_i^2$ (hình cầu tròn, chỉ thu nhỏ trọng số tiệm cận 0 chứ không bao giờ chạm hẳn 0).
- **Bài 2:** Tạo hàm tính chi phí: `Total_Cost = FP * 10 + FN * 200`. Duyệt qua các ngưỡng cắt từ 0.1 đến 0.9 với bước nhảy 0.05, chọn ngưỡng có `Total_Cost` thấp nhất (thường ngưỡng tối ưu sẽ thấp hơn 0.5 để bắt được nhiều FN hơn).

- **Bài 3:** Công thức làm mịn Target Encoding:

$$S_i = \frac{n_i \times \bar{y}_i + m \times \bar{y}_{\text{global}}}{n_i + m}$$

với m là trọng số làm mịn (Weight of prior).

- **Bài 4:** LightGBM chia nhánh theo Leaf-wise (chọn lá làm giảm sai số nhiều nhất để chia tiếp). Trên tập dữ liệu nhỏ (< 10,000 dòng), Leaf-wise rất dễ phát triển cây quá sâu và gây Overfitting nặng. Cần kiểm soát bằng `max_depth` và `min_child_samples`.
- **Bài 5:** Dùng biểu thức lọc: `df_errors = X_test[(y_test == 0) & (y_pred == 1)]` và phân tích phân phối các cột thông qua `df_errors.describe()`.

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Dự Án Machine Learning Vào CV

Customer Churn Prediction & Retention Pipeline | Python, Scikit-learn, XGBoost, SHAP

- Xây dựng hệ thống học máy dự báo khách hàng rời bỏ từ 150,000 hồ sơ khách hàng, đóng gói toàn bộ quy trình qua Scikit-Learn Pipeline ngăn chặn 100% rò rỉ dữ liệu.

- Huấn luyện và tinh chỉnh các mô hình Gradient Boosting (XGBoost, Random Forest), đạt điểm ROC-AUC 0.88 trên tập kiểm thử độc lập và cải thiện Recall lên tới 78% qua kỹ thuật cân bằng trọng số lớp (Class Weights).
- Ứng dụng lý thuyết trò chơi SHAP để giải thích các nhân tố then chốt dẫn đến tỷ lệ rời bỏ (thời gian từ lần đăng nhập cuối và loại hợp đồng), hỗ trợ phòng Marketing xây dựng chiến dịch giữ chân cá nhân hóa giúp giảm 12% tỷ lệ khách hàng rời mạng.

Chương 11

Level 8: Big Data & Lakehouse Architecture Với PySpark (Tuần 45–50)

11.1 Khi Nào Cần Big Data & Giới Hạn Của Pandas

Một hiểu lầm tai hại của nhiều kỹ sư trẻ là sử dụng Spark cho mọi tác vụ chỉ để ”làm đẹp CV”. Nếu tập dữ liệu của bạn chỉ có 2 triệu dòng (khoảng 500MB), việc nạp vào Spark thực chất sẽ **chậm hơn gấp 5 lần so với Pandas** do chi phí khởi tạo cụm phân tán (Cluster Overhead), tuần tự hóa đối tượng (Serialization), và truyền tải mạng!

Kiến Thức Cốt Lõi: Ngưỡng Chuyển Đổi Kỹ Thuật (Technology Threshold)

- **Dưới 10GB:** Sử dụng **Pandas, Polars, hoặc DuckDB** trên một máy chủ duy nhất (Single Node). DuckDB có thể xử lý 10GB dữ liệu trong vài giây mà không tốn chi phí hạ tầng.
- **Từ 10GB đến 100GB:** Tối ưu hóa bộ nhớ RAM với DuckDB hoặc dùng máy chủ cấu hình cao (Vertical Scaling).
- **Trên 100GB đến hàng Terabytes/Petabytes:** Bắt buộc sử dụng hệ thống tính toán phân tán **Apache Spark** (Horizontal Scaling).

11.2 Kiến Trúc Apache Spark Chuyên Sâu

Apache Spark là động cơ tính toán phân tán trong bộ nhớ (In-Memory Distributed Engine).

Kiến Thức Cốt Lõi: Driver, Executors & Phân Loại Biến Đổi (Transformations)

- **Driver Program:** Bộ não trung tâm. Phân tích mã nguồn, tối ưu hóa qua **Catalyst Optimizer**, chuyển đổi thành đồ thị thực thi **DAG** và giao việc cho các Executors.
- **Executors:** Các tiến trình thợ chạy song song trên các máy tính khác nhau trong cụm, lưu trữ

dữ liệu phân mảnh (Partitions) và thực thi các tính toán.

- **Narrow Transformation (Biến đổi hẹp):** Dữ liệu ở một partition đầu vào chỉ cần tính toán trong chính partition đó, không cần truyền tải qua mạng (`select`, `filter`, `map`). Cực kỳ nhanh!
- **Wide Transformation (Biến đổi rộng):** Dữ liệu cần phải được gom nhóm lại từ tất cả các máy trong cụm (`groupBy`, `join`, `distinct`). Đây là nguyên nhân gây ra hiện tượng **Shuffle (Giao tiếp mạng tốn kém)** - kẻ thù số 1 làm chậm Spark!

11.3 PySpark Thực Chiến & Kỹ Thuật Broadcast Join

Thực Chiến & Best Practices: Tối Ưu Hóa Phép Nối Với Broadcast Hash Join

Khi bạn cần nối một bảng giao dịch khổng lồ 500 triệu dòng (`fact_sales` - 200GB) với một bảng danh mục nhỏ (`dim_store` - 5MB):

- **Sort-Merge Join mặc định:** Spark sẽ băm nhỏ và truyền tải cả 200GB dữ liệu qua đường truyền mạng của cụm để gom nhóm các khóa trùng nhau → Mất 40 phút!
- **Broadcast Join:** Spark sao chép bảng nhỏ 5MB gửi tới bộ nhớ của TẤT CẢ các Executors. Mỗi máy tự động join trực tiếp trong RAM mà **không cần shuffle một byte dữ liệu nào qua mạng** → Hoàn thành chỉ trong 1.5 phút!

```

1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import broadcast, col, sum, round
3
4 # Khởi tạo SparkSession tối ưu
5 spark = SparkSession.builder \
6     .appName("EcommerceBigDataOptimization") \
7     .config("spark.sql.shuffle.partitions", "200") \
8     .config("spark.driver.memory", "4g") \
9     .config("spark.executor.memory", "8g") \
10    .getOrCreate()
11
12 # Đọc dữ liệu lớn dạng Parquet
13 fact_sales =
14     spark.read.parquet("s3a://data-lake/raw/sales_transactions/")
15 dim_store = spark.read.parquet("s3a://data-lake/raw/dim_stores/")
16
17 # Áp dụng Broadcast Join chu dòng
18 optimized_sales = fact_sales.join(
19     broadcast(dim_store),
20     fact_sales.store_id == dim_store.store_id,
21     how="inner"

```

```

21 ).groupBy("store_region").agg(
22     round(sum("amount"), 2).alias("total_regional_revenue")
23 )
24
25 # Thuc thi Action ghi ket qua
26 optimized_sales.write.mode("overwrite").parquet("s3a://data-lake/gold/regional_summa

```

11.4 Kiến Trúc Lakehouse Hiện Đại Với Delta Lake

Trước đây, Data Lake (lưu trên Amazon S3 hoặc Hadoop HDFS) gặp vô số nhược điểm: không hỗ trợ giao dịch ACID (nếu đang ghi file bị mất điện thì dữ liệu bị hỏng hoàn toàn), không thể cập nhật từng dòng (UPDATE/DELETE), và không có Schema Enforcement.

Kiến Trúc Cốt Lõi: Sức Mạnh Của Delta Lake (ACID Trên Data Lake)

Delta Lake biến đổi Data Lake thô thành một **Lakehouse** chuyên nghiệp bằng cách bổ sung một cuốn sổ nhật ký giao dịch tuần tự dạng JSON mang tên `_delta_log`:

- **ACID Transactions:** Hoặc là toàn bộ mẽ nạp thành công, hoặc là không có gì thay đổi. Không bao giờ xuất hiện file rác hỏng hóc.
- **Time Travel (Du hành thời gian):** Khả năng truy vấn lại dữ liệu chính xác như nó đã từng tồn tại ở một thời điểm trong quá khứ hoặc phiên bản commit cụ thể:

```

1 df_historical = spark.read.format("delta") \
2     .option("timestampAsOf", "2026-02-15 00:00:00") \
3     .load("/delta/customer_profiles")
4

```

- **Schema Enforcement:** Ngăn chặn việc đẩy dữ liệu sai kiểu cột vào hồ dữ liệu, bảo vệ tính toàn vẹn của hệ thống.

11.5 PROJECT 8: Big Data Transformation Engine (10+ Triệu Dòng)

Xây dựng pipeline PySpark đọc 15 triệu bản ghi sự kiện clickstream của người dùng dạng JSON thô từ Cloud Storage, thực hiện parsing, gộp nhóm hành vi theo phiên truy cập (Sessionization) bằng Window Functions, và ghi ra bằng Delta Lake được tối ưu hóa bằng kỹ thuật phân vùng `partitionBy('event_date')`.

11.6 Bài Tập Tự Luyện Level 8

Bài Tập Tự Luyện: 5 Bài Tập PySpark & Lakehouse

1. **Bài 1 (Repartition vs Coalesce):** Phân tích sự khác biệt về mặt cơ chế mạng giữa `df.repartition(10)` và `df.coalesce(10)`. Khi nào dùng `coalesce` để tối ưu hóa việc xuất file?
2. **Bài 2 (Window Function trong Spark):** Viết đoạn mã PySpark tính toán trung bình động 7 ngày của cột doanh thu (`sales_amount`) theo từng mã sản phẩm (`product_id`).
3. **Bài 3 (Xử lý Data Skew):** Trình bày kỹ thuật "Salting" trong Spark để giải quyết vấn đề Data Skew khi một giá trị khóa chiếm tới 80% tổng số bản ghi trong bảng.
4. **Bài 4 (Delta Lake Compaction):** Khi một bảng Delta có hàng triệu tệp tin Parquet siêu nhỏ (Small File Problem), hãy viết lệnh `OPTIMIZE` và `Z-ORDER BY` để dọn dẹp và tăng tốc độ truy vấn.
5. **Bài 5 (UDF Optimization):** Tại sao việc viết Python UDF (@udf) thông thường trong PySpark lại làm giảm nghiêm trọng hiệu năng so với việc dùng `pyspark.sql.functions` có sẵn hoặc Pandas UDF (Vectorized UDF với Apache Arrow)?

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- **Bài 1:** `coalesce` chỉ gộp các partition liền kề trên cùng máy chủ để giảm số lượng partition mà KHÔNG gây full shuffle mạng. `repartition` tạo ra một full shuffle qua mạng và chia đều dữ liệu ra số partition mới. Dùng `coalesce(1)` khi muốn xuất đúng 1 file CSV/Parquet duy nhất.

- **Bài 2:**

```

1 from pyspark.sql.window import Window
2 from pyspark.sql.functions import avg
3 w =
    Window.partitionBy("product_id").orderBy("event_date").rowsBetween(-6,
    0)
4 df.withColumn("rolling_avg_7d", avg("sales_amount").over(w))
5

```

- **Bài 3:** Kỹ thuật Salting: Ghép thêm một số ngẫu nhiên từ 0 đến $K - 1$ vào khóa bị lệch: `concat(skewed_key, lit("_"), rand() * K)` để phân tán các bản ghi sang nhiều executor khác nhau, sau đó tính toán 2 lần để gộp kết quả.
- **Bài 4:** Lệnh SQL trong Spark: `OPTIMIZE delta_table ZORDER BY (customer_id, event_date);`
- **Bài 5:** Python UDF thông thường buộc Spark JVM phải tuần tự hóa từng dòng dữ liệu và bản

qua tiến trình Python bên ngoài (gây nghẽn IPC - Inter-Process Communication). Trong khi đó, các hàm native chạy trực tiếp trên Java bytecode của JVM, và Pandas UDF sử dụng bộ nhớ chia sẻ Apache Arrow cực nhanh.

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Kỹ Năng PySpark Vào CV

Distributed Big Data Lakehouse Pipeline | PySpark, Delta Lake, AWS S3, Parquet

- Thiết kế và triển khai đường ống dữ liệu lớn PySpark xử lý hơn 15 triệu sự kiện tương tác người dùng hàng ngày, tối ưu hóa kích thước partition và áp dụng Broadcast Joins giúp giảm thời gian xử lý từ 2.5 giờ xuống còn 35 phút.
- Xây dựng kiến trúc Lakehouse hiện đại sử dụng Delta Lake trên AWS S3, hỗ trợ tính năng ACID Transactions và cơ chế Time Travel kiểm toán lịch sử thay đổi dữ liệu.
- Tối ưu hóa cấu trúc tệp tin thông qua Z-Ordering và Data Compaction, giúp cắt giảm 70% lượng dữ liệu cần quét khi thực hiện các truy vấn phân tích định kỳ.

Chương 12

Level 9: Nền Tảng Cloud Cho Data – AWS & GCP (Tuần 51–54)

12.1 Lựa Chọn Nền Tảng Đám Mây: AWS vs Google Cloud (GCP)

Trong kỷ nguyên điện toán đám mây, các doanh nghiệp không còn mua máy chủ vật lý đặt tại văn phòng mà thuê hạ tầng từ các nhà cung cấp Cloud lớn. Thay vì cố gắng học cả AWS, Azure và GCP cùng lúc, hãy chọn một nền tảng và làm chủ nó đến mức chuyên gia:

Nhóm Dịch Vụ	Amazon Web Services (AWS)	Google Cloud Platform (GCP)
Object Storage (Hồ chứa)	Amazon S3 (Tiêu chuẩn công nghiệp toàn cầu)	Google Cloud Storage (GCS)
Data Warehouse Serverless	Amazon Athena / Redshift	Google BigQuery (Cực kỳ mạnh mẽ cho DA/DS)
Serverless Compute	AWS Lambda	Cloud Functions / Cloud Run
Cơ Sở Dữ Liệu Quản Trị	Amazon RDS (PostgreSQL, MySQL)	Cloud SQL
Quản Lý Định Danh	AWS IAM (Cực kỳ chi tiết, bảo mật cao)	GCP IAM

12.2 Kiến Trúc Pipeline Không Máy Chủ (Serverless Event-Driven)

Mô hình không máy chủ (Serverless) cho phép bạn chạy mã nguồn xử lý dữ liệu mà không cần bận tâm về việc cài đặt hệ điều hành, vá lỗi bảo mật hay bảo trì máy chủ:

Kiến Thức Cốt Lõi: Luồng Dữ Liệu Hướng Sự Kiện (Event-Driven Ingestion)

- Đối tác đẩy một tệp CSV giao dịch bán hàng lên bucket **Amazon S3** (s3://ecommerce-landing-zone/raw/).
- Hành động đẩy file lập tức phát ra sự kiện **S3:ObjectCreated**.

3. Sự kiện này tự động kích hoạt hàm **AWS Lambda** viết bằng Python.
4. Lambda đọc tệp, kiểm tra tính hợp lệ của schema, giải mã định dạng và nạp trực tiếp vào kho dữ liệu.
5. Khi xử lý xong trong 3 giây, Lambda tự động tắt. Bạn chỉ phải trả tiền cho chính xác 3 giây đó (chi phí chỉ là 0.00001 USD)!

Thực Chiến & Best Practices: Mã Nguồn Hàm AWS Lambda Xử Lý Dữ Liệu Tự Động

```
1 import json
2 import boto3
3 import urllib.parse
4
5 s3_client = boto3.client('s3')
6
7 def lambda_handler(event, context):
8     """Tu dong kích hoạt khi co file moi duoc day len Amazon S3."""
9     # 1. Trich xuất thông tin Bucket và File Key từ sự kiện S3
10    bucket = event['Records'][0]['s3']['bucket']['name']
11    key =
12    urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])
13
14    print(f"[*] Phát hiện file mới tại: s3://{bucket}/{key}")
15
16    # 2. Đọc nội dung file trực tiếp vào RAM
17    response = s3_client.get_object(Bucket=bucket, Key=key)
18    file_content = response['Body'].read().decode('utf-8')
19    lines = file_content.splitlines()
20
21    total_records = len(lines) - 1 # Tru đi dòng header
22    print(f"[+] Số lượng bản ghi phát hiện: {total_records}")
23
24    # 3. Kích hoạt pipeline nạp tiếp theo hoặc ghi log kiểm toán
25    return {
26        'statusCode': 200,
27        'body': json.dumps(f"Xử lý thành công file {key} với {total_records} bản ghi!")
28    }
```

12.3 Bảo Mật & Nguyên Tắc Phân Quyền Tối Thiểu (Least Privilege)

Kiến Thức Cốt Lõi: Nguyên Tắc Đặc Quyền Tối Thiểu (Principle of Least Privilege)

Tuyệt đối không bao giờ gán quyền AdministratorAccess hoặc s3:* cho tài khoản dịch vụ của pipeline! Nếu khóa API của pipeline bị lộ ra ngoài, hacker có thể xóa sạch toàn bộ cơ sở dữ liệu của công ty.

Quy tắc vàng: Chỉ cấp chính xác những quyền hạn tối thiểu cần thiết để hoàn thành công việc:

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": ["s3:GetObject", "s3:PutObject"],
7       "Resource":
8         "arn:aws:s3:::ecommerce-landing-zone/daily_orders/*"
9     }
10  ]
11 }
```

Chính sách này chỉ cho phép đọc và ghi trong thư mục `daily_orders/`, không cho phép xóa (`s3:DeleteObject`) và không cho phép truy cập bất kỳ bucket nhạy cảm nào khác!

12.4 Quản Trị Chi Phí (FinOps Cho Kỹ Sư Dữ Liệu)

Trong các hệ thống Data Warehouse hiện đại như BigQuery hay AWS Athena, chi phí không tính theo giờ bật máy mà tính theo ****Lượng dữ liệu được quét (Data Scanned)****: Mỗi 1 Terabyte quét qua tốn 5 USD.

- Nếu bảng dữ liệu của bạn có 10TB và một bạn Data Analyst chạy câu lệnh: `SELECT * FROM clickstream;`, công ty sẽ mất ngay **50 USD** cho một truy vấn duy nhất!
- **Giải pháp Phân Vùng (Partitioning)**: Phân vùng bảng theo cột ngày (`PARTITION BY event_date`). Khi câu lệnh có điều kiện `WHERE event_date = '2026-03-20'`, BigQuery chỉ quét duy nhất partition của ngày hôm đó (ví dụ 10GB thay vì 10TB) → Chi phí giảm ngay **99.9%**, từ 50 USD xuống còn 0.05 USD!

12.5 PROJECT 9: Serverless Cloud Data Platform

Thiết kế kiến trúc và triển khai nền tảng dữ liệu đám mây hoàn chỉnh:

1. Khởi tạo Bucket Amazon S3 lưu trữ các tầng dữ liệu: `raw/`, `processed/`, và `analytics/`.
2. Cấu hình vòng đời dữ liệu (S3 Lifecycle Policy) tự động chuyển các tệp sau 90 ngày sang lớp lưu trữ giá rẻ S3 Glacier.

- Sử dụng AWS Athena hoặc Google BigQuery để truy vấn trực tiếp trên các tệp Parquet mà không cần dựng máy chủ cơ sở dữ liệu.

12.6 Bài Tập Tự Luyện Level 9

Bài Tập Tự Luyện: 5 Bài Tập Hạ Tầng Đám Mây & Tối Ưu Chi Phí

- Bài 1 (IAM Policy):** Viết một chính sách IAM Policy dạng JSON cho phép một Lambda Function có quyền ghi kết quả vào bucket `s3://analytics-gold-mart` nhưng cấm tuyệt đối mọi hành vi xóa file (`Deny s3:DeleteObject`).
- Bài 2 (BigQuery Partitioning & Clustering):** Viết câu lệnh SQL DDL trong BigQuery tạo bảng `user_events` phân vùng theo ngày (`event_date`) và gom cụm (Clustering) theo hai cột `user_id` và `event_type`.
- Bài 3 (S3 Storage Classes):** So sánh chi phí lưu trữ và độ trễ truy xuất giữa 3 lớp lưu trữ của AWS: S3 Standard, S3 Standard-IA (Infrequent Access), và S3 Glacier Flexible Retrieval.
- Bài 4 (Cloud Architecture):** Vẽ sơ đồ kiến trúc luồng dữ liệu (Architecture Flow) nhận dữ liệu đơn hàng từ ứng dụng di động qua API Gateway, chuyển tiếp qua SQS Queue và ghi nhận vào hồ dữ liệu.
- Bài 5 (Bảo mật Secrets):** Trình bày cách một script Python ETL trên Cloud đọc mật khẩu kết nối cơ sở dữ liệu từ AWS Secrets Manager hoặc HashiCorp Vault thay vì viết cứng (hardcode) mật khẩu trong file code.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- Bài 1:** Khai báo 2 statement: Statement 1 "Effect": "Allow" cho "s3:PutObject" trên bucket chỉ định, Statement 2 "Effect": "Deny" cho "s3:Delete*" trên toàn bộ tài nguyên.

- Bài 2:**

```

1 CREATE TABLE `project.dataset.user_events` (
2     event_id STRING,
3     user_id STRING,
4     event_type STRING,
5     event_date DATE,
6     payload JSON
7 )
8 PARTITION BY event_date
9 CLUSTER BY user_id, event_type;
10

```

- Bài 3:** S3 Standard: \$0.023/GB/tháng, truy xuất mili-giây. S3 Standard-IA:

\$0.0125/GB/tháng (rẻ gần 50%), truy xuất mili-giây nhưng có phí đọc dữ liệu. S3 Glacier: \$0.004/GB/tháng (tiết kiệm 80%), nhưng mất từ 1 đến 5 giờ để giải nén dữ liệu.

- **Bài 4:** Luồng chuẩn: Mobile App → AWS API Gateway → Amazon SQS (Đệm tải để chống nghẽn khi có flash sale) → AWS Lambda → S3 Data Lake.
- **Bài 5:** Dùng SDK `boto3.client('secretsmanager')` gọi hàm `get_secret_value(SecretId='prod/dwh/postgres')`, parse chuỗi JSON trả về và nạp vào cấu hình kết nối của SQLAlchemy/psycopg2.

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Kỹ Năng Cloud Vào CV

Serverless Cloud Data Platform | AWS (S3, Lambda, Athena, IAM), BigQuery, FinOps

- Thiết kế kiến trúc hồ dữ liệu (Data Lake) trên Amazon S3 với chính sách phân lớp vòng đời (Lifecycle Policies) tự động lưu trữ lạnh, giúp cắt giảm 45% chi phí lưu trữ hạ tầng hàng tháng.
- Triển khai pipeline không máy chủ (Serverless Ingestion) bằng AWS Lambda và Amazon S3 Event Notifications, xử lý tự động hơn 500 tệp giao dịch hàng ngày với thời gian phản hồi dưới 3 giây.
- Thiết lập cơ chế phân vùng (Partitioning) và gom cụm (Clustering) trên Google BigQuery/AWS Athena, tối ưu hóa các truy vấn phân tích giúp giảm 85% lượng dữ liệu quét (Scan Bytes).

Chương 13

Level 10: MLOps & Đưa Mô Hình Lên Production (Tuần 55–58)

13.1 Khoảng Trống Giữa Jupyter Notebook & Hệ Thống Sản Xuất

Thống kê của Gartner chỉ ra rằng hơn **85% mô hình Machine Learning không bao giờ được đưa vào sản xuất**. Lý do không phải vì thuật toán toán học dở, mà vì các nhà khoa học dữ liệu quen làm việc trong môi trường Jupyter Notebook:

- Các ô mã (cells) chạy lộn xộn, không theo thứ tự tuyến tính, không thể tái lập (Unreproducible).
- Không có cơ chế kiểm thử tự động (Unit Tests / Integration Tests).
- Không có giao diện lập trình ứng dụng (API) để các hệ thống khác (Web, Mobile App) gọi lấy kết quả dự báo thời gian thực.
- Không có công cụ theo dõi xem mô hình có bị "lão hóa" và suy giảm độ chính xác theo thời gian hay không.

****MLOps (Machine Learning Operations)**** chính là cây cầu kỹ thuật đưa mô hình từ một tệp thử nghiệm trong máy cá nhân trở thành một dịch vụ doanh nghiệp có độ tin cậy 99.9%.

13.2 MLflow: Quản Lý Vòng Đời Thử Nghiệm & Model Registry

Thực Chiến & Best Practices: Tự Động Theo Dõi Thử Nghiệm Với MLflow

Thay vì ghi chép các siêu tham số (Hyperparameters) và độ chính xác vào file Excel một cách thủ công, thư viện ****MLflow**** tự động lưu trữ toàn bộ lịch sử thử nghiệm:

```
1 import mlflow
2 import mlflow.sklearn
3 from sklearn.ensemble import RandomForestClassifier
```

```

4 from sklearn.metrics import roc_auc_score
5
6 mlflow.set_experiment("Customer_Churn_Prediction")
7
8 with mlflow.start_run(run_name="RandomForest_Depth8_Estimators200"):
9     # 1. Ghi lại các tham số huấn luyện (Parameters)
10    params = {"n_estimators": 200, "max_depth": 8, "random_state":
11    42}
12    mlflow.log_params(params)
13
14    # 2. Huấn luyện mô hình
15    model = RandomForestClassifier(**params)
16    model.fit(X_train, y_train)
17
18    # 3. Tính toán và ghi lại các chỉ số đánh giá (Metrics)
19    y_pred_proba = model.predict_proba(X_test)[: , 1]
20    auc_score = roc_auc_score(y_test, y_pred_proba)
21    mlflow.log_metric("test_roc_auc", auc_score)
22    print(f"[*] Kết quả AUC lưu vào MLflow: {auc_score:.4f}")
23
24    # 4. Lưu trữ mô hình artifact và đăng ký vào Model Registry
25    mlflow.sklearn.log_model(
26        sk_model=model,
27        artifact_path="churn_model",
28        registered_model_name="Production_Churn_Classifier"
29    )

```

13.3 Phục Vụ Mô Hình Thời Gian Thực Với FastAPI

Trong sản xuất, ứng dụng di động cần biết ngay lập tức một khách hàng có nguy cơ rời bỏ hay không khi họ vừa mở app. ****FastAPI**** là framework Python hiện đại nhất hiện nay nhờ tốc độ thực thi tiệm cận Go/NodeJS và khả năng tự động kiểm tra dữ liệu bằng Pydantic:

Thực Chiến & Best Practices: Mã Nguồn Microservice Dự Báo Hoàn Chỉnh (FastAPI)

```

1 from fastapi import FastAPI, HTTPException
2 from pydantic import BaseModel, Field
3 import joblib
4 import pandas as pd
5
6 app = FastAPI(
7     title="Customer Churn Prediction API",
8     description="Dịch vụ Microservice dự báo nguy cơ rời bỏ khách hàng thời gian thực",

```

```

9     version="1.0.0"
10 )
11
12 # 1. Tai Pipeline da dong goi khi khai dong server
13 model_pipeline = joblib.load("models/churn_prediction_v1.joblib")
14
15 # 2. Dinh nghĩa khuôn mau du lieu dau vao (Input Contract)
16 class CustomerFeatures(BaseModel):
17     tenure_months: int = Field(ge=0, description="Số tháng sử dụng dịch vụ")
18     monthly_charges: float = Field(gt=0, description="Chi phí hàng tháng")
19     total_transactions: int = Field(ge=0, description="Tổng số giao dịch")
20     days_since_last_login: int = Field(ge=0, description="Số ngày từ lần đăng nhập cuối")
21     contract_type: str = Field(pattern="^(Month-to-Month|One-Year|Two-Year)$")
22     payment_method: str
23     gender: str
24
25 # 3. Dinh nghĩa kết quả trả về (Output Response)
26 class PredictionResponse(BaseModel):
27     churn_probability: float
28     is_churn_risk: bool
29     recommended_action: str
30
31 @app.post("/predict", response_model=PredictionResponse)
32 def predict_churn(customer: CustomerFeatures):
33     try:
34         # Chuyển dữ liệu Pydantic thành DataFrame 1 dòng
35         input_data = pd.DataFrame([customer.model_dump()])
36
37         # Tính toán xác suất rời bỏ
38         prob = float(model_pipeline.predict_proba(input_data)[0, 1])
39         is_risk = prob >= 0.40
40
41         action = "Tặng Voucher giữ chân 15%" if is_risk else "Chăm sóc tiêu chuẩn"
42
43         return PredictionResponse(
44             churn_probability=round(prob, 4),
45             is_churn_risk=is_risk,
46             recommended_action=action
47         )
48     except Exception as e:

```

```

49         raise HTTPException(status_code=500, detail=f"Loi du bao noi
      bo: {str(e)}")
50
51 @app.get("/health")
52 def health_check():
53     return {"status": "HEALTHY", "model_version": "v1.0.0"}

```

13.4 Đóng Gói Docker & CI/CD Với GitHub Actions

Thực Chiến & Best Practices: Tập Workflow CI/CD Tự Động Hóa (.github/workflows/mlops.yml)

Mỗi khi bạn đẩy (push) code mới lên nhánh main, GitHub Actions sẽ tự động khởi chạy môi trường kiểm thử và đóng gói Docker Image:

```

1 name: ML Model CI/CD Pipeline
2
3 on:
4   push:
5     branches: [ main ]
6   pull_request:
7     branches: [ main ]
8
9 jobs:
10  test_and_lint:
11    runs-on: ubuntu-latest
12    steps:
13      - uses: actions/checkout@v3
14      - name: Set up Python 3.11
15        uses: actions/setup-python@v4
16        with:
17          python-version: '3.11'
18      - name: Install Dependencies
19        run: |
20          python -m pip install --upgrade pip
21          pip install -r requirements.txt
22          pip install pytest flake8
23      - name: Lint Code with Flake8
24        run: flake8 src/ --count --max-line-length=120
25      - name: Run Unit Tests with Pytest
26        run: pytest tests/
27
28  build_docker_image:
29    needs: test_and_lint

```

```
30     runs-on: ubuntu-latest
31     steps:
32     - uses: actions/checkout@v3
33     - name: Build Docker Image
34       run: docker build -t my-ml-churn-api:latest .
```

13.5 Giám Sát Mô Hình: Data Drift & Concept Drift

Sau khi đưa mô hình lên sản xuất, chất lượng của nó sẽ luôn luôn bị suy giảm theo thời gian (Model Degradation):

- **Data Drift (Lệch Dữ Liệu):** Phân phối của dữ liệu đầu vào $P(X)$ thay đổi. Ví dụ: Trước đây khách hàng chủ yếu trong độ tuổi 20–30, nay công ty chạy chiến dịch mở rộng sang nhóm khách hàng trên 50 tuổi. Phân phối độ tuổi bị lệch hẳn!
- **Concept Drift (Lệch Khái Niệm):** Mối quan hệ giữa dữ liệu đầu vào và nhãn đầu ra $P(y|X)$ thay đổi. Ví dụ: Lạm phát kinh tế khiến một người có mức thu nhập 20 triệu trước đây thuộc nhóm chi tiêu thoải mái nay lại cắt giảm chi tiêu tối đa.
- **Công cụ giám sát:** Sử dụng kiểm định thống kê **Kolmogorov-Smirnov Test (KS-Test)** hoặc chỉ số **Population Stability Index (PSI)**. Nếu giá trị $PSI > 0.25$, hệ thống cảnh báo tự động phát tín hiệu yêu cầu tái huấn luyện (Retrain) mô hình trên dữ liệu mới!

13.6 PROJECT 10: Production ML Churn Prediction Microservice

Xây dựng hệ thống phục vụ mô hình hoàn chỉnh:

1. Đóng gói mô hình Machine Learning tốt nhất vào MLflow Model Registry.
2. Xây dựng Microservice REST API bằng FastAPI với schema Pydantic.
3. Viết bộ kiểm thử tự động `pytest` bao gồm kiểm tra tính hợp lệ của response mã HTTP 200 và độ trễ phản hồi dưới 50ms.
4. Đóng gói Dockerfile đa tầng và thiết lập pipeline GitHub Actions tự động kiểm thử mỗi khi có commit.

13.7 Bài Tập Tự Luyện Level 10

Bài Tập Tự Luyện: 5 Bài Tập MLOps & Production Engineering

- Bài 1 (Pydantic Validation):** Viết một schema Pydantic kiểm tra trường email phải đúng cú pháp RFC và trường credit_score phải nằm trong đoạn [300, 850]. Bắn ra lỗi cụ thể nếu vi phạm.
- Bài 2 (Pytest cho Machine Learning):** Viết một test case bằng thư viện pytest kiểm tra tính chất đơn điệu (Directional Invariance): Khi tăng trường days_since_last_login từ 5 ngày lên 90 ngày (trong khi giữ nguyên các biến khác), xác suất rời bỏ (churn_probability) của khách hàng bắt buộc phải tăng lên.
- Bài 3 (FastAPI Async):** Viết một endpoint /predict_batch trong FastAPI hỗ trợ nhận vào danh sách lên tới 1,000 khách hàng cùng lúc và thực hiện dự báo theo lô (Batch Inference) tối ưu.
- Bài 4 (PSI Calculation):** Viết hàm Python tính toán chỉ số Population Stability Index (PSI) giữa tập dữ liệu huấn luyện tháng 1 và tập dữ liệu phục vụ thực tế tháng 6.
- Bài 5 (Docker Healthcheck):** Thêm lệnh HEALTHCHECK vào file Dockerfile để Docker daemon tự động kiểm tra endpoint /health của FastAPI mỗi 30 giây một lần.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Gợi Ý Kiểm Tra

- Bài 1:** Dùng kiểu dữ liệu EmailStr (từ thư viện pydantic[email]) và Field(ge=300, le=850).
- Bài 2:** Tạo 2 đối tượng mẫu: cust1 có days=5 và cust2 có days=90. Gọi model.predict_proba và kiểm tra assert prob2 > prob1.
- Bài 3:** Nhận tham số customers: List[CustomerFeatures], dùng pd.DataFrame([c.model_dump() for c in customers]) và gọi hàm predict_proba 1 lần duy nhất cho toàn bộ bảng.
- Bài 4:** Công thức PSI:

$$PSI = \sum (Actual\% - Expected\%) \times \ln \left(\frac{Actual\%}{Expected\%} \right)$$

- Bài 5:** Dòng lệnh trong Dockerfile:

```
1 HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
2   CMD curl -f http://localhost:8000/health || exit 1
3
```

Góc Nhìn Tuyển Dụng & Đưa Vào CV: Cách Ghi Kỹ Năng MLOps Vào CV**Production Machine Learning Microservice | FastAPI, Docker, MLflow, CI/CD**

- Đóng gói và triển khai mô hình học máy phân loại rời bỏ dưới dạng RESTful API Microservice sử dụng FastAPI và Pydantic, đạt độ trễ phản hồi dự báo thời gian thực dưới 25ms.
- Quản lý vòng đời mô hình và theo dõi siêu tham số thử nghiệm thông qua MLflow Model Registry, đảm bảo khả năng tái lập và kiểm toán phiên bản mô hình 100%.
- Xây dựng đường ống tự động hóa kiểm thử và triển khai liên tục (CI/CD) với GitHub Actions và Docker, thiết lập bộ kiểm định chất lượng mã nguồn flake8 và kiểm thử chức năng mô hình tự động bằng pytest.

Đồ Án Flagship Capstone: Modern Enterprise Data & AI Platform (Project 11)

14.1 Tầm Nhìn Của Dự Án Flagship Capstone

Đa phần ứng viên trên thị trường nộp vào CV những đồ án rời rạc: một file notebook phân tích dữ liệu Titanic, một file SQL vài câu lệnh cơ bản, hoặc một biểu đồ dashboard đơn điệu. Những dự án này không chứng minh được bạn có khả năng làm việc trong một môi trường doanh nghiệp thực tế.

Dự Án Flagship Capstone (Project 11) được thiết kế để trở thành ****tâm điểm sáng nhất trong Portfolio của bạn****. Đây là một nền tảng dữ liệu và trí tuệ nhân tạo hoàn chỉnh từ đầu đến cuối (End-to-End Platform), tích hợp mượt mà cả ba trụ cột:

1. **Data Engineering:** Tự động hóa trích xuất từ nhiều nguồn, nạp vào Data Lake S3/MinIO, điều phối bằng Apache Airflow, mô hình hóa và kiểm thử tự động bằng dbt, tối ưu hóa lưu trữ trong Data Warehouse PostgreSQL.
2. **Data Science:** Huấn luyện và theo dõi mô hình học máy phân loại nguy cơ rời bỏ khách hàng (Churn Prediction) với MLflow, đóng gói thành Microservice FastAPI phục vụ thời gian thực qua container Docker.
3. **Data Analytics:** Xây dựng mô hình Star Schema chuẩn Kimball, tính toán các chỉ số tài chính nâng cao bằng DAX và thiết kế Executive Dashboard trên Power BI phục vụ ban lãnh đạo.

14.2 Kiến Trúc Kỹ Thuật Toàn Cảnh (End-to-End Architecture)

Kiến Trúc Cốt Lõi: Luồng Dữ Liệu 6 Tầng Chuẩn Doanh Nghiệp Hiện Đại

1. **Tầng Nguồn (Source Systems):** Hệ thống giả lập các giao dịch E-commerce phát sinh liên tục (REST APIs) và tệp dữ liệu khách hàng CRM (CSV/Parquet).
2. **Tầng Hồ Dữ Liệu Thô (Bronze Layer - Data Lake):** Dữ liệu thô chưa qua xử lý được lưu

trữ nguyên trạng trên MinIO (tương thích hoàn toàn với Amazon S3) theo cấu trúc thư mục `raw/year=2026/month=03/`.

3. **Tầng Điều Phối (Orchestration):** ****Apache Airflow**** kích hoạt DAG theo lịch định kỳ, kiểm tra tính khả dụng của file nguồn, thực thi xác thực dữ liệu và gọi dbt.

4. **Tầng Biến Đổi Dữ Liệu (Silver & Gold Layers - dbt):**

- *Silver Layer (Staging):* dbt làm sạch dữ liệu, khử trùng lặp, chuẩn hóa kiểu ngày tháng và tên cột.
- *Gold Layer (Marts):* dbt xây dựng các bảng Fact và Dimension (Star Schema), thực thi hơn 30 bài kiểm định chất lượng tự động (dbt test).

5. **Tầng Phục Vụ Máy Học (Machine Learning Layer):** Mô hình Scikit-Learn/XGBoost lấy dữ liệu từ Gold Layer, trích xuất đặc trưng RFM, tính toán xác suất rời bỏ và xuất ra REST API FastAPI với độ trễ dưới 25ms.

6. **Tầng Ra Quyết Định Kinh Doanh (BI & Consumption Layer):** ****Power BI**** kết nối trực tiếp vào Data Warehouse để hiển thị các chỉ số kinh doanh cốt lõi (Doanh thu, Tăng trưởng, Khách hàng trung thành, Danh sách cảnh báo rủi ro).

14.3 Cấu Trúc Repository Chuẩn Doanh Nghiệp (Enterprise Monorepo)

Một dự án chuyên nghiệp phải được tổ chức khoa học để bất kỳ kỹ sư nào trong nhóm cũng có thể khởi chạy toàn bộ hệ sinh thái chỉ bằng một lệnh duy nhất:

```

1 modern-ecommerce-data-platform/|—
2 .github/|
3   └─ workflows/|
4     └─ ci_cd.yml # Pipeline kiểm thu và build tu dong|—
5 airflow/|
6   └─ dags/|
7     └─ ecommerce_master_dag.py # DAG điều phối chính|
8   └─ plugins/|—
9 dbt_ecommerce/|
10  └─ dbt_project.yml|
11  └─ models/|
12    └─ staging/ # Silver Layer|
13    └─ marts/ # Gold Layer (Star Schema)|
14  └─ tests/ # Kiểm định toàn vẹn dữ liệu|—
15 ml_service/|
16  └─ src/|
17    └─ app.py # FastAPI microservice|
18    └─ model_trainer.py # Script huấn luyện và log MLflow|
19  └─ tests/|
20    └─ test_api.py # Unit tests với pytest|

```

```

21   └─ Dockerfile ─┐
22  dashboards/ ─┐
23   └─ executive_performance.pbix # File Power BI Dashboard hoàn chỉnh ─┐
24  data/ ─┐      # Du lieu mau mock testing ─┐
25  docker-compose.yml ─┐      # Khoi chay Postgres, Airflow, MinIO,
    FastAPI ─┐
26  Makefile ─┐      # Lenh tat thao tac: make up, make test,
    make run ─┐
27  requirements.txt ─┐
28  README.md ─┐      # Tai lieu huong dan dinh cao
    (Architecture + Metrics)

```

14.4 Kịch Bản 15 Phút Trình Bày Dự Án Chính Phục Nhà Tuyển Dụng

Khi người phỏng vấn đề nghị: "Em hãy trình bày dự án ấn tượng nhất mà em từng xây dựng", hãy áp dụng kịch bản 4 phần:

- Phần 1: Bối Cảnh Nghiệp Vụ & Mục Tiêu (3 phút):** "Em nhận thấy bài toán phổ biến của các sàn TMĐT là dữ liệu giao dịch bị phân mảnh ở nhiều nơi khiến báo cáo kinh doanh bị chậm trễ và không kịp thời giữ chân khách hàng có nguy cơ rời bỏ. Vì vậy, em đã tự mình thiết kế và xây dựng một nền tảng dữ liệu toàn diện Modern Data Platform xử lý hơn 100,000 giao dịch/ngày, tích hợp giữa hạ tầng dữ liệu và mô hình trí tuệ nhân tạo."
- Phần 2: Kiến Trúc Kỹ Thuật & Đánh Đổi Lựa Chọn (5 phút):** Giải thích vì sao bạn chọn công nghệ này mà không chọn công nghệ khác:
 - "Em chọn mô hình ELT kết hợp dbt thay vì viết code Python biến đổi thủ công vì dbt tận dụng tối đa sức mạnh tính toán song song của DWH, tự động quản lý lineage và tích hợp sẵn bộ kiểm thử dữ liệu."
 - "Em thiết kế Data Warehouse theo mô hình Star Schema của Ralph Kimball với các Surrogate Keys để tách biệt hoàn toàn dữ liệu vận hành khỏi dữ liệu phân tích lịch sử."
- Phần 3: Trình Diễn Trực Tiếp (Live Demo) (5 phút):** Mở giao diện Webserver của Airflow cho thấy DAG chạy xanh hoàn toàn; mở terminal chạy lệnh gọi API FastAPI cho thấy kết quả dự báo churn trả về trong 18ms; mở file Power BI cho thấy các biểu đồ tương tác mượt mà.
- Phần 4: Số Liệu Đo Lường Thực Tế & Bài Học Kinh Nghiệm (2 phút):** Báo cáo các con số ấn tượng: Thời gian chạy pipeline giảm 65%, độ chính xác mô hình đạt ROC-AUC 0.88, và giải thích cách bạn xử lý sự cố khi tệp nguồn bị mất kết nối mạng (Network Retry & Dead Letter Queue).

Chiến Lược Nghề Nghiệp, Bộ CV Chuẩn ATS & 40 Câu Hỏi Phỏng Vấn Sát Thủ

15.1 Chân Dung Năng Lực Tuyển Dụng: Intern vs Junior vs Middle

Rất nhiều người tự học lầm tưởng rằng chỉ cần học xong một khóa học hoặc một cuốn sách là có thể tự nhận mình là "Middle Data Engineer" hay "Middle Data Scientist". Trong thực tế ngành dữ liệu, ranh giới giữa các cấp bậc được định hình bởi **kinh nghiệm giải quyết sự cố thực tế và trách nhiệm làm chủ (Ownership)**:

Kiến Thức Cốt Lõi: Sự Khác Biệt Bản Chất Giữa Các Cấp Bậc

• Intern / Fresher (Tuần 1–24):

- *Kỳ vọng*: Nền tảng SQL và Python vững chắc; tư duy logic trong sáng; hiểu bản chất dữ liệu; thái độ cầu tiến và có khả năng đọc tài liệu kỹ thuật.
- *Bằng chứng cần có*: Portfolio 3–4 dự án hoàn chỉnh trên GitHub có README chin chu, có thể giải thích từng dòng code mình đã viết.

• Junior (1–2 năm kinh nghiệm):

- *Kỳ vọng*: Hoàn thành độc lập các tính năng hoặc báo cáo được giao mà không cần người kèm cặp liên tục; tuân thủ quy chuẩn viết code, kiểm thử (Testing) và quy trình Git Flow; chủ động debug khi pipeline/mô hình gặp lỗi.

• Middle (2–4+ năm kinh nghiệm):

- *Kỳ vọng*: **Tư duy làm chủ (Total Ownership)** một hệ thống con (Subsystem); tham gia thiết kế kiến trúc (System Architecture); cân nhắc các bài toán đánh đổi (Trade-offs: Chi phí vs Tốc độ vs Độ tin cậy); trực tiếp làm việc với các bên kinh doanh để định nghĩa yêu cầu và chuyển hóa thành giải pháp kỹ thuật.

15.2 Công Thức Vàng Viết Bullet Point CV Chuẩn ATS

Hệ thống lọc hồ sơ tự động (Applicant Tracking System - ATS) và các Trưởng nhóm Kỹ thuật (Tech Leads) chỉ dành trung bình 6 giây để lướt qua một bản CV. Những dòng mô tả chung chung như: *"Tham gia phân tích dữ liệu bán hàng bằng Python và SQL"* sẽ bị loại ngay lập tức!

Thực Chiến & Best Practices: Công Thức 5 Thành Phần: Action + What + Tech + Scale + Result

Mỗi gạch đầu dòng trong phần Dự án của CV phải tuân thủ nghiêm ngặt công thức:

[Hành Động] + [Cái Gì] + [Công Nghệ] + [Quy Mô] + [Kết Quả Đo Lường]

So sánh các ví dụ thực tế:

- **Cách viết yếu:** *"Em viết câu lệnh SQL để truy vấn dữ liệu đơn hàng và làm dashboard Power BI cho công ty."*
- **Cách viết chuẩn mực:** *"Thiết kế kiến trúc cơ sở dữ liệu quan hệ gồm 5 bảng và viết hơn 40 truy vấn SQL nâng cao (CTEs, Window Functions), xây dựng Executive Dashboard trên Power BI hỗ trợ theo dõi 8 chỉ số kinh doanh chính cho hơn 120,000 giao dịch bán lẻ hàng tháng."*
- **Cách viết yếu:** *"Xây dựng mô hình máy học dự đoán khách hàng rời bỏ bằng Scikit-learn."*
- **Cách viết chuẩn mực:** *"Huấn luyện và tinh chỉnh mô hình XGBoost phân loại khách hàng rời bỏ trên tập dữ liệu 150,000 bản ghi, tối ưu hóa qua Stratified 5-Fold Cross Validation đạt ROC-AUC 0.88, đóng gói thành Microservice FastAPI với độ trễ phản hồi dưới 25ms."*

15.3 Bộ 3 Mẫu CV Chuẩn ATS (Sẵn Sàng Ứng Tuyển)

15.3.1 Mẫu 1: Data Analyst Intern / Fresher CV

```

1  NGUYEN VAN A
2  Hanoi, Vietnam | (+84) 912 345 678 | anhtuan.data@email.com
3  github.com/anhtuan-data | linkedin.com/in/anhtuan-data
4
5  OBJECTIVE
6  Motivated Data Analyst candidate with a strong foundation in Mathematics &
   Computer Science, proficient in SQL, Python (Pandas, NumPy),
   Statistics, and Power BI. Eager to leverage analytical skills and
   hands-on portfolio experience to deliver actionable business insights
   for enterprise growth.
7
8  EDUCATION
9  Hanoi University of Science and Technology (HUST)                2022 -
   2026
  
```

10 Bachelor of Science in Mathematics and Computer Science
 11 GPA: 3.4/4.0 | Relevant Coursework: Probability & Statistics, Database
 Systems, Data Structures
 12
 13 TECHNICAL SKILLS
 14 - Querying & Databases: SQL (PostgreSQL), Complex Joins, CTEs, Window
 Functions, Indexing
 15 - Programming & Analysis: Python (Pandas, NumPy, SciPy), Data Cleaning,
 EDA, Git/GitHub
 16 - Business Intelligence: Power BI, DAX (CALCULATE, Time Intelligence),
 Star Schema Modeling, Excel
 17 - Statistics: Hypothesis Testing, A/B Testing (Sample size, SRM,
 Two-proportion Z-test), Regression
 18
 19 FEATURED PROJECTS
 20 E-Commerce Business Analytics & Executive Dashboard | SQL, Power BI, Python
 21 - Extracted and transformed over 100,000 raw transaction records from
 PostgreSQL using advanced SQL queries with CTEs and Window Functions
 (ROW_NUMBER, LAG) to analyze monthly revenue trends.
 22 - Designed a star-schema data model with 1 Fact and 4 Dimension tables in
 Power BI, implementing 15+ custom DAX measures (YoY Growth, Rolling
 Averages, Customer Lifetime Value).
 23 - Built an interactive 3-tier Executive Dashboard delivering critical
 insights into customer retention and product sales, identifying that
 top 20% of customers generated 74% of total company revenue.
 24
 25 A/B Testing Conversion Optimization Experiment | Python, SciPy, Statsmodels
 26 - Designed and evaluated an end-to-end A/B test on 50,000 user sessions to
 measure the impact of a redesigned checkout flow on purchase
 conversion rate.
 27 - Conducted pre-experiment sample size sizing (Power = 80%, alpha = 0.05)
 and checked for Sample Ratio Mismatch (SRM) using Chi-square
 goodness-of-fit testing.
 28 - Performed Two-proportion Z-test and computed 95% confidence intervals,
 demonstrating a statistically significant +8.3% relative conversion
 lift (p = 0.004) to support executive rollout decisions.

15.3.2 Mẫu 2: Data Engineer Intern / Junior CV

1 TRAN VAN B
 2 Ho Chi Minh City, Vietnam | (+84) 987 654 321 | vanb.de@email.com
 3 github.com/vanb-dataeng | linkedin.com/in/vanb-de
 4
 5 TECHNICAL SKILLS
 6 - Data Pipelines & Orchestration: Python, Apache Airflow (TaskFlow API),
 dbt, Bash Scripting
 7 - Storage & Warehousing: PostgreSQL, MinIO/Amazon S3, Kimball Dimensional
 Modeling (Star Schema, SCD2)

```

8 - Big Data & Cloud: PySpark (DataFrames, Broadcast Join), AWS (S3, Lambda,
  Athena), Docker, Linux
9 - Data Quality & DevOps: Pydantic, dbt tests, Git, GitHub Actions (CI/CD)
10
11 FEATURED PROJECTS
12 Production-Grade Analytics Engineering Pipeline | Docker, Airflow, dbt,
  PostgreSQL
13 - Containerized a full-stack data environment running PostgreSQL, Apache
  Airflow, and dbt using Docker Compose to ensure 100% reproducible
  local and production setups.
14 - Designed and deployed daily Airflow DAGs extracting batch JSON data from
  REST APIs into raw staging storage with automated retries and failure
  notification webhooks.
15 - Implemented 18 dbt transformation models across Staging, Intermediate,
  and Mart layers with incremental processing, cutting warehouse
  transformation runtime by 55%.
16 - Established automated data quality assertions (unique, not_null,
  relationship tests) using dbt test, preventing corrupted records from
  entering production reporting tables.
17
18 Big Data Lakehouse Pipeline | PySpark, Delta Lake, AWS S3
19 - Processed 10M+ rows of clickstream log data using PySpark, performing
  sessionization and user engagement aggregations with Spark SQL Window
  Functions.
20 - Applied partition pruning and Broadcast Hash Joins to eliminate
  cross-network data shuffle, reducing job execution time from 90
  minutes to under 18 minutes.
21 - Implemented Delta Lake ACID transaction tables with Z-Ordering on
  timestamp and user ID, accelerating downstream analytical query
  performance by 4x.

```

15.3.3 Mẫu 3: Data Scientist Intern / Junior CV

```

1 LE THI C
2 Hanoi, Vietnam | (+84) 901 234 567 | lethic.ds@email.com
3 github.com/lethic-datascience | linkedin.com/in/lethic-ds
4
5 TECHNICAL SKILLS
6 - Machine Learning: Scikit-learn, XGBoost, Random Forest, Logistic
  Regression, Feature Engineering
7 - Statistical Analysis: Hypothesis Testing, A/B Testing, Dimensionality
  Reduction (PCA), SHAP
8 - Deployment & MLOps: FastAPI, Docker, MLflow (Experiment Tracking, Model
  Registry), Git, CI/CD
9 - Programming & DB: Python (NumPy, Pandas, Matplotlib, Seaborn), SQL
  (PostgreSQL)
10
11 FEATURED PROJECTS

```

- 12 Customer Churn Prediction & Model Serving Microservice | Python, XGBoost, FastAPI, Docker
- 13 - Built an end-to-end customer churn prediction pipeline on 120,000 customer records, resolving class imbalance using class-weighted loss functions.
- 14 - Engineered 25+ behavioral features and bundled preprocessing transformations using Scikit-Learn Pipeline and ColumnTransformer to completely eliminate data leakage risks.
- 15 - Tuned ensemble classifiers (XGBoost, Random Forest) via Stratified 5-Fold Cross Validation, achieving an ROC-AUC score of 0.88 and Recall of 76% on holdout test data.
- 16 - Leveraged SHAP values to explain individual feature contributions, identifying tenure and contract type as key retention drivers for targeted marketing campaigns.
- 17 - Productionized the trained pipeline into a containerized FastAPI REST microservice with Pydantic request validation, serving real-time inference with sub-25ms latency.

15.4 Ngân Hàng 40 Câu Hỏi Phỏng Vấn "Sát Thủ" Kèm Lời Giải

15.4.1 10 Câu Hỏi Cốt Lõi Về SQL & Cơ Sở Dữ Liệu

- Câu 1:** "Phân biệt sự khác nhau giữa *WHERE* và *HAVING* trong *SQL*?"

→ **Trả lời:** *WHERE* lọc dữ liệu ở cấp độ từng dòng TRƯỚC KHI các phép toán gộp nhóm (*GROUP BY*) được thực hiện và không thể chứa các hàm tổng hợp (*SUM*, *COUNT*). Ngược lại, *HAVING* lọc dữ liệu ở cấp độ nhóm SAU KHI phép gộp nhóm đã hoàn thành và chuyên dùng để lọc dựa trên kết quả của hàm tổng hợp.
- Câu 2:** "Window Functions khác gì so với *GROUP BY*?"

→ **Trả lời:** Cả hai đều tính toán trên một tập hợp dữ liệu, nhưng *GROUP BY* gom nhiều dòng lại thành một dòng duy nhất đại diện cho nhóm, trong khi **Window Function** giữ nguyên từng dòng ban đầu của bảng dữ liệu và trả kết quả tính toán vào một cột mới trên từng dòng đó.
- Câu 3:** "Phân biệt *ROW_NUMBER()*, *RANK()* và *DENSE_RANK()*?"

→ **Trả lời:** Khi có các giá trị bằng nhau: *ROW_NUMBER* luôn đánh số liên tục không trùng (1, 2, 3, 4); *RANK* cho các giá trị bằng nhau cùng chung thứ tự nhưng sẽ nhảy cóc thứ bậc tiếp theo (1, 2, 2, 4); *DENSE_RANK* cho các giá trị bằng nhau cùng thứ bậc và KHÔNG nhảy cóc thứ bậc tiếp theo (1, 2, 2, 3).
- Câu 4:** "Index B-Tree hoạt động như thế nào và tại sao nó giúp tăng tốc độ truy vấn?"

→ **Trả lời:** B-Tree là cấu trúc cây tự cân bằng lưu trữ dữ liệu theo thứ tự đã sắp xếp. Thay vì phải quét tuần tự toàn bộ bảng với độ phức tạp $O(n)$, B-Tree cho phép tìm kiếm dữ liệu qua việc duyệt các nút từ gốc đến lá theo phép chia nhị phân với độ phức tạp chỉ $O(\log n)$.
- Câu 5:** "Khi nào một câu lệnh *SQL* không thể tận dụng được Index?"

→ **Trả lời:** Khi truy vấn bọc hàm tính toán quanh cột có index (ví dụ *WHERE YEAR(date_col)*

= 2026); khi sử dụng toán tử LIKE '%keyword' bắt đầu bằng dấu phần trăm; hoặc khi vi phạm quy tắc tiền tố bên trái (Leftmost Prefix) của Composite Index.

6. **Câu 6:** "ACID trong cơ sở dữ liệu là gì?"

→ **Trả lời:** Atomicity (Tính nguyên tử - hoặc thành công hết hoặc hủy bỏ toàn bộ), Consistency (Tính nhất quán - tuân thủ mọi ràng buộc dữ liệu), Isolation (Tính cô lập - các giao dịch song song không can thiệp lẫn nhau), Durability (Tính bền vững - dữ liệu đã commit sẽ được lưu vĩnh viễn ngay cả khi mất điện).

7. **Câu 7:** "Thế nào là Star Schema và Snowflake Schema?"

→ **Trả lời:** Star Schema có các bảng Dimension kết nối trực tiếp với bảng Fact ở trung tâm (dạng phi chuẩn hóa, ít phép JOIN, truy vấn nhanh). Snowflake Schema tiếp tục chuẩn hóa các bảng Dimension thành nhiều bảng nhỏ hơn (tiết kiệm dung lượng nhưng tăng số lượng JOIN, truy vấn chậm hơn).

8. **Câu 8:** "Làm thế nào để tìm khách hàng có chi tiêu lớn thứ 2 trong mỗi phòng ban?"

→ **Trả lời:** Dùng CTE kết hợp DENSE_RANK() OVER(PARTITION BY department_id ORDER BY spending DESC) AS rnk, sau đó lọc WHERE rnk = 2.

9. **Câu 9:** "Cạm bẫy Fan-Out trong phép JOIN là gì?"

→ **Trả lời:** Khi nối một bảng 1 với bảng N theo quan hệ 1 – Nhiều, các dòng của bảng 1 bị nhân bản lên N lần. Nếu vội vàng tính SUM() trên bảng 1, kết quả doanh thu sẽ bị thổi phồng sai lệch. Cần tiền tổng hợp qua CTE trước khi JOIN.

10. **Câu 10:** "Làm thế nào để tối ưu một câu lệnh query chạy chậm?"

→ **Trả lời:** Chạy EXPLAIN ANALYZE để xác định nút thắt cổ chai (Seq Scan, Hash Join tràn đĩa); tạo Index phù hợp trên các cột trong JOIN và WHERE; tránh dùng SELECT *; hạn chế Correlated Subqueries; và xem xét phân vùng bảng (Partitioning) nếu bảng quá lớn.

15.4.2 10 Câu Hỏi Về Python, Pandas & Kỹ Thuật Lập Trình

1. **Câu 11:** "Generator trong Python là gì và tại sao nó quan trọng cho Kỹ sư Dữ liệu?"

→ **Trả lời:** Generator là hàm sử dụng từ khóa yield trả về từng phần tử theo cơ chế Lazy Evaluation. Nó cho phép xử lý các luồng dữ liệu khổng lồ hàng chục Gigabytes mà tiêu tốn bộ nhớ RAM gần như bằng 0, ngăn chặn lỗi MemoryError.

2. **Câu 12:** "Vectorization trong NumPy/Pandas là gì và tại sao nó nhanh hơn vòng lặp for?"

→ **Trả lời:** Vectorization thực thi tính toán trên toàn bộ mảng dữ liệu cùng lúc thông qua mã C cấp thấp được tối ưu hóa chỉ thị phần cứng SIMD (Single Instruction, Multiple Data), loại bỏ hoàn toàn chi phí lặp và kiểm tra kiểu động của Python interpreter.

3. **Câu 13:** "Phân biệt .loc và .iloc trong Pandas?"

→ **Trả lời:** .loc truy cập theo nhãn (Labels: tên cột, tên chỉ số index); .iloc truy cập theo vị trí số nguyên (Integer positions: từ 0 đến n - 1).

4. **Câu 14:** "Kể tên 3 cơ chế dữ liệu khuyết trong thống kê và cách xử lý?"

→ **Trả lời:** MCAR (Khuyết hoàn toàn ngẫu nhiên - có thể drop hoặc điền mean); MAR (Khuyết phụ thuộc biến quan sát khác - dùng conditional imputation); MNAR (Khuyết phụ thuộc vào chính giá trị của nó - không được điền mean bừa bãi mà phải tạo cột cờ flag báo khuyết).

5. **Câu 15:** "*Decorators trong Python hoạt động như thế nào?*"

→ **Trả lời:** Decorator là một hàm nhận vào một hàm khác làm đối số và mở rộng hành vi của hàm đó mà không sửa đổi trực tiếp mã nguồn của nó. Thường dùng cho ghi log (@log), đo thời gian (@timer), hoặc xác thực quyền truy cập.

6. **Câu 16:** "*Làm thế nào để xử lý một tệp CSV 50GB trên máy tính chỉ có 8GB RAM bằng Python?*"

→ **Trả lời:** Đọc dữ liệu theo từng khối (chunksize) qua `pd.read_csv(file, chunksize=100000)`; tối ưu hóa kiểu dữ liệu cột (category, int32 thay vì int64); hoặc sử dụng các công cụ hiện đại như **DuckDB** hoặc **Polars**.

7. **Câu 17:** "*Sự khác nhau giữa copy nông (Shallow Copy) và copy sâu (Deep Copy)?*"

→ **Trả lời:** Shallow copy tạo đối tượng mới nhưng chỉ sao chép tham chiếu của các đối tượng con bên trong; sửa đổi đối tượng con ở bản copy sẽ ảnh hưởng đến bản gốc. Deep copy tạo bản sao độc lập hoàn toàn cho cả đối tượng cha và toàn bộ các đối tượng lồng nhau bên trong.

8. **Câu 18:** "*Tính lũy đẳng (Idempotency) trong lập trình hàm dữ liệu là gì?*"

→ **Trả lời:** Là tính chất đảm bảo một hàm hay tiến trình khi thực thi nhiều lần với cùng một tham số đầu vào sẽ luôn trả về cùng một kết quả và không gây ra tác dụng phụ ngoài ý muốn (Side Effects).

9. **Câu 19:** "*Thư viện Pydantic giải quyết vấn đề gì trong Data Pipelines?*"

→ **Trả lời:** Pydantic thực thi việc kiểm tra kiểu dữ liệu tĩnh và động (Data Validation & Parsing) tại thời điểm chạy (Runtime), tự động ép kiểu và phát hiện các bản ghi sai định dạng trước khi chúng làm sập pipeline.

10. **Câu 20:** "*GIL (Global Interpreter Lock) trong Python là gì và ảnh hưởng thế nào đến tính toán dữ liệu?*"

→ **Trả lời:** GIL là khóa ngăn không cho nhiều luồng (threads) thực thi mã Python bytecode cùng lúc trên nhiều lõi CPU. Để xử lý dữ liệu nặng CPU-bound, Kỹ sư Dữ liệu phải sử dụng đa tiến trình (multiprocessing) hoặc các thư viện C cấp thấp như NumPy/PySpark.

15.4.3 10 Câu Hỏi Về Thống Kê, A/B Testing & Machine Learning

1. **Câu 21:** "*Giải thích ý nghĩa thực sự của giá trị p-value cho một người không chuyên?*"

→ **Trả lời:** p -value là thước đo mức độ "bất ngờ" của kết quả. Giả sử tính năng mới hoàn toàn không có tác dụng gì, thì p -value là xác suất ta vẫn tình cờ nhìn thấy sự chênh lệch lớn như vậy chỉ do may mắn ngẫu nhiên. Nếu $p < 0.05$, điều ngẫu nhiên đó quá khó xảy ra, ta kết luận tính năng mới thực sự có hiệu quả.

2. **Câu 22:** "*Định lý Giới Hạn Trung Tâm (CLT) phát biểu điều gì?*"

→ **Trả lời:** Với mẫu lấy ngẫu nhiên đủ lớn ($n \geq 30$), phân phối của giá trị trung bình mẫu sẽ xấp xỉ phân phối chuẩn, bất kể phân phối của quần thể gốc ban đầu có hình dạng như thế nào.

3. **Câu 23:** "*Hiện tượng Sample Ratio Mismatch (SRM) trong A/B Testing là gì?*"

→ **Trả lời:** Là khi tỷ lệ phân bổ người dùng thực tế giữa nhóm Control và Treatment bị lệch có ý nghĩa thống kê so với tỷ lệ cấu hình ban đầu (ví dụ cấu hình 50:50 nhưng nhận về 55:45). Báo hiệu lỗi kỹ thuật nghiêm trọng làm vô hiệu hóa toàn bộ kết quả thử nghiệm.

4. **Câu 24:** "Tại sao Accuracy không phải là chỉ số tốt cho bài toán phát hiện gian lận (Fraud Detection)?"
 → **Trả lời:** Do mất cân bằng dữ liệu cực độ (tỷ lệ gian lận chỉ 0.1%). Một mô hình luôn đoán là "Bình thường" sẽ đạt Accuracy 99.9% nhưng bỏ sót 100% kẻ gian lận. Cần dùng Precision, Recall, PR-AUC hoặc F1-Score.
5. **Câu 25:** "Data Leakage là gì và cách phòng tránh tuyệt đối?"
 → **Trả lời:** Là hiện tượng thông tin từ tập kiểm thử hoặc thông tin trong tương lai bị rò rỉ vào quá trình huấn luyện mô hình. Phòng tránh bằng cách luôn chia train/test TRƯỚC KHI tiền xử lý và đóng gói toàn bộ qua Scikit-Learn Pipeline.
6. **Câu 26:** "Độ đánh đổi giữa Bias và Variance (Bias-Variance Tradeoff) là gì?"
 → **Trả lời:** High Bias dẫn đến Underfitting (mô hình quá đơn giản, không học được quy luật). High Variance dẫn đến Overfitting (mô hình quá phức tạp, học vẹt cả nhiễu của tập train nhưng thất bại trên tập test). Mục tiêu là tìm điểm cân bằng cực tiểu hóa tổng sai số.
7. **Câu 27:** "Khác biệt cơ bản giữa Random Forest và Gradient Boosting (XGBoost)?"
 → **Trả lời:** Random Forest dùng kỹ thuật Bagging (huấn luyện song song nhiều cây độc lập sâu rồi lấy trung bình để giảm Variance). Gradient Boosting dùng kỹ thuật Boosting (huấn luyện tuần tự các cây nông, mỗi cây sau cố gắng sửa sai số của các cây trước đó để giảm Bias).
8. **Câu 28:** "Giá trị SHAP trong giải thích mô hình Machine Learning đại diện cho điều gì?"
 → **Trả lời:** Dựa trên lý thuyết trò chơi, SHAP value đo lường mức độ đóng góp cận biên của từng đặc trưng đầu vào vào việc làm tăng hay giảm xác suất dự báo so với giá trị kỳ vọng cơ sở của toàn bộ tập dữ liệu.
9. **Câu 29:** "Tại sao ta không nên kiểm tra p-value mỗi ngày trong suốt thời gian chạy A/B Test (Peeking Problem)?"
 → **Trả lời:** Việc kiểm tra liên tục và dừng thử nghiệm khi thấy $p < 0.05$ làm sai lầm loại I (False Positive Rate) tích lũy tăng vọt từ 5% lên tới 30–50% do hiện tượng kiểm định nhiều lần ngẫu nhiên.
10. **Câu 30:** "Phân biệt Data Drift và Concept Drift trong sản xuất?"
 → **Trả lời:** Data Drift là khi phân phối dữ liệu đầu vào thay đổi $P(X)$ nhưng mối quan hệ không đổi. Concept Drift là khi mối quan hệ giữa đầu vào và nhãn thay đổi $P(y|X)$ (hành vi bản chất của con người đã thay đổi).

15.4.4 10 Câu Hỏi Về Data Engineering, System Design & Hạ Tầng

1. **Câu 31:** "Tại sao Parquet lại nhanh hơn CSV khi thực hiện các câu truy vấn phân tích?"
 → **Trả lời:** Parquet lưu trữ dữ liệu theo dạng cột (Columnar format) kết hợp nén Snappy. Khi query chỉ cần lấy 3 cột trên bảng 100 cột, hệ thống chỉ đọc các khối dữ liệu của 3 cột đó từ ổ đĩa (I/O Pruning), giảm tới 95% lưu lượng đọc đĩa.
2. **Câu 32:** "Broadcast Hash Join trong PySpark hoạt động như thế nào?"
 → **Trả lời:** Khi nối bảng lớn với bảng nhỏ, Spark sao chép toàn bộ bảng nhỏ gửi tới bộ nhớ RAM của tất cả các Executor, cho phép thực hiện phép join nội bộ tại từng máy mà không cần shuffle dữ liệu lớn qua mạng.

3. **Câu 33:** *"dbt giải quyết những điểm yếu nào của các script SQL truyền thống?"*
→ **Trả lời:** dbt mang các chuẩn mực kỹ thuật phần mềm vào SQL: quản lý phiên bản với Git, quản lý phụ thuộc tự động qua lệnh `ref()`, tự động hóa kiểm thử dữ liệu, và tự động sinh biểu đồ Data Lineage.
4. **Câu 34:** *"Tại sao cần đặt `catchup=False` trong Apache Airflow?"*
→ **Trả lời:** Nếu `catchup=True`, khi bật một DAG có `start_date` trong quá khứ, Airflow Scheduler sẽ lập tức kích hoạt tất cả các mẻ chạy trong quá khứ chưa được thực thi, gây quá tải và làm sập toàn bộ hệ thống.
5. **Câu 35:** *"Cơ chế Time Travel trong Delta Lake hoạt động như thế nào?"*
→ **Trả lời:** Delta Lake duy trì một nhật ký giao dịch tuần tự `_delta_log`. Dữ liệu cũ không bị xóa ngay mà được bảo toàn; Time Travel cho phép truy vấn lại trạng thái dữ liệu ở phiên bản commit hoặc mốc thời gian chỉ định trong quá khứ.
6. **Câu 36:** *"Phân biệt SCD Type 1 và SCD Type 2 trong mô hình hóa dữ liệu?"*
→ **Trả lời:** SCD Type 1 ghi đè trực tiếp dữ liệu mới lên dữ liệu cũ (mất lịch sử). SCD Type 2 tạo ra một dòng mới với khóa thay thế mới, duy trì lịch sử bằng các cột ngày bắt đầu, ngày kết thúc và cờ hiệu `is_current`.
7. **Câu 37:** *"Partitioning và Clustering trong BigQuery/Athena giúp tiết kiệm chi phí thế nào?"*
→ **Trả lời:** BigQuery tính phí theo số byte dữ liệu được quét. Bằng cách phân vùng theo ngày và gom cụm theo ID, truy vấn có điều kiện lọc sẽ chỉ quét các khối dữ liệu cần thiết, giảm lượng byte quét và chi phí tới 90–99%.
8. **Câu 38:** *"Thiết kế một pipeline xử lý dữ liệu đơn hàng chịu lỗi (Fault-tolerant) như thế nào?"*
→ **Trả lời:** Thiết kế pipeline có tính lũy đẳng (Idempotent load); bổ sung cơ chế tự động thử lại (Retries with exponential backoff); đẩy các bản ghi lỗi vào Dead Letter Queue / Quarantine để kiểm tra sau; và cấu hình hệ thống cảnh báo tức thời qua Slack/PagerDuty.
9. **Câu 39:** *"Sự khác nhau giữa Batch Processing và Stream Processing?"*
→ **Trả lời:** Batch processing xử lý các khối dữ liệu tĩnh tích lũy theo chu kỳ (mỗi giờ, mỗi ngày) với độ trễ từ vài phút đến vài giờ (Airflow, Spark Batch). Stream processing xử lý từng sự kiện ngay khi nó vừa phát sinh với độ trễ tính bằng mili-giây (Kafka, Flink, Spark Streaming).
10. **Câu 40:** *"Một kỹ sư dữ liệu cấp Middle khác Fresher ở điểm nào khi thiết kế hệ thống?"*
→ **Trả lời:** Fresher chỉ tập trung vào việc làm cho code chạy được; Middle tập trung vào tính bền vững: khả năng mở rộng (Scalability), giám sát chất lượng dữ liệu (Data Observability), tối ưu chi phí hạ tầng (FinOps), và khả năng tự động khôi phục sau sự cố (Disaster Recovery).

Workbook Phần 1: 20 Bài Tập Thực Hành Excel & Báo Cáo Nghiệp Vụ

Kiến Thức Cốt Lõi: Mục Tiêu Luyện Tập Phần 1

Phần này cung cấp **20 bài tập thực hành Excel chuyên sâu** từ cơ bản đến nâng cao. Mỗi bài tập đều mô phỏng tình huống thực tế tại các phòng ban (Tài chính, Kinh doanh, Marketing, Vận hành), đi kèm công thức chuẩn và giải thích ngắn gọn để bạn tự kiểm tra.

Bài Tập Tự Luyện: Hệ Thống 20 Bài Tập Excel Thực Chiến

- Bài 1 (XLOOKUP Cơ Bản):** Cho bảng Danh mục nhân viên A1:D100 gồm các cột [Mã NV, Họ Tên, Phòng Ban, Lương]. Tại ô F2, người dùng nhập Mã NV "NV105". Hãy viết công thức XLOOKUP tại ô G2 để tra cứu Họ Tên và tại H2 để tra cứu Lương, trả về chuỗi "Không tìm thấy" nếu mã không tồn tại.
- Bài 2 (XLOOKUP Hai Chiều):** Bảng ma trận giá cước vận chuyển A1:E10 có dòng là Tỉnh/Thành đến và cột là Khối lượng hàng (1kg, 2kg, 5kg, 10kg). Viết công thức XLOOKUP lồng nhau (2-Way XLOOKUP) để tra cứu chính xác cước phí dựa trên 2 điều kiện: Tỉnh đến tại G2 và Khối lượng tại H2.
- Bài 3 (INDEX & MATCH Kinh Điển):** Tại sao trong các phiên bản Excel cũ, cặp hàm INDEX/MATCH lại vượt trội hơn VLOOKUP? Viết công thức tra cứu Tên Khách Hàng ở cột A dựa trên Số Điện Thoại ở cột C.
- Bài 4 (SUMIFS Đa Điều Kiện):** Bảng doanh số A1:E5000 gồm [Ngày, Nhân Viên, Khu Vực, Ngành Hàng, Doanh Thu]. Hãy viết công thức tính tổng doanh thu thỏa mãn 3 điều kiện: Khu vực là "Miền Bắc", Ngành hàng là "Điện Tử", và phát sinh trong Quý 1 năm 2026 (từ 01/01/2026 đến 31/03/2026).
- Bài 5 (COUNTIFS Đếm Đơn Hàng):** Đếm số lượng đơn hàng có giá trị trên \$500 của nhân viên "Nguyễn Văn A" được giao thành công trong tháng 02/2026.

6. **Bài 6 (Dynamic Array – UNIQUE):** Cho cột B2:B10000 chứa danh sách khách hàng có nhiều tên trùng lặp. Viết công thức mảng động xuất ra danh sách khách hàng duy nhất và tự động sắp xếp theo thứ tự bảng chữ cái A–Z.
7. **Bài 7 (Dynamic Array – FILTER):** Viết công thức mảng động trích xuất toàn bộ các dòng từ bảng A2:E1000 thỏa mãn điều kiện: Trạng thái đơn hàng là "COMPLETED" và Giá trị đơn hàng > \$1,000.
8. **Bài 8 (Tính Hoa Hồng Đa Bậc):** Chính sách hoa hồng: Doanh số $\leq$ \$10,000: hoa hồng 3%; Từ \$10,001 đến \$30,000: hoa hồng 5%; Trên \$30,000: hoa hồng 8%. Viết công thức IFS hoặc XLOOKUP xấp xỉ (match_mode = -1) để tính tiền hoa hồng cho doanh số tại ô C2.
9. **Bài 9 (Tách Chuỗi Họ Tên):** Cột A chứa "Họ và Tên" đầy đủ (ví dụ: "Nguyễn Văn An"). Sử dụng các hàm xử lý chuỗi (TEXTBEFORE, TEXTAFTER hoặc LEFT, RIGHT, FIND) để tách riêng Tên gọi và Họ đệm sang 2 cột riêng biệt.
10. **Bài 10 (Chuẩn Hóa Ngày Tháng):** Cột A chứa ngày tháng ở định dạng text hỗn loạn: "20260315", "15/03/2026", "2026.03.15". Trình bày quy trình dùng Power Query để chuẩn hóa toàn bộ về định dạng chuẩn Date (DD/MM/YYYY).
11. **Bài 11 (Tính Tăng Trưởng MoM):** Cho bảng doanh thu 12 tháng tại cột B (B2:B13). Viết công thức tính tỷ lệ tăng trưởng phần trăm so với tháng trước (MoM Growth %) tại ô C3 và bắt lỗi chia cho 0 nếu tháng trước doanh thu bằng 0 bằng hàm IFERROR.
12. **Bài 12 (Tự Động Đánh Số Thứ Tự):** Viết công thức đánh số thứ tự 1, 2, 3... tại cột A sao cho số thứ tự chỉ hiện khi cột B có nhập tên, và tự động bỏ trống nếu cột B để trống.
13. **Bài 13 (Unpivot Trong Power Query):** Một bảng dữ liệu theo dõi doanh thu có các cột: [Mã SP, Tháng 1, Tháng 2, Tháng 3... Tháng 12]. Giải thích vì sao cấu trúc này vi phạm nguyên tắc Tidy Data và nêu các bước Unpivot Columns trong Power Query để đưa về 3 cột: [Mã SP, Tháng, Doanh Thu].
14. **Bài 14 (Pivot Table – Sắp Xếp Top N):** Trình bày các bước thiết lập Pivot Table để tự động lọc ra Top 5 nhân viên có doanh số cao nhất mà không làm thủ công.
15. **Bài 15 (Pivot Table – Calculated Field):** Tạo một trường tính toán mới (Calculated Field) trong Pivot Table mang tên "Lợi Nhuận Thuần" với công thức: Doanh Thu * 0.25 - Chi Phí Vận Chuyển.
16. **Bài 16 (Xác Thực Dữ Liệu – Data Validation):** Thiết lập ô D2 chỉ cho phép người dùng nhập ngày trong tương lai (sau ngày hiện tại TODAY()) và hiển thị thông báo lỗi tùy chỉnh khi nhập sai.
17. **Bài 17 (Conditional Formatting Theo Điều Kiện):** Thiết lập định dạng có điều kiện tự động tô màu nền đỏ nhạt cho toàn bộ dòng dữ liệu nếu cột Trạng Thái là "OVERDUE" (Quá hạn thanh toán).

18. **Bài 18 (XLOOKUP Tìm Kiếm Ngược Từ Dưới Lên):** Trong bảng nhật ký xuất kho, một sản phẩm xuất kho nhiều lần. Viết công thức XLOOKUP tìm giá xuất kho của **lần xuất gần đây nhất** (lần xuất cuối cùng trong bảng) bằng tham số `search_mode = -1`.
19. **Bài 19 (Tính Số Ngày Làm Việc Thực Tế):** Viết công thức tính số ngày làm việc thực tế giữa ngày bắt đầu dự án tại A2 và ngày kết thúc tại B2, tự động trừ đi Thứ Bảy, Chủ Nhật và danh sách ngày nghỉ lễ tại dải ô `Holiday_List` bằng hàm `NETWORKDAYS.INTL`.
20. **Bài 20 (Ghép Nối Dữ Liệu TEXTJOIN):** Cho cột danh sách email khách hàng từ C2:C50. Viết công thức gom tất cả các email này thành một chuỗi duy nhất phân tách bằng dấu chấm phẩy (;) để dán vào hòm thư Outlook, tự động bỏ qua các ô trống.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Lời Giải Ngắn Gọn Cho 20 Bài Tập Excel

- **Bài 1:** `G2 = XLOOKUP(F2, A2:A100, B2:B100, "Không tìm thấy"); H2 = XLOOKUP(F2, A2:A100, D2:D100, "Không tìm thấy")`.
- **Bài 2:** `=XLOOKUP(G2, A2:A10, XLOOKUP(H2, B1:E1, B2:E10))`.
- **Bài 3:** VLOOKUP chỉ tìm từ trái sang phải. INDEX/MATCH tìm kiếm linh hoạt hai chiều: `=INDEX(A2:A100, MATCH(Phone_Val, C2:C100, 0))`.
- **Bài 4:** `=SUMIFS(E2:E5000, C2:C5000, "Miền Bắc", D2:D5000, "Điện Tử", A2:A5000, ">=2026-01-01", A2:A5000, "<=2026-03-31")`.
- **Bài 5:** `=COUNTIFS(B2:B5000, "Nguyễn Văn A", E2:E5000, ">500", A2:A5000, ">=2026-02-01", A2:A5000, "<=2026-02-28", D2:D5000, "COMPLETED")`.
- **Bài 6:** `=SORT(UNIQUE(B2:B10000))`.
- **Bài 7:** `=FILTER(A2:E1000, (D2:D1000="COMPLETED") * (E2:E1000>1000), "Không có đơn hàng")`.
- **Bài 8:** `=IFS(C2<=10000, C2*0.03, C2<=30000, C2*0.05, C2>30000, C2*0.08)`.
- **Bài 9:** Tên: `=TEXTAFTER(A2, " ", -1)`. Họ đệm: `=TEXTBEFORE(A2, " ", -1)`.
- **Bài 10:** Trong Power Query: Chọn cột → *Using Locale* → chọn kiểu Date với định dạng nguồn tương ứng → Power Query tự parse sang định dạng chuẩn của máy tính.
- **Bài 11:** `=IFERROR((B3 - B2) / B2, 0)`.
- **Bài 12:** `=IF(B2<>"", ROW()-1, "")` (hoặc dùng hàm `=IF(ISBLANK(B2), "", COUNTA($B$2:B2))`).

- **Bài 13:** Cấu trúc ngang (cột là các tháng) khiến công thức SUMIFS và Pivot Table không thể lọc theo tháng. Unpivot biến các cột tháng thành 1 cột định danh duy nhất (Dữ liệu dạng Tidy/Đọc).
- **Bài 14:** Nhấp chuột phải vào tên Nhân viên trong Pivot → *Filter* → *Top 10...* → đổi số 10 thành 5 → chọn trường sắp xếp theo Doanh Thu.
- **Bài 15:** Chọn PivotTable → tab *PivotTable Analyze* → *Fields, Items, & Sets* → *Calculated Field* → nhập công thức.
- **Bài 16:** Tab *Data* → *Data Validation* → Allow: *Date* → Data: *greater than* → Start date: =TODAY().
- **Bài 17:** Bôi đen toàn bộ bảng \$A\$2:\$E\$100 → *Conditional Formatting* → *New Rule* → *Use a formula...* → nhập công thức: =\$D2="OVERDUE" (khóa cột D).
- **Bài 18:** =XLOOKUP(Product_ID, A2:A1000, C2:C1000, "N/A", 0, -1).
- **Bài 19:** =NETWORKDAYS.INTL(A2, B2, 1, Holiday_List) (số 1 quy định T7 và CN là ngày nghỉ).
- **Bài 20:** =TEXTJOIN("; ", TRUE, C2:C50).

Workbook Phần 2: 20 Bài Tập Thực Hành Lập Trình Python & Pandas

Kiến Thức Cốt Lõi: Mục Tiêu Luyện Tập Phần 2

Phần này cung cấp **20 bài tập code Python & Pandas thực chiến**, bao gồm cả lập trình giải thuật xử lý dữ liệu và thao tác phân tích nâng cao, giúp bạn làm chủ 100% các kỹ năng code được yêu cầu trong CV.

Bài Tập Tự Luyện: Hệ Thống 20 Bài Tập Python & Pandas Thực Chiến

1. **Bài 1 (Khử Trùng Lặp List):** Viết hàm Python nhận vào một danh sách số nguyên có thứ tự bất kỳ, trả về danh sách các phần tử duy nhất được sắp xếp tăng dần mà không dùng hàm `set()`.
2. **Bài 2 (Đếm Tần Suất Ký Tự):** Viết hàm nhận vào một chuỗi văn bản, trả về một dictionary đếm số lần xuất hiện của từng từ (Word Frequency Counter), bỏ qua dấu câu và không phân biệt chữ hoa/thường.
3. **Bài 3 (Tìm Cặp Số Có Tổng Bằng K):** Cho một mảng số nguyên và một số K . Viết thuật toán tìm 2 số trong mảng có tổng đúng bằng K với độ phức tạp thời gian tối ưu $O(n)$ sử dụng Hash Map (Dictionary).
4. **Bài 4 (Flatten Nested List):** Viết hàm đệ quy làm phẳng một danh sách lồng nhau nhiều cấp (ví dụ: `[1, [2, [3, 4], 5], 6]` thành `[1, 2, 3, 4, 5, 6]`).
5. **Bài 5 (Xử Lý Ngoại Lệ An Toàn):** Viết hàm `safe_divide(a, b)` chuyển đổi cả hai tham số sang float và thực hiện phép chia. Bắt và xử lý riêng biệt các lỗi `ValueError`, `ZeroDivisionError`, và `TypeError`.
6. **Bài 6 (Decorator Tính Thời Gian):** Viết một decorator `@benchmark` đo thời gian chạy của hàm và in ra thông báo nếu hàm chạy quá 1.0 giây.

7. **Bài 7 (Generator Sinh Chuỗi Fibonacci):** Viết một Generator sử dụng `yield` sinh vô hạn dãy số Fibonacci (0, 1, 1, 2, 3, 5, 8...) với mức tiêu thụ RAM luôn cố định $O(1)$.
8. **Bài 8 (NumPy Matrix Operations):** Cho ma trận A kích thước (1000, 50) và vector B kích thước (50,). Hãy dùng NumPy tính tích vô hướng $A \times B$ và chuẩn hóa vector kết quả về khoảng [0, 1] bằng Min-Max Scaling.
9. **Bài 9 (Pandas Memory Optimization):** Viết hàm nhận vào một DataFrame, tự động ép kiểu các cột số nguyên từ `int64` xuống `int32` hoặc `int16`, và các cột chuỗi có ít hơn 50 giá trị duy nhất sang `category` để giảm ít nhất 50% dung lượng RAM.
10. **Bài 10 (Pandas Missing Data Imputation):** Cho cột `income` có 5% giá trị null. Hãy viết code điền giá trị khuyết bằng giá trị trung vị của nhóm ngành nghề tương ứng (`groupby('occupation')`).
11. **Bài 11 (Pandas Regex Extraction):** Cột `product_code` chứa các chuỗi như "VN-TECH-2026-X01". Dùng biểu thức chính quy Regex tách thành 4 cột riêng biệt: `Country`, `Department`, `Year`, `ItemID`.
12. **Bài 12 (Pandas Time Series Resampling):** Cho bảng dữ liệu giao dịch phát sinh từng giây. Hãy viết code gộp nhóm (Resample) theo từng khung giờ (Hourly), tính tổng doanh thu và số lượng đơn hàng của mỗi giờ.
13. **Bài 13 (Rolling Window Metrics):** Sử dụng hàm `rolling()` của Pandas tính độ lệch chuẩn động 14 ngày (14-day Rolling Volatility) của giá cổ phiếu.
14. **Bài 14 (Tách Cột List Thành Nhiều Dòng):** Bảng dữ liệu có cột `tags` chứa danh sách các thể loại sản phẩm dạng list (ví dụ: ['Tech', 'Sale', 'New']). Dùng hàm `explode()` để biến mỗi phần tử trong list thành một dòng độc lập.
15. **Bài 15 (Pivot & Melt):** Trình bày sự khác biệt giữa hai thao tác `pivot()` và `melt()` trong Pandas. Viết code chuyển đổi bảng từ Wide format sang Long format.
16. **Bài 16 (Phát Hiện Outlier Bằng IQR):** Viết hàm Python nhận vào DataFrame và tên cột số, trả về DataFrame con chỉ chứa các dòng có giá trị ngoại lai nằm ngoài khoảng $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$.
17. **Bài 17 (Custom Aggregation Function):** Sử dụng hàm `agg()` với một hàm tùy chỉnh để tính chỉ số chênh lệch giữa giá trị lớn nhất và nhỏ nhất ($\text{Range} = \text{Max} - \text{Min}$) cho từng nhóm khách hàng.
18. **Bài 18 (Xử Lý Trùng Lặp Có Điều Kiện):** Bảng lịch sử đăng nhập có các cột [`user_id`, `login_timestamp`, `device`]. Hãy giữ lại duy nhất lần đăng nhập MỖI NHẤT của từng người dùng bằng hàm `sort_values()` và `drop_duplicates()`.
19. **Bài 19 (Pandas Merge Indicators):** Khi thực hiện phép nối `merge(df1, df2, how='outer')`, làm thế nào để biết một dòng dữ liệu đến từ bảng bên trái, bảng bên phải hay xuất hiện ở cả hai bảng bằng tham số `indicator=True`?

20. **Bài 20 (Batch Processing File Lớn):** Viết script Python sử dụng tham số `chunksize=50000` trong `pd.read_csv` để tính tổng doanh thu của một tệp CSV dung lượng 10GB mà không làm tràn bộ nhớ máy tính.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Lời Giải Ngắn Gọn Cho 20 Bài Tập Python

• **Bài 1:**

```
1 def unique_sorted(lst):
2     res = []
3     for x in sorted(lst):
4         if not res or x != res[-1]: res.append(x)
5     return res
6
```

- **Bài 2:** Dùng `re.findall(r'\w+', text.lower())` kết hợp với `collections.Counter` hoặc duyệt dict đếm số lần xuất hiện.
- **Bài 3:** Duyệt mảng, với mỗi số x , kiểm tra nếu $K - x$ đã có trong `seen_dict`. Nếu có, trả về $(x, K - x)$; nếu chưa, lưu x vào `seen_dict`. Độ phức tạp $O(n)$.

• **Bài 4:**

```
1 def flatten(lst):
2     res = []
3     for item in lst:
4         res.extend(flatten(item)) if isinstance(item, list) else
5         res.append(item)
6     return res
```

- **Bài 5:** Dùng khối `try: ... except (ValueError, TypeError): ... except ZeroDivisionError: ....`
- **Bài 6:** Decorator sử dụng `time.perf_counter()`, tính `elapsed = end - start`, nếu `elapsed > 1.0` thì in cảnh báo `print("[!] Canh bao qua tai")`.
- **Bài 7:**

```
1 def fib_gen():
2     a, b = 0, 1
3     while True:
4         yield a
5         a, b = b, a + b
6
```

- **Bài 8:** `C = np.dot(A, B); C_norm = (C - C.min()) / (C.max() - C.min())`.

- **Bài 9:** Duyệt qua các cột số nguyên: `pd.to_numeric(df[col], downcast='integer');` với cột object: `if df[col].nunique() < 50: df[col] = df[col].astype('category');`
- **Bài 10:** `df['income'] = df.groupby('occupation')['income'].transform(lambda x: x.fillna(x.median()))`
- **Bài 11:** `df[['Country', 'Dept', 'Year', 'ID']] = df['product_code'].str.extract(r'^(\w+)-(\w+)-(\d+)-(\w+)$')`
- **Bài 12:** `df.set_index('timestamp').resample('1H').agg({'revenue': 'sum', 'order_id': 'count'})`
- **Bài 13:** `df['volatility_14d'] = df['close_price'].rolling(window=14).std()`
- **Bài 14:** `df_exploded = df.explode('tags')`
- **Bài 15:** `pivot()` xoay từ dòng thành cột (dài sang rộng). `melt()` xoay từ cột thành dòng (rộng sang dài): `pd.melt(df, id_vars=['id'], value_vars=['month1', 'month2'])`
- **Bài 16:** Tính `q1 = df[col].quantile(0.25)`, `q3 = df[col].quantile(0.75)`, `iqr = q3 - q1`. Lọc: `df[(df[col] < q1 - 1.5*iqr) | (df[col] > q3 + 1.5*iqr)]`
- **Bài 17:** `df.groupby('segment')['spending'].agg(lambda x: x.max() - x.min())`
- **Bài 18:** `df.sort_values(by=['user_id', 'login_timestamp'], ascending=[True, False]).drop_duplicates(subset=['user_id'], keep='first')`
- **Bài 19:** Khi đặt `indicator=True`, Pandas sinh cột `_merge` với 3 giá trị: 'left_only', 'right_only', và 'both'.
- **Bài 20:**

```

1 total_rev = 0.0
2 for chunk in pd.read_csv("large_sales.csv", chunksize=50000):
3     total_rev += chunk['amount'].sum()
4 print("Tong doanh thu:", total_rev)
5

```

Workbook Phần 3: 20 Bài Tập Thực Hành SQL & Cơ Sở Dữ Liệu Nâng Cao

Kiến Thức Cốt Lõi: Mục Tiêu Luyện Tập Phần 3

Phần này cung cấp **20 bài tập SQL thực chiến** từ các câu hỏi phỏng vấn kỹ thuật thực tế tại các công ty công nghệ lớn, kiểm tra từ tư duy thiết kế schema đến tối ưu hóa truy vấn phức tạp.

Bài Tập Tự Luyện: Hệ Thống 20 Bài Tập SQL Thực Chiến

- Bài 1 (DDL & Constraints):** Viết câu lệnh tạo bảng `dim_employees` có các ràng buộc: `emp_id` là khóa chính, `email` duy nhất, `salary` phải > 0 , và `hire_date` mặc định là ngày hiện tại.
- Bài 2 (Conditional Update):** Viết câu lệnh cập nhật tăng lương 10% cho tất cả nhân viên thuộc phòng ban 'Engineering' có thâm niên làm việc trên 2 năm.
- Bài 3 (Tìm Khách Hàng Không Mua Hàng):** Cho bảng `customers` và bảng `orders`. Viết truy vấn tìm tất cả khách hàng chưa từng thực hiện bất kỳ đơn hàng nào bằng 2 cách: (a) Dùng `LEFT JOIN`; (b) Dùng `NOT EXISTS`.
- Bài 4 (Aggregation với HAVING):** Tìm tất cả người bán (`seller_id`) có tổng doanh thu vượt quá \$100,000 và đã bán được trên 500 đơn hàng trong năm 2025.
- Bài 5 (Xử Lý Trùng Lặp Fan-Out Trap):** Cho bảng `orders` và bảng `payments`. Một đơn hàng có thể thanh toán qua nhiều đợt. Viết truy vấn tính tổng doanh thu chính xác của từng khách hàng mà không bị nhân đôi số tiền do phép `JOIN`.
- Bài 6 (Self Join – Cặp Đôi Cùng Thành Phố):** Viết truy vấn tìm tất cả các cặp khách hàng khác nhau nhưng sống cùng một thành phố, đảm bảo không bị trùng lặp hoán vị (tức là nếu có cặp (A, B) thì không xuất hiện cặp (B, A)).
- Bài 7 (Window Function – ROW_NUMBER vs DENSE_RANK):** Viết câu lệnh tìm

- 3 sản phẩm đắt nhất của TỪNG danh mục (category). Giải thích vì sao nên dùng DENSE_RANK() thay vì ROW_NUMBER() nếu có các sản phẩm bằng giá nhau.
8. **Bài 8 (Tính Doanh Thu Tích Lũy – Running Total):** Viết câu lệnh tính doanh thu tích lũy cộng dồn từ đầu tháng đến ngày hiện tại (Month-To-Date Running Total) theo từng ngày bán hàng.
 9. **Bài 9 (Window Function – LEAD & Chênh Lệch Thời Gian):** Cho bảng lịch sử chuyển trạng thái đơn hàng. Viết truy vấn sử dụng hàm LEAD() để tính thời gian (số giờ) một đơn hàng nằm ở trạng thái 'PROCESSING' trước khi chuyển sang trạng thái tiếp theo.
 10. **Bài 10 (Window Function – NTILE Phân Hạng):** Chia toàn bộ người dùng thành 5 nhóm bằng nhau (Quintiles: 1 đến 5) dựa trên số lượt xem trang web trong tháng qua.
 11. **Bài 11 (Tăng Trưởng Doanh Thu MoM):** Viết truy vấn sử dụng CTE và hàm LAG() để tính tỷ lệ phần trăm tăng trưởng doanh thu so với tháng trước (% MoM Growth) cho từng tháng trong năm 2025.
 12. **Bài 12 (Tìm Chuỗi Hoạt Động Liên Tục – Consecutive Days):** Cho bảng nhật ký đăng nhập logins(user_id, login_date). Tìm tất cả người dùng đã đăng nhập hệ thống liên tục từ 5 ngày trở lên.
 13. **Bài 13 (Recursive CTE – Cây Danh Mục Sản Phẩm):** Cho bảng danh mục categories(cat_id, cat_name, parent_id). Viết một Recursive CTE để hiển thị toàn bộ cây phả hệ danh mục kèm cấp bậc độ sâu (Level: 1, 2, 3...).
 14. **Bài 14 (Pivot Dữ Liệu Bằng CASE WHEN):** Chuyển đổi dữ liệu doanh số từ dạng dọc sang dạng ngang: Viết truy vấn hiển thị doanh thu của từng nhân viên phân tách thành 4 cột riêng biệt: Q1_Sales, Q2_Sales, Q3_Sales, Q4_Sales.
 15. **Bài 15 (Truy Vấn Tương Quan – Correlated Subquery):** Viết truy vấn tìm các nhân viên có mức lương cao hơn mức lương trung bình của chính phòng ban mà người đó đang làm việc.
 16. **Bài 16 (Xóa Bản Ghi Trùng Lặp Bằng CTE):** Một bảng bị lỗi hệ thống khiến một số đơn hàng bị nhân bản 2–3 dòng hoàn toàn giống nhau. Viết lệnh DELETE kết hợp CTE và ROW_NUMBER() để chỉ giữ lại đúng 1 bản ghi duy nhất cho mỗi đơn hàng.
 17. **Bài 17 (Phân Tích Cohort Retention):** Viết truy vấn tính toán tỷ lệ giữ chân khách hàng (Retention Rate %) theo từng tháng kể từ tháng mua hàng đầu tiên (Tháng 0, Tháng 1, Tháng 2...).
 18. **Bài 18 (Tối Ưu Hóa Query Chậm Bằng EXPLAIN):** Phân tích lý do câu lệnh sau bị Seq Scan: `SELECT * FROM orders WHERE order_date + INTERVAL '7 days' > NOW( );`. Viết lại câu lệnh tối ưu (Sargable Query) để tận dụng B-Tree Index trên cột order_date.
 19. **Bài 19 (Chỉ Mục Bao Phủ – Covering Index):** Giải thích khái niệm Covering Index với mệnh đề INCLUDE trong PostgreSQL. Viết lệnh DDL tạo Index giúp câu lệnh sau đạt

trạng thái **Index Only Scan**: `SELECT order_id, total_amount FROM orders WHERE customer_id = 'C01';`

20. **Bài 20 (Upsert với ON CONFLICT)**: Viết câu lệnh nạp dữ liệu vào bảng `dim_user_profiles`: Nếu `user_id` chưa tồn tại thì chèn mới; nếu đã tồn tại thì cập nhật `last_login = NOW()` và `login_count = login_count + 1`.

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Lời Giải Ngắn Gọn Cho 20 Bài Tập SQL

• **Bài 1:**

```
1 CREATE TABLE dim_employees (
2     emp_id VARCHAR(50) PRIMARY KEY,
3     email VARCHAR(100) UNIQUE NOT NULL,
4     salary NUMERIC(12, 2) CHECK (salary > 0),
5     hire_date DATE DEFAULT CURRENT_DATE
6 );
7
```

- **Bài 2:** `UPDATE dim_employees SET salary = salary * 1.10 WHERE dept = 'Engineering' AND hire_date <= CURRENT_DATE - INTERVAL '2 years';`
- **Bài 3:** (a) `SELECT c.* FROM customers c LEFT JOIN orders o ON c.id = o.cust_id WHERE o.id IS NULL;`
(b) `SELECT * FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.id);`
- **Bài 4:** `SELECT seller_id, SUM(price), COUNT(order_id) FROM order_items WHERE EXTRACT(YEAR FROM order_date) = 2025 GROUP BY seller_id HAVING SUM(price) > 100000 AND COUNT(order_id) > 500;`
- **Bài 5:** Dùng CTE tiền tổng hợp thành toán: `WITH pay AS (SELECT order_id, SUM(amount) AS tot FROM payments GROUP BY order_id) SELECT o.customer_id, SUM(p.tot) FROM orders o JOIN pay p ON o.order_id = p.order_id GROUP BY o.customer_id;`
- **Bài 6:** `SELECT a.name, b.name, a.city FROM customers a JOIN customers b ON a.city = b.city AND a.id < b.id;`
- **Bài 7:** `DENSE_RANK` giữ nguyên thứ tự không nhảy cóc khi có đồng giá: `WITH r AS (SELECT *, DENSE_RANK() OVER(PARTITION BY category ORDER BY price DESC) AS rnk FROM products) SELECT * FROM r WHERE rnk <= 3;`

- **Bài 8:** `SUM(amount) OVER(PARTITION BY DATE_TRUNC('month', order_date) ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).`
- **Bài 9:** `EXTRACT(EPOCH FROM (LEAD(status_time) OVER(PARTITION BY order_id ORDER BY status_time) - status_time)) / 3600.0.`
- **Bài 10:** `NTILE(5) OVER(ORDER BY pageviews DESC).`
- **Bài 11:** `ROUND((rev - LAG(rev) OVER(ORDER BY month)) / LAG(rev) OVER(ORDER BY month) * 100.0, 2).`
- **Bài 12:** Dùng kỹ thuật `login_date - (ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY login_date) * INTERVAL '1 day')` AS grp, sau đó `GROUP BY user_id, grp HAVING COUNT(*) >= 5.`
- **Bài 13:**

```

1 WITH RECURSIVE cat_tree AS (
2     SELECT cat_id, cat_name, parent_id, 1 AS depth FROM
   categories WHERE parent_id IS NULL
3     UNION ALL
4     SELECT c.cat_id, c.cat_name, c.parent_id, ct.depth + 1 FROM
   categories c JOIN cat_tree ct ON c.parent_id = ct.cat_id
5 ) SELECT * FROM cat_tree ORDER BY depth, cat_name;
6

```

- **Bài 14:** `SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 1 THEN amount ELSE 0 END) AS Q1_Sales....`
- **Bài 15:** `SELECT * FROM employees e WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);`
- **Bài 16:** `WITH dupes AS (SELECT ctid, ROW_NUMBER() OVER(PARTITION BY order_id ORDER BY ctid) AS rn FROM orders) DELETE FROM orders WHERE ctid IN (SELECT ctid FROM dupes WHERE rn > 1);`
- **Bài 17:** Xác định `cohort_month = MIN(order_date)`, đếm số khách hàng hoạt động ở các tháng $T + 0, T + 1, T + 2$ chia cho quy mô cohort ban đầu.
- **Bài 18:** Bỏ phép tính quanh cột: Đổi thành `WHERE order_date > NOW() - INTERVAL '7 days'.`
- **Bài 19:** `CREATE INDEX idx_cust_incl ON orders (customer_id) INCLUDE (order_id, total_amount);`
- **Bài 20:**

```
1 INSERT INTO dim_user_profiles (user_id, last_login, login_count)
2 VALUES ('USR101', NOW(), 1)
3 ON CONFLICT (user_id) DO UPDATE SET
4     last_login = NOW(),
5     login_count = dim_user_profiles.login_count + 1;
6
```

Chương 19

Workbook Phần 4: 20 Bài Tập Xác Suất, Thống Kê & A/B Testing

Kiến Thức Cốt Lõi: Mục Tiêu Luyện Tập Phần 4

Phần này cung cấp **20 bài tập tính toán và giải thích thống kê chuyên sâu**, rèn luyện tư duy định lượng vững chắc để trả lời các câu hỏi phỏng vấn hóc búa nhất về A/B Testing và suy luận dữ liệu.

Bài Tập Tự Luyện: Hệ Thống 20 Bài Tập Thống Kê & A/B Testing

- Bài 1 (Mean vs Median):** Cho tập dữ liệu thời gian phản hồi máy chủ: [120, 125, 130, 135, 140, 145, 1200] (mili-giây). Hãy tính Mean và Median. Giải thích tại sao trong giám sát hệ thống hạ tầng (SRE/DE), người ta luôn dùng Percentile (P95, P99) thay vì Mean.
- Bài 2 (Độ Lệch Chuẩn & Quy Tắc 3-Sigma):** Tỷ lệ nhấp chuột (CTR) trung bình là $\mu = 5.0\%$ với độ lệch chuẩn $\sigma = 0.5\%$. Giả sử CTR tuân theo phân phối chuẩn, có bao nhiêu phần trăm khả năng CTR trong một ngày sẽ rơi vào khoảng [4.0%, 6.0%]?
- Bài 3 (Định Lý Giới Hạn Trung Tâm – CLT):** Một quần thể có phân phối lệch phải rất nặng với trung bình $\mu = 50$ và phương sai $\sigma^2 = 100$. Nếu ta lấy mẫu ngẫu nhiên gồm $n = 100$ quan sát, hãy xác định phân phối, kỳ vọng và sai số chuẩn (Standard Error) của giá trị trung bình mẫu $\bar{X}$.
- Bài 4 (Ước Lượng Khoảng Tin Cậy 95%):** Một mẫu gồm $n = 400$ người dùng cho thấy chi tiêu trung bình là $\bar{x} = \$120$ với độ lệch chuẩn mẫu $s = \$30$. Hãy tính khoảng tin cậy 95% (95% Confidence Interval) cho mức chi tiêu trung bình của toàn bộ người dùng.
- Bài 5 (Tính Kích Thước Mẫu A/B Test):** Doanh nghiệp muốn chạy A/B Testing trên nút Mua Hàng. Baseline Conversion Rate hiện tại là $p = 10\%$. Bạn muốn phát hiện mức tăng tương đối tối thiểu $MDE = 10\%$ (tức là tăng lên 11%), với mức ý nghĩa $\alpha = 0.05$ và độ nhạy thống kê $Power = 80\%$. Tính số lượng người dùng tối thiểu cần cho mỗi nhóm.
- Bài 6 (Kiểm Tra Lỗi Phân Luồng SRM):** Bạn cấu hình chia traffic 50:50 cho 2 nhóm. Sau

7 ngày, nhóm Control nhận được 52,000 lượt truy cập, nhóm Treatment nhận được 48,000 lượt. Sử dụng kiểm định Chi-Square goodness-of-fit để xác định xem thử nghiệm có bị dính lỗi Sample Ratio Mismatch (SRM) không.

7. **Bài 7 (Two-Proportion Z-Test):** Nhóm Control có $n_1 = 10,000$ người và $x_1 = 800$ chuyển đổi ($CR_1 = 8.0\%$). Nhóm Treatment có $n_2 = 10,000$ người và $x_2 = 900$ chuyển đổi ($CR_2 = 9.0\%$). Tính Z-statistic và đưa ra kết luận kinh doanh ở mức ý nghĩa $\alpha = 0.05$.
8. **Bài 8 (Ý Nghĩa Thực Sự Của P-Value):** Một thử nghiệm A/B Testing cho kết quả $p = 0.03$. Phát biểu sau đây đúng hay sai: "*Có 97% khả năng là nhóm Treatment tốt hơn nhóm Control*"? Hãy giải thích định nghĩa chuẩn xác của p-value.
9. **Bài 9 (Sai Lầm Loại I vs Loại II):** Trong bài toán phát hiện giao dịch gian lận ngân hàng, sai lầm Loại I (α) và sai lầm Loại II (β) tương ứng với tình huống nghiệp vụ nào? Doanh nghiệp nên ưu tiên giảm thiểu sai lầm nào hơn?
10. **Bài 10 (Nghịch Lý Simpson):** Một sản TMĐT thử nghiệm tính năng mới: Trên máy tính (Desktop), tính năng mới tăng CR từ 10% lên 12%; Trên điện thoại (Mobile), tính năng mới tăng CR từ 2% lên 3%. Nhưng khi gộp chung cả hai thiết bị, CR của nhóm mới lại THẤP HƠN nhóm cũ! Hãy phân tích nguyên nhân toán học gây ra nghịch lý này.
11. **Bài 11 (Peeking Problem & Early Stopping):** Tại sao việc kiểm tra dashboard kết quả A/B Test mỗi ngày một lần và bấm nút dừng thử nghiệm ngay khi thấy $p < 0.05$ lại là hành vi sai lầm nghiêm trọng?
12. **Bài 12 (Hiệu Ứng Novelty Effect):** Khi ra mắt một giao diện hoàn toàn mới, số liệu lượt nhấp tăng vọt trong tuần đầu tiên nhưng giảm dần và đi ngang ở tuần thứ 3. Đây là hiệu ứng gì và cách khắc phục trong thiết kế thử nghiệm?
13. **Bài 13 (Định Lý Bayes Trong Kiểm Tra Sức Khỏe):** Tỷ lệ mắc một căn bệnh hiếm là 0.1% ($P(Disease) = 0.001$). Một xét nghiệm y tế có độ nhạy (True Positive Rate) là 99% và tỷ lệ dương tính giả (False Positive Rate) là 5%. Nếu một người nhận kết quả dương tính, xác suất người đó thực sự mắc bệnh là bao nhiêu?
14. **Bài 14 (Confusion Matrix & Imbalanced Classes):** Cho mô hình phân loại nợ xấu (Default): $TP = 80, FP = 20, FN = 120, TN = 9,780$. Hãy tính Accuracy, Precision, Recall và F1-Score. Nhận xét về chỉ số Accuracy trong trường hợp này.
15. **Bài 15 (Precision-Recall Trade-off):** Làm thế nào để điều chỉnh ngưỡng xác suất phân loại (Classification Threshold) nhằm tăng Recall từ 60% lên 85%? Việc này sẽ ảnh hưởng như thế nào đến chỉ số Precision?
16. **Bài 16 (Ý Nghĩa Của Đường Cong ROC-AUC):** Chỉ số $AUC = 0.85$ có ý nghĩa xác suất như thế nào khi lấy ngẫu nhiên 1 mẫu tích cực (Positive) và 1 mẫu tiêu cực (Negative)?
17. **Bài 17 (Kiểm Định Chi-Square Độc Lập):** Bảng quan sát giữa Thiết bị (Mobile vs Desktop) và Quyết định mua hàng (Mua vs Không mua). Trình bày các bước tính giá trị kỳ vọng (Expected Frequencies) và thống kê Chi-Square.

18. **Bài 18 (Phân Phối Poisson):** Trung bình một trung tâm dữ liệu gặp $\lambda = 2$ sự cố phần cứng mỗi tuần. Tính xác suất để trong một tuần cụ thể, trung tâm này không gặp bất kỳ sự cố nào ($k = 0$).
19. **Bài 19 (Kiểm Định Student's T-Test):** Khi nào bắt buộc phải sử dụng kiểm định t-test thay vì Z-test khi so sánh giá trị trung bình của hai nhóm?
20. **Bài 20 (Power Analysis):** Nếu kích thước mẫu thu thập được trong thực tế nhỏ hơn kích thước mẫu tính toán ban đầu, điều này ảnh hưởng trực tiếp như thế nào đến độ nhạy thống kê ($Power = 1 - \beta$) và nguy cơ của doanh nghiệp là gì?

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Lời Giải Ngắn Gọn Cho 20 Bài Tập Thống Kê

- **Bài 1:** Mean = 265ms; Median = 135ms. Ngoại lai 1200ms làm sai lệch Mean; P95/P99 phản ánh trải nghiệm của 95–99% người dùng thực tế.
- **Bài 2:** $[4.0\%, 6.0\%] = [\mu - 2\sigma, \mu + 2\sigma]$. Theo quy tắc phân phối chuẩn, có **95.45%** dữ liệu rơi vào khoảng này.
- **Bài 3:** Theo CLT, với $n = 100$, $\bar{X} \sim \mathcal{N}(\mu = 50, SE = \sigma/\sqrt{n} = 10/10 = 1.0)$.
- **Bài 4:** $SE = 30/\sqrt{400} = 1.5$. Khoảng tin cậy 95%: $120 \pm 1.96 \times 1.5 = [117.06, 122.94]$ USD.
- **Bài 5:** $MDE = 10\% \times 10\% = 0.01$. Áp dụng công thức cỡ mẫu: cần khoảng **31,200 người dùng cho MỖI nhóm** (tổng cộng 62,400 người dùng).
- **Bài 6:** Expected = 50,000. $\chi^2 = (52000 - 50000)^2/50000 + (48000 - 50000)^2/50000 = 80 + 80 = 160$. Với bậc tự do 1, ngưỡng tới hạn là 3.84; $\chi^2 = 160 \gg 3.84 \rightarrow p < 10^{-15}$. Khẳng định chắc chắn thử nghiệm bị lỗi phân luồng SRM nghiêm trọng!
- **Bài 7:** Pooled $p = 1700/20000 = 0.085$. $SE = \sqrt{0.085 \times 0.915 \times (2/10000)} \approx 0.00394$. $Z = (0.09 - 0.08)/0.00394 \approx 2.536$. Vì $Z = 2.536 > 1.96$ ($p \approx 0.011 < 0.05$), sự gia tăng có ý nghĩa thống kê!
- **Bài 8:** Phát biểu đó HOÀN TOÀN SAI. P-value là $P(\text{Dữ liệu cực đoan như vậy} | H_0 \text{ đúng})$, không phải xác suất của giả thuyết $P(H_1 | \text{Data})$.
- **Bài 9:** Loại I: Khách hàng lương thiện bị chặn giao dịch. Loại II: Kẻ gian lận lọt lưới. Ngân hàng ưu tiên giảm Loại II (tăng Recall) vì thiệt hại tài chính do gian lận lớn hơn chi phí xác minh lại.
- **Bài 10:** Do phân bổ tỷ trọng traffic không đều (Confounding Variable). Nếu nhóm mới nhận quá nhiều traffic Mobile (vốn có CR tự nhiên thấp), tổng số chuyển đổi bị kéo tụt xuống dù ở từng thiết bị nó đều thắng.
- **Bài 11:** Vi phạm kiểm định giả thuyết nhiều lần (Multiple Testing Problem), làm tỷ lệ dương tính giả tích lũy tăng từ 5% lên tới gần 30–50%.

- **Bài 12:** Hiệu ứng hiếu kỳ (Novelty Effect). Khắc phục: Chạy thử nghiệm ít nhất 2–3 tuần và chỉ phân tích trên tập người dùng quen (không tính người dùng mới).
- **Bài 13:** $P(D|+) = \frac{0.99 \times 0.001}{(0.99 \times 0.001) + (0.05 \times 0.999)} = \frac{0.00099}{0.05094} \approx 1.94\%$.
- **Bài 14:** Accuracy = $(80 + 9780)/10000 = 98.6\%$ (Ảo tưởng!); Precision = $80/100 = 80\%$; Recall = $80/200 = 40\%$ (Bỏ lọt 60% nợ xấu!); F1 = $2 \times 0.8 \times 0.4 / (0.8 + 0.4) \approx 0.533$.
- **Bài 15:** Hạ thấp ngưỡng phân loại (ví dụ từ 0.5 xuống 0.35) sẽ giúp bắt được nhiều ca tích cực hơn (tăng Recall), nhưng chấp nhận số lượng báo động giả tăng lên (làm giảm Precision).
- **Bài 16:** Xác suất để mô hình xếp hạng điểm số (predicted score) của một mẫu Positive cao hơn một mẫu Negative ngẫu nhiên là đúng bằng 85%.
- **Bài 17:** $E_{ij} = (\text{Tổng dòng } i \times \text{Tổng cột } j) / \text{Tổng số quan sát}$. Sau đó tính $\chi^2 = \sum (O_{ij} - E_{ij})^2 / E_{ij}$.
- **Bài 18:** $P(X = 0) = \frac{e^{-2} \times 2^0}{0!} = e^{-2} \approx 13.53\%$.
- **Bài 19:** Bắt buộc dùng t-test khi kích thước mẫu nhỏ ($n < 30$) và phương sai tổng thể σ^2 chưa được biết (phải ước lượng qua độ lệch chuẩn mẫu s).
- **Bài 20:** Power giảm xuống dưới 80%. Doanh nghiệp đối mặt nguy cơ tăng Sai lầm Loại II: Tính năng mới thực sự tốt nhưng không đủ nhạy để phát hiện, dẫn đến quyết định hủy bỏ tính năng một cách oan uổng.

Workbook Phần 5: 20 Tình Huống Phân Tích Nghiệp Vụ & Đo Lường Hiệu Quả (Business Analytics & Metrics)

Kiến Thức Cốt Lõi: Mục Tiêu Luyện Tập Phần 5

Phần này cung cấp **20 tình huống và bài toán kinh doanh định lượng thực tế** mà mọi Data Analyst, Product Analyst và Business Intelligence Engineer phải đối mặt hàng ngày. Các bài tập rèn luyện kỹ năng phân tích chỉ số tài chính, phễu chuyển đổi, định giá, phân khúc khách hàng, và cây chỉ số nghiệp vụ (KPI Trees).

Bài Tập Tự Luyện: Hệ Thống 20 Bài Toán Nghiệp Vụ & Phân Tích Chỉ Số

- Bài 1 (Tính Customer Lifetime Value – CLV):** Một sản phẩm thương mại điện tử có Giá trị đơn hàng trung bình $AOV = \$85$, Tần suất mua hàng trung bình $f = 3.2$ lần/năm, Biên lợi nhuận gộp $Gross\ Margin = 40\%$, và Tỷ lệ rời bỏ khách hàng hàng năm $Churn\ Rate = 20\%$. Hãy tính Giá trị vòng đời khách hàng (CLV) theo mô hình đơn giản và phân tích tác động nếu Churn Rate giảm xuống còn 15%.
- Bài 2 (CAC & Thời Gian Hoàn Vốn Payback Period):** Trong Quý 1, tổng chi phí Marketing và Sales là \$120,000. Công ty mang về 3,000 khách hàng mới, trong đó 1,000 khách đến từ kênh tự nhiên (Organic) và 2,000 khách đến từ quảng cáo trả phí (Paid). Doanh thu trung bình trên mỗi khách hàng hàng tháng $ARPU = \$25$, biên lợi nhuận gộp 80%. Tính Blended CAC, Paid CAC, và thời gian hoàn vốn (CAC Payback Period) theo tháng.
- Bài 3 (Phân Tích Cohort Retention & Churn):** Nhóm khách hàng đăng ký tháng 01/2026 gồm 1,000 người. Số lượng người dùng còn hoạt động qua các tháng lần lượt là: Tháng 1 (1,000), Tháng 2 (650), Tháng 3 (520), Tháng 4 (480), Tháng 5 (460). Hãy tính Retention Rate từ M0 đến M4. Nhận xét về độ dốc của đường cong lưu giữ và xác định thời điểm sản phẩm đạt trạng thái ổn định (Retention Curve Flattening).

4. **Bài 4 (Tối Ưu Hóa Phễu Chuyển Đổi – Funnel Drop-off):** Dữ liệu phễu thanh toán của ứng dụng bán lẻ: Lướt xem sản phẩm: 200,000 → Thêm vào giỏ hàng: 50,000 → Bấm thanh toán: 15,000 → Điền thông tin giao hàng: 12,000 → Thanh toán thành công: 3,600. Tính Conversion Rate của toàn phễu (End-to-End CR) và xác định bước có tỷ lệ thất thoát (Drop-off Rate) nghiêm trọng nhất cần ưu tiên cải tiến UX.
5. **Bài 5 (Nghịch Lý Simpson Trong Chiến Dịch Tiếp Thị):** Đội Marketing chạy 2 chiến dịch (Campaign A và Campaign B) trên 2 nền tảng Facebook và Google. Dữ liệu ghi nhận:
- Facebook: Campaign A có $CR = 2.0\%$ (20/1,000 clicks); Campaign B có $CR = 2.5\%$ (100/4,000 clicks).
 - Google: Campaign A có $CR = 8.0\%$ (320/4,000 clicks); Campaign B có $CR = 8.5\%$ (85/1,000 clicks).
- Tính tổng Conversion Rate của từng chiến dịch gộp cả hai nền tảng. Giải thích tại sao Campaign B thắng ở cả hai nền tảng riêng lẻ nhưng khi gộp lại Campaign A lại có tỷ lệ chuyển đổi cao hơn gấp bội.
6. **Bài 6 (Cây Chỉ Số Nghiệp Vụ – KPI Tree Decomposition):** Doanh thu tháng này giảm 12% so với tháng trước. Ban giám đốc yêu cầu phân rã nguyên nhân dựa trên công thức: $\text{Doanh Thu} = \text{Traffic} \times \text{Conversion Rate} \times \text{AOV}$. Biết rằng Traffic tăng 10%, AOV tăng 5%. Hãy tính xem Conversion Rate đã thay đổi bao nhiêu phần trăm và đưa ra kết luận điều tra nguyên nhân.
7. **Bài 7 (Độ Co Giãn Của Cầu Theo Giá – Price Elasticity of Demand):** Một công ty SaaS tăng giá gói thuê bao từ \$40/tháng lên \$48/tháng (+20%). Lượng người dùng trả phí giảm từ 5,000 xuống còn 4,200 người (-16%). Tính hệ số co giãn của cầu theo giá (E_d). Quyết định tăng giá này có làm tăng hay giảm tổng doanh thu của công ty?
8. **Bài 8 (Mô Hình Phân Bổ Tiếp Thị Đa Kênh – Attribution Models):** Một khách hàng thực hiện hành trình: Nhấp Quảng cáo Facebook (\$0) → Xem bài viết Blog Organic → Nhấp Google Search Ads → Mua hàng trị giá \$180. Hãy phân bổ doanh thu cho từng kênh theo 3 mô hình: First-Touch, Last-Touch, và Linear Attribution. Phân tích điểm yếu chí mạng của Last-Touch Attribution.
9. **Bài 9 (Phân Khúc Khách Hàng RFM):** Trình bày quy tắc xếp hạng Recency (R), Frequency (F), Monetary (M) từ 1 đến 5 điểm. Một khách hàng có điểm RFM là 1 - 5 - 5 (R=1, F=5, M=5) thuộc nhóm khách hàng nào? Doanh nghiệp cần hành động chiến lược gì ngay lập tức đối với nhóm này?
10. **Bài 10 (Kinh Tế Học Đơn Vị – Unit Economics Trong Giao Đồ Ăn):** Một đơn hàng trà sữa có giá trị đơn (GTV) là 100,000 VND. Nguồn thu: Phí giao hàng thu từ khách: 15,000 VND; Hoa hồng chiết khấu từ quán ăn: 20% GTV. Chi phí phát sinh: Trả tiền shipper: 18,000 VND; Voucher khuyến mãi công ty tài trợ: 8,000 VND; Phí xử lý cổng thanh toán: 1.5% GTV; Chi phí hỗ trợ khách hàng và vận hành: 3,000 VND. Tính Contribution Margin (Biên lợi nhuận đóng góp) trên đơn hàng này. Doanh nghiệp đang lãi hay lỗ trên từng đơn?

11. **Bài 11 (Vòng Quay Tồn Kho & Điểm Đặt Hàng Lại – DSI & ROP):** Giá vốn hàng bán (COGS) hàng năm là \$1,440,000. Giá trị hàng tồn kho trung bình là \$120,000. Doanh số tiêu thụ trung bình mỗi ngày là 40 sản phẩm. Thời gian nhà cung cấp giao hàng (Lead Time) là 10 ngày. Lượng hàng tồn kho an toàn (Safety Stock) duy trì ở mức 150 sản phẩm. Hãy tính Số ngày tồn kho bình quân (DSI) và Điểm đặt hàng lại (Reorder Point – ROP).
12. **Bài 12 (Net Promoter Score – NPS):** Khảo sát 2,000 khách hàng về mức độ sẵn sàng giới thiệu sản phẩm: 1,100 người chấm 9–10 điểm; 600 người chấm 7–8 điểm; 300 người chấm 0–6 điểm. Hãy tính chỉ số NPS. Vì sao việc nhiều nhà quản lý loại bỏ nhóm Passives (7–8 điểm) ra khỏi phân tích là một sai lầm nguy hiểm?
13. **Bài 13 (Tỷ Lệ Triệt Tiêu Doanh Số – Cannibalization Rate):** Hãng ra mắt dòng tai nghe không dây mới "AirBuds Pro" với giá \$200, bán được 10,000 chiếc trong tháng đầu. Tuy nhiên, doanh số của dòng tai nghe cũ "AirBuds Basic" (giá \$120) bị giảm từ 15,000 chiếc xuống còn 9,000 chiếc. Tính tỷ lệ triệt tiêu (Cannibalization Rate) và cho biết quyết định ra mắt sản phẩm mới có làm tăng tổng doanh thu không?
14. **Bài 14 (Tỷ Lệ Khiếu Nại Bồi Hoàn – Chargeback Rate):** Một công ty thương mại điện tử xử lý 50,000 giao dịch trong tháng 3. Đội phòng chống gian lận ghi nhận 450 giao dịch bị chủ thẻ khiếu nại lừa đảo và ngân hàng thực hiện hoàn tiền (Chargeback). Tính Chargeback Rate. So sánh với ngưỡng giới hạn 1.0% (hoặc 0.9%) của các tổ chức thẻ quốc tế (Visa/Mastercard) và nêu hậu quả nếu vượt ngưỡng.
15. **Bài 15 (Phân Tích Giỏ Hàng – Market Basket Analysis):** Trong 5,000 giao dịch tại siêu thị: Có 1,000 hóa đơn chứa Bánh mì, 800 hóa đơn chứa Sữa tươi, và 400 hóa đơn chứa CẢ Bánh mì và Sữa tươi. Hãy tính Support, Confidence và Lift của luật kết hợp: {Bánh mì} → {Sữa tươi}. Nhận xét xem hai sản phẩm này có mối quan hệ tương hỗ thực sự hay chỉ xuất hiện ngẫu nhiên.
16. **Bài 16 (SaaS Net Revenue Retention – NRR & Quick Ratio):** Đầu tháng, công ty có Doanh thu định kỳ tháng (Beginning MRR) là \$200,000. Trong tháng diễn ra các biến động:
- Khách hàng cũ nâng cấp gói (Expansion MRR): +\$24,000
 - Khách hàng cũ hạ gói (Contraction MRR): -\$6,000
 - Khách hàng hủy hợp đồng hoàn toàn (Churn MRR): -\$10,000
 - Khách hàng hoàn toàn mới (New MRR): +\$30,000
- Tính Tỷ lệ duy trì doanh thu thuần (NRR) và Chỉ số tăng trưởng nhanh (SaaS Quick Ratio). Doanh nghiệp có đạt tiêu chuẩn tăng trưởng bền vững của các quỹ đầu tư mạo hiểm (NRR > 100%) không?
17. **Bài 17 (Chỉ Số Chăm Sóc Khách Hàng – CSAT vs FCR):** Nhóm hỗ trợ khách hàng đạt Thời gian phản hồi lần đầu (First Response Time – FRT) cực nhanh là 1.5 phút (vượt KPI đề ra là 5 phút), nhưng điểm hài lòng khách hàng (CSAT) lại giảm từ 4.6 xuống 3.7 sao. Khi phân tích sâu, chỉ số First Contact Resolution (FCR – Giải quyết triệt để vấn đề ngay lần liên hệ đầu) chỉ đạt 42%. Phân tích xung đột lợi ích giữa chỉ số đo lường tốc độ (Speed Metric) và chất lượng giải quyết (Quality Metric).

18. **Bài 18 (Tác Động Của Tỷ Lệ Hoàn Hàng E-Commerce):** Một cửa hàng thời trang trực tuyến bán 8,000 đơn hàng/tháng, giá bán bình quân \$60/đơn, giá vốn hàng bán \$25/đơn. Chi phí vận chuyển chiều đi do shop chịu là \$4/đơn. Tỷ lệ đổi trả hàng (Return Rate) là 20%. Với mỗi đơn đổi trả, shop mất thêm \$5 chi phí vận chuyển hoàn về và kiểm định lại kho hàng. Tính lợi nhuận gộp thực tế của shop sau khi đã trừ toàn bộ chi phí đổi trả.
19. **Bài 19 (Hoạch Định Năng Suất – Capacity Utilization Trong Vận Tải):** Một đội xe vận tải có 50 xe tải. Mỗi xe có khả năng chạy tối đa 26 ngày/tháng, bình quân chở được 10 tấn hàng/chuyến (mỗi ngày chạy 1 chuyến). Trong tháng, toàn đội xe đã thực hiện 1,040 chuyến xe và vận chuyển tổng cộng 8,840 tấn hàng. Tính Tỷ lệ sử dụng số chuyến (Trip Utilization Rate) và Tỷ lệ lấp đầy tải trọng bình quân (Capacity Load Factor).
20. **Bài 20 (Thiết Kế Cây Chỉ Số Bắc Đẩu – North Star Metric):** Đối với nền tảng gọi xe công nghệ (Ride-hailing platform kết nối Hành khách và Tài xế), giải thích tại sao chọn "*Số lượt tải ứng dụng (App Installs)*" hoặc "*Tổng Doanh Thu (GMV)*" làm North Star Metric là một sai lầm ngắn hạn. Hãy đề xuất 1 North Star Metric chuẩn xác và 4 Input Metrics thuộc các chiều: Cung (Supply), Cầu (Demand), Chất lượng khớp lệnh (Matching Quality), và Trải nghiệm người dùng (UX).

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Lời Giải Ngắn Gọn Cho 20 Bài Toán Nghiệp Vụ

- **Bài 1:** Doanh thu năm/khách = $85 \times 3.2 = \$272$. Lợi nhuận gộp năm = $272 \times 40\% = \$108.8$. $CLV = 108.8/0.20 = \$544$. Nếu Churn Rate giảm xuống 15%, $CLV = 108.8/0.15 = \$725.33$ (tăng trưởng tới **33.3%** giá trị trọn đời).
- **Bài 2:** Blended CAC = $120000/3000 = \$40$. Paid CAC = $120000/2000 = \$60$. Lợi nhuận gộp hàng tháng/khách = $25 \times 80\% = \$20$. Thời gian hoàn vốn đối với khách Paid = $60/20 = 3.0$ tháng. Đây là chỉ số rất tốt (chuẩn thị trường dưới 12 tháng).
- **Bài 3:** Retention Rates: M0 = 100%, M1 = 65%, M2 = 52%, M3 = 48%, M4 = 46%. Đường cong giảm mạnh ở tháng đầu (mất 35% người dùng do Onboarding hoặc chưa thấy giá trị cốt lõi), sau đó phẳng dần từ M2 đến M4 (giảm nhẹ 2–4%/tháng), chứng minh sản phẩm đã đạt Product-Market Fit với tệp khách hàng trung thành ở mức $\sim 46\%$.
- **Bài 4:** End-to-End CR = $3600/200000 = 1.8\%$. Drop-off rates từng bước: Xem → Giỏ hàng: 75%; Giỏ hàng → Bấm TT: 70%; Bấm TT → Điền thông tin: 20%; Điền thông tin → TT thành công: 70%. Hai điểm rơi nghiêm trọng nhất là Xem → Thêm giỏ hàng (75%) và Điền thông tin → Hoàn tất thanh toán (70% do lỗi cổng thanh toán, phí ship bất ngờ, hoặc bắt buộc đăng ký phức tạp).
- **Bài 5:** Campaign A: Tổng CR = $(20 + 320)/(1000 + 4000) = 340/5000 = 6.8\%$. Campaign B: Tổng CR = $(100 + 85)/(4000 + 1000) = 185/5000 = 3.7\%$. Nghịch lý xảy ra vì Campaign A đổ tới 80% traffic vào Google (kênh có CR cơ bản cao 8%), trong khi Campaign B đổ tới 80% traffic vào Facebook (kênh có CR cơ bản thấp 2.5%). Biến ẩn (Confounding Variable) chính là tỷ trọng phân bổ nền tảng.

- **Bài 6:** Gọi tỷ lệ thay đổi của CR là x . Ta có phương trình chỉ số: $(1 - 0.12) = (1 + 0.10) \times (1 + x) \times (1 + 0.05) \Leftrightarrow 0.88 = 1.155 \times (1 + x) \Rightarrow 1 + x = 0.88/1.155 \approx 0.7619$. Như vậy, Conversion Rate đã **giảm sút tới 23.81%**. Cần rà soát ngay lập tức luồng checkout, khuyến mãi, hoặc chất lượng nguồn traffic mới kéo về.
- **Bài 7:** $\% \Delta P = +20\%$. $\% \Delta Q = (4200 - 5000)/5000 = -16\%$. $E_d = -16\%/20\% = -0.8$. Vì $|E_d| < 1$, cầu co giãn ít (Inelastic Demand). Doanh thu cũ: $5000 \times 40 = \$200,000$; Doanh thu mới: $4200 \times 48 = \$201,600$ (tăng \$1,600). Quyết định tăng giá làm tăng tổng doanh thu.
- **Bài 8:** First-Touch: Facebook hưởng 100% (\$180); Last-Touch: Google Ads hưởng 100% (\$180); Linear: Mỗi kênh (FB, Blog, Google) hưởng 33.33% (\$60). Điểm yếu của Last-Touch: Bỏ qua hoàn toàn vai trò xây dựng nhận thức thương hiệu (Awareness) của các kênh đầu phễu, dẫn đến quyết định sai lầm khi cắt ngân sách Facebook khiến nguồn khách mới cạn kiệt.
- **Bài 9:** Nhóm 1-5-5 là nhóm **"At Risk / Cannot Lose Them"** (Khách VIP có nguy cơ rời bỏ): Đã từng mua sắm cực kỳ thường xuyên và chi tiêu rất lớn, nhưng đã lâu không quay lại. Hành động chiến lược: Kích hoạt chiến dịch Re-engagement cá nhân hóa ngay, gọi điện chăm sóc trực tiếp từ quản lý tài khoản (Account Manager), hoặc tặng ưu đãi độc quyền cao cấp.
- **Bài 10:** Doanh thu gộp đơn hàng = $15000 + (100000 \times 20\%) = 35,000$ VND. Tổng chi phí trực tiếp trên đơn = $18000 + 8000 + (100000 \times 1.5\%) + 3000 = 18000 + 8000 + 1500 + 3000 = 30,500$ VND. Contribution Margin = $35000 - 30500 = +4,500$ VND (đạt tỷ suất lợi nhuận đóng góp 4.5%). Đơn hàng có lãi biên dương.
- **Bài 11:** Vòng quay tồn kho = $\text{COGS}/\text{Inventory} = 1440000/120000 = 12$ vòng/năm. $DSI = 365/12 \approx 30.42$ ngày. Điểm đặt hàng lại: $ROP = (\text{Lead Time} \times \text{Daily Demand}) + \text{Safety Stock} = (10 \times 40) + 150 = 550$ sản phẩm. Khi kho còn 550 sản phẩm, hệ thống phải tự động tạo đơn nhập hàng.
- **Bài 12:** $\% \text{Promoters} = 1100/2000 = 55\%$. $\% \text{Detractors} = 300/2000 = 15\%$. $NPS = 55\% - 15\% = +40$. Bỏ qua Passives là sai lầm vì nhóm này chiếm tới 30% khách hàng; họ có mức độ trung thành thấp, rất dễ bị đối thủ lôi kéo chỉ bằng một chương trình giảm giá nhỏ nếu doanh nghiệp không kịp thời nâng cao trải nghiệm của họ.
- **Bài 13:** Lượng máy Basic sụt giảm: $15000 - 9000 = 6,000$ chiếc. Cannibalization Rate = $6000/10000 = 60\%$ (cứ 10 chiếc Pro bán ra thì có 6 chiếc lấy mất thị phần của Basic). Doanh thu cũ: $15000 \times 120 = \$1,800,000$. Doanh thu mới: $(9000 \times 120) + (10000 \times 200) = 1,080,000 + 2,000,000 = \$3,080,000$ (tăng \$1,280,000). Quyết định ra mắt sản phẩm mới cực kỳ thành công.
- **Bài 14:** Chargeback Rate = $450/50000 = 0.90\%$. Tỷ lệ này đã chạm sát ngưỡng cảnh báo đỏ của Visa/Mastercard (thường là 0.9% - 1.0%). Doanh nghiệp đối mặt nguy cơ: Bị xếp vào chương trình giám sát rủi ro cao (Monitoring Program), phạt phí xử lý giao dịch tăng vọt từ \$15 lên \$50 mỗi đơn bồi hoàn, và nguy cơ nặng nhất là bị tước quyền thanh toán thẻ. Cần bật ngay xác thực 3D-Secure 2.0 và quy tắc kiểm duyệt địa chỉ IP/CVV.

- **Bài 15:** $\text{Support}(\text{Bánh mì} \rightarrow \text{Sữa}) = 400/5000 = \mathbf{8.0\%}$. $\text{Confidence} = 400/1000 = \mathbf{40.0\%}$ (40% người mua Bánh mì sẽ mua Sữa). Tần suất nền của Sữa: $P(\text{Sữa}) = 800/5000 = 16\%$. $\text{Lift} = 40\%/16\% = \mathbf{2.5}$. Vì $\text{Lift} = 2.5 > 1$, sự xuất hiện của Bánh mì làm tăng khả năng mua Sữa tươi lên gấp 2.5 lần so với ngẫu nhiên $\rightarrow$ Rất nên tạo combo hoặc đặt gần quầy kệ.
- **Bài 16:**
$$NRR = \frac{200000 + 24000 - 6000 - 10000}{200000} \times 100\% = \frac{208000}{200000} \times 100\% = \mathbf{104.0\%}$$
. Doanh nghiệp đạt chuẩn tăng trưởng bền vững (vượt mốc 100%, tức là ngay cả khi không có khách hàng mới, doanh thu từ khách hàng cũ vẫn tự tăng trưởng 4%).
$$\text{Quick Ratio} = \frac{\text{New MRR} + \text{Expansion MRR}}{\text{Churn MRR} + \text{Contraction MRR}} = \frac{30000 + 24000}{10000 + 6000} = \frac{54000}{16000} = \mathbf{3.375}$$
 (rất khỏe mạnh, vì > 2.0).
- **Bài 17:** Khi nhân viên hỗ trợ bị áp lực KPI "Phản hồi dưới 2 phút", họ có xu hướng gửi câu trả lời đóng khung mẫu (canned responses) nhanh chóng để đóng ticket mà không thực sự lắng nghe hoặc giải quyết triệt để vấn đề gốc rễ. Khách hàng phải nhắn tin lại nhiều lần khiến CSAT tụt dốc. Cần cân bằng KPI bằng cách đưa FCR làm chỉ số bảo vệ (Guardrail Metric) và chỉ tính đạt KPI khi $\text{CSAT} \geq 4.5$.
- **Bài 18:** Số đơn hoàn trả: $8000 \times 20\% = 1,600$ đơn. Số đơn thành công thực nhận: $8000 - 1600 = 6,400$ đơn. Doanh thu thực nhận: $6400 \times 60 = \$384,000$. Giá vốn COGS hàng đã bán: $6400 \times 25 = \$160,000$. Phí ship chiều đi của toàn bộ 8,000 đơn ban đầu: $8000 \times 4 = \$32,000$. Chi phí hoàn hàng phát sinh: $1600 \times 5 = \$8,000$. Tổng lợi nhuận gộp sau hoàn hàng: $384000 - 160000 - 32000 - 8000 = \mathbf{\$184,000}$. Tỷ suất lợi nhuận gộp thực tế: $184000/384000 \approx \mathbf{47.9\%}$.
- **Bài 19:** Năng lực vận chuyển tối đa về số chuyến: $50 \times 26 = 1,300$ chuyến. $\text{Trip Utilization Rate} = 1040/1300 = \mathbf{80.0\%}$. Khối lượng hàng tối đa có thể chở trên 1,040 chuyến đã chạy: $1040 \times 10 = 10,400$ tấn. $\text{Capacity Load Factor} = 8840/10400 = \mathbf{85.0\%}$. Hiệu suất toàn diện của đội xe: $80\% \times 85\% = 68.0\%$.
- **Bài 20:** Lượt tải app và GMV là chỉ số bề nổi (Vanity Metrics): Tải app nhiều nhưng gỡ app ngay không tạo ra giá trị; GMV cao có thể do đốt voucher trợ giá thiếu bền vững. **North Star Metric:** "Số chuyến xe hoàn thành thành công hàng tuần (Weekly Completed Trips)". 4 Input Metrics cốt lõi: (1) Cung: Tỷ lệ tài xế online giờ cao điểm (Driver Active Hours); (2) Cầu: Tỷ lệ mở ứng dụng và tìm chuyến (Search-to-Request Conversion); (3) Chất lượng khớp lệnh: Thời gian chờ xe trung bình (ETA dưới 4 phút) và Tỷ lệ hủy chuyến của tài xế (Driver Cancellation Rate $< 3\%$); (4) Trải nghiệm: Điểm đánh giá chuyến đi trung bình ≥ 4.8 sao.

Workbook Phần 6: 20 Bài Tập Thực Hành Data Engineering, Big Data & MLOps

Kiến Thức Cốt Lõi: Mục Tiêu Luyện Tập Phần 6

Phần này cung cấp **20 bài tập thực hành kỹ thuật chuyên sâu** dành cho kỹ sư dữ liệu (Data Engineer) và kỹ sư MLOps. Nội dung bao quát toàn diện từ mô hình hóa dữ liệu kho Kimball (Star Schema, SCD2), thiết kế đường ống ETL/ELT có tính lũy đẳng (Idempotency), Docker đa tầng, Apache Airflow TaskFlow, dbt models/tests, tối ưu hóa PySpark với Broadcast Join, Delta Lake ACID, kiến trúc luồng dữ liệu Kafka, triển khai Microservice FastAPI độ trễ thấp, và tự động hóa CI/CD với GitHub Actions.

Bài Tập Tự Luyện: Hệ Thống 20 Bài Tập Thực Chiến DE & MLOps

- Bài 1 (Thiết Kế Kimball Star Schema):** Cho hệ thống E-commerce với hàng triệu giao dịch. Hãy thiết kế lược đồ hình sao gồm: Bảng Fact `fact_order_items` (hạt nhân Grain: từng dòng sản phẩm trong đơn) và 4 bảng Dimension (`dim_customer`, `dim_product`, `dim_date`, `dim_store`). Chỉ ra khóa thay thế (Surrogate Key) và một Degenerate Dimension trong bảng Fact.
- Bài 2 (Xử Lý Lịch Sử Biến Đổi Chậm SCD Type 2):** Khi khách hàng chuyển nhà từ Hà Nội vào TP.HCM, làm thế nào để lưu vết lịch sử nhằm đảm bảo các báo cáo doanh thu theo vùng trong quá khứ không bị sai lệch? Viết cấu trúc DDL của bảng `dim_customer` có chứa các cột kỹ thuật quản lý SCD2 (`valid_from`, `valid_to`, `is_current`).
- Bài 3 (Nguyên Tắc Lũy Đẳng – Idempotent Pipeline):** Pipeline ETL chạy hàng ngày lúc 01:00 AM để nạp dữ liệu ngày hôm trước vào bảng `fact_sales`. Giả sử mạng bị đứt giữa chừng và Airflow tự động kích hoạt retry chạy lại. Trình bày chiến lược xử lý dữ liệu (Partition Overwrite / Merge) để đảm bảo pipeline chạy lại 10 lần thì dữ liệu trong kho vẫn chính xác, không sinh ra bản ghi trùng lặp (duplicate).
- Bài 4 (Xác Thực Schema Dữ Liệu Với Pydantic):** Viết một schema Pydantic bằng Python

để xác thực luồng dữ liệu thanh toán từ Webhook bên ngoài. Yêu cầu: `transaction_id` kiểu chuỗi không rỗng, `amount` kiểu float dương (> 0), `currency` chỉ nhận giá trị trong tập `["VND", "USD"]`, và `created_at` phải ở định dạng chuẩn ISO datetime.

5. **Bài 5 (Tối Ưu Hóa Dockerfile Đa Tầng – Multi-Stage Build):** Một ứng dụng Ingestion viết bằng Python có kích thước Docker Image lên tới 1.2GB khi build với image gốc `ubuntu:latest` và cài đầy đủ trình biên dịch C/C++. Hãy viết Dockerfile tối ưu sử dụng `python:3.11-slim`, kỹ thuật Multi-stage build, và cấu hình chạy bằng người dùng không có quyền root (`non-root user`) để nén kích thước xuống dưới 150MB.
6. **Bài 6 (Thiết Kế Cụm Dịch Vụ Với Docker Compose):** Viết file cấu hình `docker-compose.yml` để khởi chạy đồng bộ 3 dịch vụ: PostgreSQL Database (lưu dữ liệu có mount volume bền vững), Redis Cache (cổng 6379), và một container Python Worker. Yêu cầu cấu hình cơ chế kiểm tra sức khỏe (`healthcheck`) đảm bảo Worker chỉ khởi động khi PostgreSQL đã sẵn sàng nhận kết nối.
7. **Bài 7 (Apache Airflow TaskFlow API):** Viết một DAG Airflow hoàn chỉnh bằng cú pháp hiện đại TaskFlow (`@dag` và `@task`) thực hiện quy trình 3 bước: `extract()` (trích xuất danh sách đơn hàng từ API), `transform(raw_data)` (lọc bỏ đơn hủy và tính thuế), `load(clean_data)` (ghi vào Database). Cấu hình tham số tự động thử lại 3 lần với khoảng thời gian cách nhau 5 phút.
8. **Bài 8 (Airflow Trigger Rule & Cảnh Báo Lỗi):** Trong một pipeline phức tạp có nhiều nhánh rẽ, bạn muốn có một task cuối cùng gửi thông báo tổng kết lên kênh Slack bất kể các task phía trước thành công hay thất bại. Hãy cho biết cần thiết lập tham số `trigger_rule` của task này là gì và giải thích cơ chế hoạt động.
9. **Bài 9 (Xây Dựng dbt Staging Model):** Viết mô hình dbt SQL `stg_orders.sql` đọc dữ liệu thô từ bảng nguồn `raw.jaffle_shop.orders`. Thực hiện các thao tác: đổi tên cột kiểu `camelCase` sang `snake_case`, ép kiểu `order_date` sang kiểu `DATE`, chuyển đổi giá tiền từ `cents` sang `dollars`, và lọc bỏ các đơn hàng thử nghiệm có `is_test = true`.
10. **Bài 10 (Kiểm Thử Chất Lượng Dữ Liệu Bằng dbt Tests):** Viết file cấu hình `schema.yml` trong dbt để khai báo các bài test tự động cho bảng `stg_customers`: Cột `customer_id` phải là duy nhất (`unique`) và không được rỗng (`not_null`); Cột `status` chỉ được phép nhận các giá trị định trước (`accepted_values`). Viết thêm 1 bài Singular Test SQL kiểm tra không có đơn hàng nào có tổng tiền âm (< 0).
11. **Bài 11 (Chiến Lược Cập Nhật Tăng Dần – dbt Incremental):** Khi bảng đơn hàng tích lũy hơn 100 triệu dòng, việc build lại toàn bộ (`full-refresh`) mỗi ngày sẽ gây cạn kiệt tài nguyên máy chủ. Hãy cấu hình cấu trúc model dbt `materialized='incremental'` với khóa duy nhất `unique_key='order_id'` và viết đoạn code Jinja lọc lấy các bản ghi có `updated_at` mới hơn giá trị lớn nhất hiện có trong bảng đích.
12. **Bài 12 (PySpark Schema Enforcement & Partitioning):** Viết đoạn mã PySpark đọc một tệp CSV lớn (dung lượng 50GB) bằng cách định nghĩa trước cấu trúc Schema tường minh

- (StructType) thay vì dùng `inferSchema=True`. Sau đó, thực hiện ghi dữ liệu ra định dạng Parquet nén Snappy, phân vùng (partition) theo 2 cột `year` và `month`.
13. **Bài 13 (Tối Ưu Hóa Phép Join Bằng Broadcast Hash Join):** Trong Spark, khi thực hiện phép Join giữa bảng `fact_transactions` (1 tỷ bản ghi, kích thước 500GB) và bảng `dim_store` (500 cửa hàng, kích thước 2MB), hiện tượng xáo trộn dữ liệu (Data Shuffling) qua mạng sẽ làm chậm pipeline nghiêm trọng. Viết câu lệnh Spark tối ưu bằng `broadcast()` và phân tích nguyên lý hoạt động của Broadcast Hash Join.
 14. **Bài 14 (PySpark Window Functions):** Cho DataFrame `df_sales` gồm `[department, employee_id, salary, sale_amount]`. Hãy viết mã PySpark sử dụng Window Specification để: (1) Tính tổng lũy kế doanh số (Running Total) của từng nhân viên theo thời gian; (2) Xếp hạng top 3 nhân viên có doanh số cao nhất trong từng phòng ban bằng hàm `dense_rank()`.
 15. **Bài 15 (Kiến Trúc Lakehouse Với Delta Lake ACID):** Trình bày sự khác biệt cốt lõi giữa Parquet thuần túy và Delta Lake. Viết câu lệnh SQL/PySpark để thực hiện tính năng *Time Travel* (truy vấn lại dữ liệu tại phiên bản quá khứ `VERSION AS OF 3` hoặc tại thời điểm ngày hôm qua) và giải thích tác dụng của lệnh `VACUUM`.
 16. **Bài 16 (Hạ Tầng Luồng Dữ Liệu Thời Gian Thực Với Apache Kafka):** Một hệ thống clickstream người dùng tạo ra 10,000 sự kiện/giây. Để đảm bảo thứ tự thời gian của các sự kiện do cùng một người dùng tạo ra (in-order delivery) khi ghi vào Kafka Topic có 12 partitions, bạn cần thiết lập tham số gì khi Producer gửi message? Hiện tượng Consumer Lag là gì và cách mở rộng quy mô (Scale out) nhóm Consumer?
 17. **Bài 17 (Triển Khai Mô Hình ML Bằng Microservice FastAPI):** Viết mã nguồn ứng dụng FastAPI phục vụ dự đoán nguy cơ khách hàng rời bỏ (Churn Prediction). Yêu cầu: Định nghĩa Schema đầu vào bằng Pydantic; Load mô hình Scikit-Learn đã huấn luyện một lần duy nhất lúc server khởi động bằng cơ chế `lifespan context manager`; Trả về kết quả JSON gồm `churn_prediction` (0 hoặc 1) và `churn_probability` (float).
 18. **Bài 18 (Kiểm Thử Tải Hệ Thống Phục Vụ ML Bằng Locust):** Viết một script kiểm thử tải (Load Testing) bằng công cụ Locust trong Python để giả lập 200 người dùng đồng thời gửi request liên tục đến endpoint `/predict` của mô hình ML. Đo đạc các chỉ số: Requests Per Second (RPS), Thời gian phản hồi trung vị (Median Response Time), và phân vị P99.
 19. **Bài 19 (Thiết Lập Đường Ống CI/CD Tự Động Với GitHub Actions):** Viết file workflow cấu hình GitHub Actions `.github/workflows/data_pipeline_ci.yml` tự động kích hoạt khi có Pull Request vào nhánh `main`. Quy trình gồm: Khởi tạo môi trường Ubuntu, cài đặt Python 3.11, kiểm tra chất lượng mã nguồn bằng `flake8`, và tự động chạy toàn bộ bài kiểm thử đơn vị với `pytest`.
 20. **Bài 20 (Giám Sát Độ Lệch Dữ Liệu MLOps – Data Drift Detection):** Sau khi triển khai mô hình Machine Learning lên môi trường Production được 3 tháng, độ chính xác của mô hình bị suy giảm nghiêm trọng. Giải thích khái niệm Data Drift và Concept Drift. Trình bày phương

pháp định lượng độ lệch dữ liệu đầu vào sử dụng chỉ số PSI (Population Stability Index) và ngưỡng kích hoạt huấn luyện lại mô hình (Retraining Trigger).

Lời Giải & Hướng Dẫn Kiểm Tra: Đáp Án & Lời Giải Ngắn Gọn Cho 20 Bài Tập DE & MLOps

- **Bài 1:** Bảng Fact: `fact_order_items(order_item_key, order_number, customer_key, product_key, date_key, store_key, quantity, unit_price, discount_amount, net_amount)`. Surrogate Key: Các khóa nguyên tự tăng *_key. Degenerate Dimension: `order_number` (mã hóa đơn lưu trực tiếp trên Fact mà không cần tạo bảng Dim riêng vì không có thuộc tính mô tả bổ sung).
- **Bài 2:** DDL bảng SCD2: `CREATE TABLE dim_customer (customer_sk BIGINT PRIMARY KEY, customer_id INT, full_name VARCHAR(100), city VARCHAR(50), valid_from DATE NOT NULL, valid_to DATE, is_current BOOLEAN NOT NULL)`. Khi khách đổi địa chỉ: `UPDATE` bản ghi cũ gán `valid_to = CURRENT_DATE - 1, is_current = FALSE`; `INSERT` bản ghi mới với địa chỉ mới, `valid_from = CURRENT_DATE, valid_to = '9999-12-31', is_current = TRUE`.
- **Bài 3:** Áp dụng mô hình **Idempotent Ingestion**: Sử dụng chiến lược Partition Overwrite: `INSERT OVERWRITE TABLE fact_sales PARTITION (sale_date = '2026-03-15') SELECT ...` hoặc dùng lệnh `MERGE INTO fact_sales USING staging_sales ON ... WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...`. Dù Airflow chạy lại 10 lần thì kết quả của ngày 15/03 luôn được ghi đè chính xác 1 lần duy nhất.
- **Bài 4:** Khai báo Pydantic Schema:

```
1 from pydantic import BaseModel, Field
2 from datetime import datetime
3 from typing import Literal
4
5 class TransactionPayload(BaseModel):
6     transaction_id: str = Field(..., min_length=1)
7     amount: float = Field(..., gt=0)
8     currency: Literal["VND", "USD"]
9     created_at: datetime
10
```

- **Bài 5:** Sử dụng `python:3.11-slim`, Stage 1 (builder) cài đặt wheel packages; Stage 2 (final) chỉ sao chép wheel đã build, dọn dẹp cache `--no-cache-dir`, tạo người dùng `RUN useradd -m appuser && USER appuser`. Image cuối cùng không chứa gcc hay header files, giảm kích thước từ 1.2GB xuống ~ 115MB.
- **Bài 6:** Cấu hình Docker Compose:

```

1 services:
2   postgres:
3     image: postgres:15
4     environment:
5       POSTGRES_DB: analytics
6     volumes: [pgdata:/var/lib/postgresql/data]
7     healthcheck:
8       test: ["CMD-SHELL", "pg_isready -U postgres"]
9       interval: 5s
10      timeout: 5s
11      retries: 5
12   worker:
13     build: .
14     depends_on:
15       postgres:
16         condition: service_healthy
17

```

- **Bài 7:** Sử dụng cú pháp TaskFlow:

```

1 @dag(schedule="@daily", start_date=datetime(2026, 1, 1),
2       default_args={"retries": 3, "retry_delay":
3         timedelta(minutes=5)})
4 def etl_pipeline():
5     @task
6     def extract(): return [{"id": 1, "status": "COMPLETED",
7       "amt": 100}]
8     @task
9     def transform(data): return [x for x in data if x["status"]
10      != "CANCEL"]
11     @task
12     def load(clean): print(f"Loaded {len(clean)} rows.")
13     load(transform(extract()))
14 etl_pipeline()
15

```

- **Bài 8:** Cấu hình `trigger_rule=TriggerRule.ALL_DONE`. Mặc định Airflow là `all_success` (nếu task trước fail thì task sau bị skip). Với `all_done`, task thông báo sẽ luôn được thực thi sau khi tất cả upstream tasks đã hoàn tất chu kỳ chạy, bất kể thành công, thất bại hay bị hủy.
- **Bài 9:** dbt Model `stg_orders.sql`:

```

1 select
2   id as order_id,
3   user_id as customer_id,
4   cast(order_date as date) as order_date,

```

```

5     amount_cents / 100.0 as order_amount_usd,
6     status as order_status
7 from {{ source('raw_shop', 'orders') }}
8 where is_test = false
9

```

- **Bài 10:** Trong schema.yml: Khai báo columns: - name: customer_id, tests: [unique, not_null]. Test tùy biến Singular (tests/assert_no_negative_orders.sql):

```

1 SELECT order_id, order_total
2 FROM {{ ref('stg_orders') }}
3 WHERE order_total < 0
4

```

dbt sẽ báo lỗi (FAIL) nếu câu query này trả về bất kỳ dòng nào.

- **Bài 11:** Cấu hình dbt Incremental:

```

1 {{ config(materialized='incremental', unique_key='order_id') }}
2 select * from {{ ref('stg_orders') }}
3 {% if is_incremental() %}
4     where updated_at > (select max(updated_at) from {{ this }})
5 {% endif %}
6

```

- **Bài 12:** Ép Schema và ghi partitioned Parquet:

```

1 schema = StructType([StructField("id", LongType()),
2                             StructField("val", DoubleType()),
3                             StructField("year", IntegerType()),
4                             StructField("month", IntegerType())])
5 df = spark.read.schema(schema).csv("data/raw/*.csv")
6 df.write.partitionBy("year",
7                     "month").mode("overwrite").parquet("data/warehouse/fact/")
8

```

Tránh được thao tác quét toàn bộ dữ liệu (OOM crash) do inferSchema gây ra.

- **Bài 13:** Câu lệnh: df_joined = fact_df.join(broadcast(dim_store), "store_id"). Cơ chế: Spark sao chép toàn bộ bảng nhỏ (2MB) gửi đến bộ nhớ của tất cả các Worker Nodes, loại bỏ hoàn toàn giai đoạn Shuffle mạng của bảng lớn (500GB), chuyển Shuffle Hash Join thành Broadcast Hash Join với tốc độ nhanh gấp 10–50 lần.
- **Bài 14:** PySpark Window Functions:

```

1 from pyspark.sql.window import Window
2 import pyspark.sql.functions as F
3

```

```

4 w_running =
    Window.partitionBy("employee_id").orderBy("date").rowsBetween(Window.unboundedPreceding,
    Window.currentRow)
5 df = df.withColumn("running_total",
    F.sum("sale_amount").over(w_running))
6
7 w_rank =
    Window.partitionBy("department").orderBy(F.desc("sale_amount"))
8 df_top3 = df.withColumn("rank",
    F.dense_rank().over(w_rank)).filter("rank <= 3")
9

```

- **Bài 15:** Parquet chỉ là định dạng tệp lưu trữ dạng cột tĩnh không có transaction log. Delta Lake bổ sung tầng giao dịch ACID thông qua tệp nhật ký `_delta_log/`. Time travel: `SELECT * FROM delta_table VERSION AS OF 3` hoặc `TIMESTAMP AS OF '2026-03-20'`. Lệnh `VACUUM` dùng để dọn dẹp vật lý các tệp dữ liệu rác cũ đã bị đánh dấu xóa vượt quá thời gian lưu giữ (retention period) để tiết kiệm dung lượng lưu trữ.
- **Bài 16:** Để đảm bảo thứ tự sự kiện của cùng 1 user, Producer bắt buộc phải đặt `partition_key = user_id`. Kafka đảm bảo toàn bộ message có cùng key sẽ rơi vào đúng 1 partition duy nhất (nơi thứ tự được bảo toàn 100%). Consumer Lag là số lượng bản ghi đã được ghi vào Topic nhưng Consumer chưa kịp xử lý. Scale out: Tăng số lượng Consumer trong Consumer Group lên tối đa bằng số lượng Partitions (12 consumers).
- **Bài 17:** Ứng dụng FastAPI Serving chuẩn MLOps:

```

1 from contextlib import asynccontextmanager
2 from fastapi import FastAPI
3 import joblib
4
5 ml_models = {}
6 @asynccontextmanager
7 async def lifespan(app: FastAPI):
8     ml_models["churn"] = joblib.load("churn_model.joblib")
9     yield
10    ml_models.clear()
11
12 app = FastAPI(lifespan=lifespan)
13 @app.post("/predict")
14 def predict(payload: CustomerFeatures):
15     prob =
        ml_models["churn"].predict_proba([[payload.total_spend,
        payload.tenure]])[0][1]
16     return {"churn_prediction": int(prob >= 0.5),
        "churn_probability": float(prob)}
17

```

- **Bài 18:** Kịch bản kiểm thử tải Locust:

```

1 from locust import HttpUser, task, between
2
3 class MLUser(HttpUser):
4     wait_time = between(0.1, 0.5)
5     @task
6     def predict_churn(self):
7         payload = {"total_spend": 250.0, "tenure": 14}
8         self.client.post("/predict", json=payload)
9

```

Chạy lệnh: `locust -f locustfile.py --headless -u 200 -r 20 --run-time 1m`. Mục tiêu: Đảm bảo P99 latency < 50ms và 0% lỗi ở tải 500 RPS.

- **Bài 19:** Cấu hình GitHub Actions CI:

```

1 name: Data Pipeline CI
2 on: [pull_request]
3 jobs:
4   test:
5     runs-on: ubuntu-latest
6     steps:
7       - uses: actions/checkout@v3
8       - uses: actions/setup-python@v4
9         with: { python-version: '3.11' }
10      - run: pip install -r requirements-test.txt
11      - run: flake8 . --max-line-length=100
12      - run: pytest tests/ -v --cov=.
13

```

- **Bài 20:** Data Drift: Phân phối của dữ liệu đầu vào $P(X)$ thay đổi qua thời gian. Concept Drift: Mối quan hệ giữa dữ liệu đầu vào và nhãn mục tiêu $P(Y | X)$ thay đổi. Chỉ số PSI: $PSI = \sum (Actual\% - Baseline\%) \times \ln(Actual\%/Baseline\%)$. Quy tắc MLOps: Nếu $PSI < 0.1$: Dữ liệu ổn định, không thay đổi đáng kể; Nếu $0.1 \leq PSI < 0.25$: Bắt đầu có độ lệch nhẹ, cần theo dõi sát sao; Nếu $PSI \geq 0.25$: Độ lệch nghiêm trọng (Significant Drift), hệ thống MLOps phải tự động phát tín hiệu cảnh báo và kích hoạt pipeline huấn luyện lại mô hình (Model Retraining).

Phụ Lục: Checklist Năng Lực & Tổng Kết Hành Trình Data Pro

22.1 Checklist 35 Tiêu Chí Đánh Giá Năng Lực Trước Khi Ứng Tuyển

Đừng nộp hồ sơ dựa trên cảm tính. Hãy tự đánh giá bản thân một cách trung thực dựa trên bảng 35 tiêu chí kiểm định năng lực thực chiến dưới đây. Chỉ cần bạn đạt được **80% số tiêu chí của từng cấp độ**, bạn đã hoàn toàn sẵn sàng vượt qua các vòng phỏng vấn kỹ thuật khắt khe nhất:

STT	Tiêu Chí Kỹ Thuật Bắt Buộc	Đạt (✓)
TẦNG 1: NỀN TẢNG BẮT BUỘC (Tuần 1–14)		
1	Sử dụng thành thạo Git (Branch, Merge, PR, Gitignore bảo mật).	☐
2	Thao tác dòng lệnh Linux để phân tích tệp log nhanh (grep, awk, cut, sort, uniq).	☐
3	Phân biệt rõ sự khác nhau về cơ chế I/O giữa CSV, JSON và Parquet.	☐
4	Viết mã Pythonic: Sử dụng Comprehensions, Generators, Type Hints, và Decorator.	☐
5	Thành thạo NumPy Vectorization và các phép biến đổi ma trận thay thế vòng lặp for.	☐
6	Thành thạo Pandas: GroupBy Aggregations, Reshaping, Merge/Join, Datetime operations.	☐
7	Nắm vững thứ tự thực thi logic của câu lệnh SQL (FROM → WHERE → GROUP BY).	☐
8	Viết thành thạo các loại Window Functions (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD).	☐
9	Sử dụng thành thạo CTEs và Recursive CTE để xử lý dữ liệu cấu trúc cây.	☐
10	Hiểu cơ chế Index B-Tree, nguyên tắc Leftmost Prefix và đọc được EXPLAIN ANALYZE.	☐
TẦNG 2: SẴN SÀNG ĐI LÀM DATA ANALYST (Tuần 15–24)		
11	Phân biệt rõ Mean vs Median trên dữ liệu lệch và phát hiện Outliers qua IQR.	☐
12	Nắm vững Định lý Giới Hạn Trung Tâm (CLT) và ước lượng Khoảng Tin Cây 95%.	☐
13	Hiểu bản chất p-value, Sai lầm Loại I (α) và Sai lầm Loại II (β).	☐
14	Thiết kế được thử nghiệm A/B Testing: Tính Sample Size, MDE, và kiểm tra SRM.	☐
15	Thành thạo Excel nâng cao: XLOOKUP, Dynamic Arrays, và Power Query ETL.	☐
16	Thiết kế mô hình Star Schema chuẩn mực trong Power BI (Quan hệ 1 – *, Single Direction).	☐
17	Làm chủ hàm CALCULATE () trong DAX và hiểu cơ chế Context Transition.	☐
18	Viết thành thạo các hàm Time Intelligence DAX (YTD, YoY Growth %, Rolling Average).	☐
19	Xây dựng được Executive Dashboard 3 tầng phục vụ trực tiếp cho ban lãnh đạo.	☐
20	Có ít nhất 3 dự án portfolio hoàn chỉnh trên GitHub (CLI, EDA, SQL DB, Power BI).	☐
TẦNG 3: CHUYÊN SÂU DATA ENGINEER & DATA SCIENTIST (Tuần 25–58)		
21	Nắm vững phương pháp mô hình hóa chiều Kimball (Fact, Dimension, Surrogate Key).	☐
22	Thiết kế được bảng lịch sử biến đổi chậm SCD Type 2.	☐
23	Xây dựng Data Pipeline có tính lũy đẳng (Idempotency) và trích xuất tăng dần (Incremental).	☐
24	Sử dụng Pydantic để xác thực dữ liệu và ghi log hệ thống chuẩn sản xuất.	☐
25	Khởi chạy hệ sinh thái dữ liệu đa dịch vụ bằng Docker Compose.	☐
26	Viết được Apache Airflow DAG theo TaskFlow API với cơ chế Retry và Alerting.	☐
27	Xây dựng mô hình biến đổi dữ liệu đa tầng bằng dbt (Staging → Marts) và viết test tự động.	☐
28	Đóng gói toàn bộ tiền xử lý ML qua Scikit-Learn Pipeline ngăn chặn 100% Data Leakage.	☐
29	Đánh giá mô hình dựa trên Confusion Matrix, Precision-Recall và ROC-AUC thay vì Accuracy.	☐
30	Ứng dụng SHAP values để giải thích tác động của từng đặc trưng mô hình đến kinh doanh.	☐
31	Viết mã PySpark tối ưu hóa: Áp dụng Broadcast Hash Join và kiểm soát Partitioning.	☐
32	Viết được các Job xử lý dữ liệu theo định dạng Parquet, Delta Lake, và Time Travel.	☐

22.2 Lời Kết: Tinh Thần Của Một Kỹ Sư Dữ Liệu Chân Chính

Ngành dữ liệu là một cuộc chạy marathon, không phải một cuộc chạy nước rút. Không ai có thể trở thành chuyên gia chỉ sau một đêm. Tuy nhiên, nếu bạn kiên trì đi qua từng chương của cuốn sách này, gỡ từng dòng mã nguồn, tự tay sửa các lỗi phát sinh trong từng bài tập tự luyện và biến các dự án thành sản phẩm sống trên GitHub của riêng mình, bạn sẽ nhận ra rằng: ****Khoảng cách giữa bạn và những kỹ sư hàng đầu tại các tập đoàn công nghệ lớn thực chất chỉ là số giờ bạn đã thực sự ngồi xuống và giải quyết các bài toán dữ liệu thực tế.****

Hãy luôn giữ cho mình ngọn lửa tò mò về bản chất toán học của các thuật toán, tư duy kỷ luật của kỹ thuật phần mềm, và sự nhạy bén đối với các quyết định kinh doanh. Chúc bạn gặt hái thành công rực rỡ trên con đường chinh phục thế giới dữ liệu!